Gabriel Silveira de Azevedo
Mateus Carvalho Costa Nogueira

Captação e recomendação de potenciais doadores de sangue: desenvolvimento de uma RESTful API, usando Domain Driven Design (DDD)

Campos dos Goytacazes
2026

Gabriel Silveira de Azevedo
Mateus Carvalho Costa Nogueira

Captação e recomendação de potenciais doadores de sangue: desenvolvimento de uma RESTful API, usando Domain Driven Design (DDD)

Monografia apresentada ao Instituto Federal de Educação, Ciência e Tecnologia Fluminense como parte das exigências para a conclusão do Curso de Bacharelado em Sistemas de Informação.

Instituto Federal de Educação, Ciência e Tecnologia Fluminense
Campus Campos Centro
Curso de Bacharelado em Sistemas de Informação

Orientador: Mark Douglas Azevedo Jacyntho

Campos dos Goytacazes
2026

Gabriel Silveira de Azevedo
Mateus Carvalho Costa Nogueira

Captação e recomendação de potenciais doadores de sangue: desenvolvimento de uma RESTful API, usando Domain Driven Design (DDD)

Monografia apresentada ao Instituto Federal de Educação, Ciência e Tecnologia Fluminense como parte das exigências para a conclusão do Curso de Bacharelado em Sistemas de Informação.

Trabalho aprovado. Campos dos Goytacazes, _____ de _____ de 2026.

Mark Douglas Azevedo Jacyntho
Orientador

Membro Titular 1
Avaliador

Membro Titular 2
Avaliador

Lista de abreviaturas e siglas

API	Application Programming Interface
DDD	Domain Driven Design
FIFO	First In, First Out
JWT	JSON Web Token
LTS	Long Term Support
NoSQL	Not Only SQL
OMS	Organização Mundial da Saúde
OO	Orientação a Objetos
OPAS	Organização Pan-Americana da Saúde
REST	Representational State Transfer
SOLID	Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion
TIC	Tecnologias da Informação e Comunicação
UML	Unified Modeling Language

Lista de ilustrações

Figura 1 – Distribuição de artigos por ano 26
Figura 2 – Distribuição de artigos por país 26
Figura 3 – Diagrama de processos do desenvolvimento do TCC 31
Figura 4 – Diagrama de processos do Desenvolvimento da API 32
Figura 5 – Diagrama de caso de uso de acesso e perfis base 40
Figura 6 – Diagrama de caso de uso de requisitante 41
Figura 7 – Diagrama de caso de uso de doador 42
Figura 8 – Diagrama de análise dos participantes 44
Figura 9 – Diagrama de análise do processo de doação 45
Figura 10 – Diagrama de design: participantes, papéis e controle de acesso 46
Figura 11 – Diagrama de design: agregado DonationRequest 47
Figura 12 – Diagrama de design: agregado Donation 48
Figura 13 – Diagrama de sequência: DonationRequest 54
Figura 14 – Execução do fluxo de criação de solicitação de doação via cliente HTTP Insomnia 56
Figura 15 – Cenário ilustrativo de alocação contínua restrita por teto proporcional 68

Lista de tabelas

Tabela 1 – Classificação das palavras-chave 24
Tabela 2 – Expressão de busca usada no Scopus 25
Tabela 3 – Critérios de inclusão e exclusão 25
Tabela 4 – Lista de artigos selecionados 27
Tabela 5 – Requisitos funcionais 37
Tabela 6 – Requisitos não funcionais 38
Tabela 7 – Lista de casos de uso 39
Tabela 8 – Especificação dos casos de uso críticos 43
Tabela 9 – Entidades e suas responsabilidades 50
Tabela 10 – Autenticação e Acesso Público (Auth) 51
Tabela 11 – Participantes (Party) 51
Tabela 12 – Papéis e Perfis (Role) 52
Tabela 13 – Solicitações de Doação (Request) 52
Tabela 14 – Doações (Donation) 53
Tabela 15 – Composição consolidada da suíte por camada arquitetural 79
Tabela 16 – Distribuição e cobertura da suíte por contexto delimitado 80
Tabela 17 – Resultado consolidado da execução da suíte automatizada 83
Tabela 18 – Matriz exaustiva de testes automatizados da RESTful API Blood Match . . 94

Resumo

Transfusão sanguínea é essencial em emergências, cirurgias e tratamentos crônicos. No Brasil, contudo, a captação de doadores de sangue frequentemente permanece aquém da demanda. Na prática, pedidos *ad-hoc* circulam em redes sociais e as ferramentas atuais de divulgação se mostram incipientes, sem um mecanismo sistemático que aproxime requisitantes e doadores por critérios de compatibilidade, o que alonga o tempo para localizar candidatos adequados e reduz a conversão em doações de sangue efetivas.

Este trabalho tem como objetivo central desenvolver um sistema, especificamente uma RESTful API, voltado para solicitação e, posterior, captação e recomendação de potenciais doadores de sangue. Destacam-se como requisitos principais da proposta: a recomendação a potenciais doadores de solicitações compatíveis — considerando tipagem sanguínea e proximidade geográfica —, a notificação a doadores elegíveis e, sob o ponto de vista da Engenharia de Software, o emprego de uma arquitetura robusta, estruturada e escalável, segundo os princípios SOLID e o paradigma *Domain-Driven Design* (DDD).

O sistema foi desenvolvido sob uma abordagem qualitativa, exploratória-descritiva, iniciando com mapeamento sistemático da literatura, seguido pela modelagem cuidadosa do domínio de conhecimento de doação sanguínea, estabelecendo, pois, a linguagem ubíqua usada ao longo de todo ciclo de vida do projeto.

Como resultados, foram obtidos gerenciamento de solicitações e doações disponibilizados por meio da RESTful API criada, recomendação dirigida entre oferta e demanda e uma suíte de 323 testes automatizados executados com 100% de sucesso (atingindo 99,89% de cobertura de linhas e 95,80% de ramificações no núcleo do sistema). Após avaliados os resultados, percebeu-se que as regras de compatibilidade, elegibilidade, recomendação e alocação de doações às metas coincidem com o comportamento esperado, e que a organização arquitetural do software em camadas favorece a manutenibilidade da solução.

Palavras-chave: doação de sangue; RESTful API; Domain-Driven Design; recomendação de solicitações; engenharia de software.

Abstract

Blood transfusion is essential in emergencies, surgeries and chronic treatments. In Brazil, however, blood donor recruitment frequently falls short of demand. In practice, *ad-hoc* requests circulate on social media and current dissemination tools prove incipient, without a systematic mechanism that matches requesters and donors by compatibility criteria, which lengthens the time to locate suitable candidates and reduces conversion into actual blood donations.

This work aims to develop a system, specifically a RESTful API, for requesting and, subsequently, recruiting and recommending potential blood donors. The main requirements of the proposal are: the recommendation of compatible requests to potential donors — considering blood type and geographic proximity —, the notification of eligible donors, and, from a Software Engineering standpoint, the use of a robust, structured and scalable architecture, following the SOLID principles and the Domain-Driven Design (DDD) paradigm.

The system was developed under a qualitative, exploratory-descriptive approach, starting with a systematic literature mapping, followed by careful modeling of the blood donation knowledge domain, thereby establishing the ubiquitous language used throughout the entire project lifecycle. As results, the management of requests and donations was made available through the RESTful API, directed matching between supply and demand, and a suite of 323 automated tests executed with 100% success (achieving 99.89% line coverage and 95.80% branch coverage across system core components). After the results were evaluated, it was observed that the rules of compatibility, eligibility, recommendation and allocation of donations to request goals match the expected behavior, and that the layered software architectural organization favors the maintainability of the solution.

Keywords: blood donation; RESTful API; Domain-Driven Design; request recommendation; software engineering.

Sumário

1 INTRODUÇÃO 11
1.1 Problema e contexto 11
1.2 Objetivos 12
1.3 Justificativa 12
1.4 Organização da monografia 13

2 REFERENCIAL TEÓRICO 14
2.1 Orientação a Objetos 14
2.1.1 Abstração 14
2.1.2 Encapsulamento 15
2.1.3 Herança 15
2.1.4 Polimorfismo 16
2.2 Princípios SOLID 16
2.2.1 *Single Responsibility Principle* (SRP) 17
2.2.2 *Open/Closed Principle* (OCP) 17
2.2.3 *Liskov Substitution Principle* (LSP) 17
2.2.4 *Interface Segregation Principle* (ISP) 18
2.2.5 *Dependency Inversion Principle* (DIP) 18
2.3 *Domain-Driven Design* (DDD) 18
2.3.1 Padrões de *design* estratégico 19
2.3.2 Padrões de *design* tático 19
2.3.3 Arquitetura e isolamento do domínio 20
2.4 Lógica de Recomendação em Sistemas de Saúde 20
2.5 Controle de Concorrência em Sistemas Distribuídos 21

3 TRABALHOS RELACIONADOS 23
3.1 Protocolo de pesquisa 23
3.1.1 Objetivo e questões de pesquisa 23
3.1.2 Instrumentos 24
3.1.3 Critérios de seleção e procedimentos 24
3.2 Caracterização da pesquisa 25
3.3 Resultados e discussões 27
3.4 Considerações 28

4 MÉTODOS E RECURSOS 30
4.1 Visão geral 30

4.2 Instrumentos metodológicos da pesquisa . . . 32
4.2.1 Pesquisa exploratória-descritiva . . . 32
4.3 Recursos e Ferramentas Tecnológicas . . . 33
4.4 Procedimentos Metodológicos e Aplicação Prática . . . 34
4.4.1 Modelagem guiada pelo *Domain-Driven Design* (DDD) . . . 34
4.4.2 Evolução estrutural e persistência de dados . . . 34
4.4.3 Integração de testes automatizados e isolamento de regras . . . 35
4.4.4 Prototipação visual e documentação de *endpoints* . . . 35

5 RESTFUL API PROPOSTA: REQUISITOS, ANÁLISE, DESIGN E IMPLEMENTAÇÃO . . . 36
5.1 Visão geral da solução . . . 36
5.2 Levantamento de requisitos . . . 36
5.3 Modelagem do sistema . . . 38
5.3.1 Casos de uso . . . 38
5.3.1.1 Detalhamento dos casos de uso . . . 42
5.3.2 Diagramas de classes . . . 43
5.4 Domínio e arquitetura . . . 48
5.4.1 Linguagem ubíqua e organização em camadas . . . 48
5.4.2 Entidades, agregados e objetos de valor . . . 49
5.5 Endpoints e recursos da RESTful API . . . 50
5.6 Implementação e fluxo de dados . . . 53
5.6.1 Visão geral do fluxo de execução . . . 53
5.6.2 Camada de Apresentação: REST Controller e DTOs . . . 54
5.6.3 Camada de Aplicação: Casos de Uso e o Padrão *execute* . . . 56
5.6.4 Camada de Domínio: Agregado, Invariantes no Construtor e *Static Factory Method* . . . 58
5.6.5 Camada de Infraestrutura: Mapeamento de Esquemas e Persistência . . . 59
5.6.5.1 Consultas de Proximidade com Indexação Geoespacial . . . 60
5.6.5.2 Composição versus Herança na Arquitetura do Repositório . . . 61
5.6.6 Segurança e autenticação . . . 63
5.6.7 Serviços externos e documentação da API . . . 63
5.7 Recomendação de solicitações e cumprimento de metas . . . 64
5.7.1 Recomendação de solicitações para o doador . . . 64
5.7.2 Notificação de doadores potenciais . . . 65
5.7.3 Snapshot do cumprimento de metas . . . 67
5.8 Controle Otimista de Concorrência . . . 72
5.8.1 Caso de uso de atualização de solicitação . . . 73
5.8.2 Os dois momentos de checagem de versão . . . 74

6 AVALIAÇÃO DA PROPOSTA DE SOLUÇÃO 77
6.1 Sistema entregue 77
6.2 Plano e método de avaliação 78
6.3 Organização e estratégia da suíte de testes 78
6.3.1 Verificação formal das regras críticas de negócio 80
6.3.1.1 Cumprimento dinâmico de metas: teto proporcional e ordenação FIFO 80
6.3.1.2 Compatibilidade sanguínea e invariantes biológicas 81
6.3.1.3 Elegibilidade temporal, sanitária e motor de recomendação 82
6.3.1.4 Segurança, isolamento de participantes e autorização HTTP 82
6.4 Resultados da execução e análise de cobertura 83
6.5 Avaliação de manutenibilidade 84
6.6 Sugestões de melhorias 84
6.7 Considerações sobre a avaliação 85

7 CONSIDERAÇÕES FINAIS 86
7.1 Alcance dos objetivos 86
7.2 Principais contribuições 87
7.3 Limitações 88
7.4 Trabalhos futuros 88
7.5 Considerações finais 88

Referências 90

APÊNDICES 93

APÊNDICE A – MATRIZ DE COBERTURA DA SUÍTE DE TESTES AUTOMATIZADOS 94

1 Introdução

1.1 Problema e contexto

De acordo com a Organização Mundial da Saúde (2020) em colaboração com a Organização Pan-Americana da Saúde (2020), a taxa de doação de sangue por mil habitantes é de 32,6 em países de alta renda, 15,1 em países de renda média-alta, 8,1 em países de renda média-baixa e 4,4 em países de baixa renda. Ademais, a OMS (2020) estabelece que qualquer país deve ter no mínimo 1% da população doando sangue para suprir as necessidades básicas. Nesse cenário, a transfusão sanguínea permanece essencial à medicina contemporânea: é requerida em emergências oriundas de acidentes, em procedimentos cirúrgicos de grande porte e no tratamento de pacientes com condições patológicas crônicas que necessitam de transfusões regulares. As doações periódicas, portanto, sustentam os estoques e salvam vidas.

Apesar de sua importância, verifica-se que o sistema de doação de sangue no território brasileiro sofre com o baixo número de doadores diante de uma alta demanda. Conforme aponta o diretor do Hemorio, Luiz Amorim (2023), mesmo o Brasil estando dentro dos índices sugeridos pela OMS, o nível de sangue doado não é suficiente para suprir a demanda real. É comum observar, nas redes sociais, pessoas requisitando doações extraordinárias em virtude da necessidade de um conhecido — evidência da escassez nos bancos de sangue.

Do ponto de vista tecnológico, as soluções atuais de captação — pedidos disseminados em redes sociais e ferramentas de divulgação de alcance genérico — mostram-se extremamente limitadas na conexão entre oferta e demanda.

Falta um mecanismo sistemático que aproxime requisitantes e doadores segundo critérios de compatibilidade, o que resulta em tempo excessivo para localizar doadores adequados e em baixa taxa de conversão das solicitações em doações efetivas.

Diante dessa lacuna, este trabalho propõe uma RESTful API que intermedia as solicitações de unidades de saúde e pacientes e os doadores disponíveis, recomendando correspondências compatíveis de forma a promover a captação efetiva de possíveis doadores.

Essa abordagem apresenta a vantagem de tornar a captação mais efetiva, ao substituir a divulgação genérica por um encontro dirigido entre oferta e demanda. Do ponto de vista arquitetural, é fundamental reconhecer que sistemas de saúde e de doação de sangue operam sobre regras de negócio densas, mutáveis e de extrema criticidade. Diante dessa complexidade, o desenvolvimento pautado no *Domain-Driven Design* (DDD) (Evans, 2003) entra como a resposta técnica ideal para evitar a construção de um modelo de domínio anêmico[1]. A abordagem aproxima, de forma orgânica, o *software* da realidade clínica dos hemocentros através da Lin-

[1] Modelo de domínio anêmico é um antipadrão no qual as classes de domínio contêm apenas dados (atributos e *getters*/*setters*), sem comportamento ou regras de negócio, delegando toda a lógica a serviços externos e, assim, contrariando os princípios da orientação a objetos e do *Domain-Driven Design*.

guagem Ubíqua, garantindo que os termos do negócio e as restrições sanitárias sejam traduzidos fielmente para o código. Além disso, a adoção dos princípios SOLID (Martin, 2000) garante uma arquitetura robusta e escalável, promovendo futuras expansões e um longo ciclo de vida ao sistema, que estará preparado para receber integrações com facilidade.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver um sistema de informação para gerenciamento de doações de sangue, permitindo o registro de solicitações de unidade de saúde e pacientes e, posterior, captação e recomendação de potenciais doadores de sangue. O sistema deve recomendar solicitações a potenciais doadores à luz dos critérios de tipagem sanguínea e de proximidade geográfica, bem como notificar os doadores elegíveis.

Objetivos específicos:

O1 Definir a linguagem ubíqua do projeto estritamente alinhada ao processo de doação sanguínea, identificando participantes, papéis, solicitações e regras de negócio.

O2 Identificar sistemas com funcionalidades similares.

O3 Projetar e desenvolver o sistema com base nos padrões e preceitos do paradigma *Domain-Driven Design* (DDD) e dos princípios SOLID.

O4 Utilizar uma arquitetura de software que promova a clara separação das camadas do sistema (domínio, aplicação, REST, infraestrutura), bem como minimize o acoplamento dos componentes do sistema.

O5 Prover a interface de acesso ao *backend* do sistema seguindo o estilo arquitetural REST com o protocolo HTTP, disponibilizando, portanto, uma RESTful API de acesso.

O6 Recomendar solicitações compatíveis a doadores e notificar candidatos elegíveis.

1.3 Justificativa

A viabilidade e a relevância da proposta baseiam-se em evidências de que a sistematização tecnológica gera resultados expressivos. O estado do Ceará, por exemplo, destaca-se no cenário hemoterápico nacional, possuindo 2,21% da população doadora de sangue (Ceará. Secretaria da Saúde, 2021) — mais que o dobro do mínimo recomendado pela OMS —, estabelecendo-se como referência nacional. Esse desempenho está diretamente relacionado ao sistema integrado e unificado dos hemocentros do estado, coordenado pelo Centro de Hematologia e Hemoterapia do Ceará (Hemoce), que atende a demanda de transfusão sanguínea dos 184 municípios cearenses por meio da Hemorrede Pública Estadual, abrangendo mais de 480 unidades de saúde (Centro

de Hematologia e Hemoterapia do Ceará — HEMOCE, 2022). Em 2024, o estado alcançou um marco histórico com 110.784 doações, o maior número registrado em 30 anos (Ceará. Secretaria da Saúde, 2025), demonstrando que um sistema bem estruturado e integrado pode efetivamente suprir a demanda por sangue e derivados.

Ademais, no plano social e de saúde coletiva, a proposta alinha-se diretamente ao Objetivo de Desenvolvimento Sustentável 3 (ODS 3 — Saúde e Bem-Estar) da Agenda 2030 da Organização das Nações Unidas (Organização das Nações Unidas, 2015), cujo propósito inclui assegurar o acesso universal a serviços de saúde essenciais e a insumos terapêuticos vitais e seguros. Ao estruturar um canal computacional capaz de aproximar demandas emergenciais ou programadas de voluntários compatíveis, a solução busca reduzir o tempo de resposta e mitigar desabastecimentos nos bancos de sangue, apoiando intervenções cirúrgicas, atendimentos a traumas e o suporte continuado a pacientes crônicos.

1.4 Organização da monografia

O Capítulo 2 apresenta o referencial teórico (orientação a objetos, SOLID, DDD, lógica de recomendação e controle de concorrência otimista). O Capítulo 3 relata o mapeamento sistemático da literatura e os trabalhos relacionados. O Capítulo 4 descreve métodos, instrumentos e cronograma. O Capítulo 5 detalha a RESTful API proposta (requisitos, análise, design e implementação). O Capítulo 6 trata da avaliação. O Capítulo 7 reúne as considerações finais.

2 Referencial teórico

Este capítulo apresenta os pilares teóricos que alicerçam a arquitetura e o desenvolvimento do sistema de API proposto, sendo dividido em três tópicos complementares: primeiramente, os princípios da Orientação a Objetos (OO), que trazem a base conceitual sobre a qual todo o desenvolvimento é edificado; em seguida, são abordados os princípios SOLID, que fortalecem as diretrizes para o *design* das classes e módulos, tornando-os mais flexíveis e manuteníveis; e, por fim, é apresentado o *Domain-Driven Design* (DDD), que orienta o desenvolvimento e o entendimento do sistema de forma alinhada ao domínio do negócio. A decisão por apresentá-los nessa ordem não é aleatória: cada tema é tratado como uma camada que complementa a anterior, formando um conjunto de conhecimentos coeso e aplicável a domínios com regras de negócio complexas e relativamente estáveis, como é o caso da doação sanguínea.

2.1 Orientação a Objetos

A OO é um paradigma de programação que tem como objetivo organizar o código em torno de objetos, estruturas que encapsulam dados e comportamentos, simulando entidades e eventos do mundo real (Sommerville, 2011). Embora tenha surgido nos anos 1960 com a linguagem de programação Simula, a OO só ganhou força mais tarde, com linguagens como Smalltalk, C++ e Java, estabelecendo-se como paradigma fundamental no desenvolvimento de *software* (Booch, 2007). A disseminação desse paradigma na comunidade de desenvolvedores decorre de sua capacidade de modelar problemas do mundo real de forma intuitiva, aproximando a estrutura do código e a organização das classes da estrutura do problema que se pretende resolver (Sommerville, 2011).

A OO é estruturada em quatro pilares fundamentais e complementares: abstração, encapsulamento, herança e polimorfismo.

2.1.1 Abstração

A abstração permite que os programadores se preocupem apenas com as características essenciais de um objeto, ignorando detalhes desnecessários ao modelo e ao contexto em questão (Booch, 2007). Em vez de lidar com cada detalhe, desenvolvedores trabalham em níveis mais altos de abstração. Isso ocorre porque, segundo Sommerville (2011), a abstração ajuda a controlar a complexidade do *software*, mantendo apenas o essencial do domínio na visão dos programadores.

A definição de contratos entre tipos surge principalmente por meio de classes abstratas e interfaces, que estabelecem obrigações funcionais sem detalhar o processo interno (Gamma

et al., 1994). Implementações concretas surgem depois, moldadas conforme as particularidades de cada classe descendente.

Na prática deste projeto, essa abstração reflete-se na criação da entidade genérica de participante do sistema, reunindo características comuns a diferentes atores — como nome, contato e endereço — independentemente de se tratar de um doador, de um solicitante ou de uma instituição de saúde. Especializações dessa abstração assumem comportamentos e regras próprias de cada papel, sem que o restante da aplicação precise conhecer particularidades internas para operar em alto nível.

2.1.2 Encapsulamento

O encapsulamento é responsável por controlar o acesso aos dados internos de um objeto, exigindo que qualquer interação aconteça por meio de vias específicas, o que preserva a coerência do estado interno (Sommerville, 2011). Ainda segundo Booch (2007), esconder os detalhes de um objeto que não contribuem para suas características essenciais faz parte do conceito, de maneira que a interface pública e a implementação interna permaneçam separadas, permitindo alterações na implementação que não afetam o código cliente que utiliza a classe (Sommerville, 2011).

De maneira prática, o encapsulamento usa modificadores de acesso para definir quais atributos e métodos podem ser vistos ou utilizados externamente. Assim, os dados ficam protegidos, enquanto métodos específicos permitem interação controlada. De acordo com Bloch (2018), essa prática permite validações e garante a consistência do objeto.

Para o cenário hemoterápico, a aplicação desse princípio assegura a proteção direta de regras críticas relativas à elegibilidade do doador e à validade das solicitações de sangue: informações como tipo sanguíneo, data da solicitação e prazo limite não sofrem alterações arbitrárias externas, sendo modificadas exclusivamente por operações que validam a consistência do estado (como garantir que o prazo limite seja posterior à data de abertura).

2.1.3 Herança

O conceito de herança possibilita o reaproveitamento e a especialização de comportamentos por subclasses, sendo fundamental para a reutilização de código e o estabelecimento de relações hierárquicas entre tipos (Sommerville, 2011). Segundo Booch (2007), a herança funciona como o meio pelo qual uma classe herda métodos e propriedades de outra, concretizando vínculos do tipo “é-um” entre entidades. Dessa forma, surgem arranjos hierárquicos que organizam classes representando relacionamentos ontológicos do contexto do problema.

A herança pode ocorrer de duas principais maneiras: a herança simples, em que uma classe deriva de uma única superclasse, e a herança múltipla, em que uma classe herda de mais de uma superclasse, recebendo todos os atributos e métodos não privados das classes-base. Em ambos os casos é possível sobrescrever métodos para implementação específica (Booch,

2007). Sommerville (2011) destaca que a herança facilita a manutenção do sistema, uma vez que mudanças na superclasse são automaticamente propagadas para as subclasses.

Ao modelar os diferentes atores envolvidos, a herança permite derivar especializações (como pessoas físicas e instituições) a partir de uma estrutura comum de participante, concentrando atributos compartilhados na classe base e evitando a duplicação de regras cadastrais.

2.1.4 Polimorfismo

Formas distintas de um mesmo comportamento surgem quando objetos diversos utilizam uma interface idêntica, conforme descrito por Booch (2007). Segundo Sommerville (2011), o polimorfismo permite que métodos funcionem de formas diversas em diferentes níveis de abstração, característica essencial na programação orientada a objetos.

Existem dois tipos principais de polimorfismo: o polimorfismo de sobrecarga (*overloading*), em que operações distintas com o mesmo nome coexistem com assinaturas diferentes, e o polimorfismo de sobrescrita (*overriding*), em que subclasses redefinem métodos herdados da superclasse (Booch, 2007). O polimorfismo de subtipo, também conhecido como polimorfismo de inclusão, permite que uma referência de uma superclasse ou interface aponte para objetos de qualquer uma de suas subclasses, possibilitando que o sistema determine em tempo de execução qual implementação específica deve ser invocada (Sommerville, 2011). Essa característica reduz o acoplamento entre componentes e facilita a adição de novos tipos ao sistema sem modificar código existente. Para que o polimorfismo de subtipo funcione corretamente, é fundamental que as classes derivadas possam substituir suas classes mãe sem alterar o comportamento esperado do sistema, princípio conhecido como *Liskov Substitution Principle* (LSP), segundo o qual objetos de uma classe derivada devem poder ser usados no lugar de objetos da classe base sem comprometer a correção do programa (Martin, 2003).

Em uma API voltada à captação, o polimorfismo viabiliza tratar múltiplos perfis de uso por meio de interfaces unificadas, permitindo que a aplicação execute rotinas genéricas — tais como envio de notificações ou verificação de autorização — sem necessitar de condicionais explícitas sobre o tipo concreto de participante envolvido.

2.2 Princípios SOLID

Com origem na década de 2000, SOLID — acrônimo para *Single Responsibility*, *Open-Closed*, *Liskov Substitution*, *Interface Segregation* e *Dependency Inversion* — reúne cinco princípios fundamentais voltados ao *design* orientado a objetos em desenvolvimento de *software*, originalmente pensados por Robert C. Martin (2003). Posteriormente, em 2004, Michael Feathers organizou essas contribuições na abreviação que tornou o conjunto mais acessível pelo acrônimo SOLID.

Partindo dos conceitos definidos por Martin, surgem diretrizes capazes de tornar os sistemas mais estruturados, desacoplados conforme a necessidade e mais simples de manter (Martin, 2008). Embora nem sempre aplicado integralmente, segundo Martin (2017), o uso rigoroso dessas diretrizes faz com que o *software* cresça com maior controle, reduzindo consequências indesejadas ao modificar funcionalidades ou responder a novas demandas.

A fundação das diretrizes SOLID está ligada à maneira como elas lidam com falhas típicas no processo de criação de *softwares*, como código rígido, frágil ou pouco reaproveitável (Martin, 2003). Embora cada princípio trate de um aspecto distinto do projeto orientado a objetos, todos convergem para objetivos comuns: diminuir dependências entre partes distintas, fortalecer a coesão interna das classes e permitir expansões futuras sem alterar o que já está pronto. Em domínios com lógica de negócio complexa, como o da doação sanguínea, seguir tais princípios torna-se especialmente relevante.

Abaixo, os cinco princípios supracitados são detalhados um a um.

2.2.1 *Single Responsibility Principle* (SRP)

Este princípio estabelece que uma classe deve possuir apenas uma única razão para mudar, concentrando-se em uma responsabilidade coesa (Martin, 2003). Como destaca Booch (2007), a separação de responsabilidades é essencial para gerir a "complexidade organizada", garantindo que alterações em uma regra de negócio não propaguem erros inesperados em partes não relacionadas do sistema.

2.2.2 *Open/Closed Principle* (OCP)

Preconiza que as entidades de *software* (classes, módulos, funções) devem estar abertas para extensão, mas fechadas para modificação (Martin, 2003). O objetivo é permitir que o comportamento de um sistema seja estendido sem a necessidade de alterar o código-fonte original, o que é alcançado por meio do uso de abstrações e polimorfismo. Padrões de projeto como *Strategy* e *Decorator* são exemplificações práticas deste princípio (Gamma *et al.*, 1994).

2.2.3 *Liskov Substitution Principle* (LSP)

Formulado por Barbara Liskov, este princípio define que objetos de uma classe derivada devem ser capazes de substituir objetos da classe base sem comprometer a semântica do programa (Martin, 2003). A conformidade com o LSP garante que a hierarquia de herança seja logicamente consistente, assegurando que o polimorfismo de subtipo não resulte em comportamentos inesperados.

2.2.4 *Interface Segregation Principle* (ISP)

Orienta que os clientes não devem ser forçados a depender de interfaces que não utilizam (Martin, 2003). Em termos práticos, é preferível criar interfaces específicas e granulares em vez de uma interface única e sobrecarregada. Essa prática minimiza o impacto de mudanças e respeita o conceito de ocultação de informação e encapsulamento (Booch, 2007).

2.2.5 *Dependency Inversion Principle* (DIP)

Define que módulos de alto nível não devem depender de módulos de baixo nível, mas ambos devem depender de abstrações (Martin, 2003). O fundamento deste princípio é "programar para uma interface, não para uma implementação", reduzindo o acoplamento e facilitando a injeção de dependências, o que aumenta a flexibilidade e a testabilidade do sistema (Martin, 2008).

2.3 *Domain-Driven Design* (DDD)

Proposto originalmente por Evans (2003), o DDD — também conhecido como Projeto Orientado ao Domínio — é uma abordagem de desenvolvimento de *software* voltada para sistemas complexos cuja principal dificuldade reside na lógica de negócio. Por isso, tem como foco central o entendimento profundo das regras de negócio, estabelecendo que o *design* do *software* deve ser guiado por um modelo que reflita fielmente o domínio — o problema central que a aplicação visa resolver.

Segundo Vernon (2013), o DDD não deve ser encarado como uma tecnologia ou metodologia rígida, mas sim como um conjunto de ferramentas que permite lidar com a complexidade no coração do *software*, priorizando o que é estrategicamente mais importante para a organização.

A base fundamental do DDD está na colaboração estreita entre desenvolvedores e especialistas do domínio — aqueles que conhecem todas as nuances do negócio. Para que esse diálogo seja possível e produtivo, o DDD propõe a criação de uma Linguagem Ubíqua: um vocabulário comum e rigoroso, compartilhado por todos os membros da equipe, possibilitando um contato produtivo entre desenvolvedores, que a princípio não conhecem o domínio, e especialistas, que são a fonte de conhecimento para o desenvolvimento. Conforme Vernon (2016), essa linguagem não deve existir apenas em documentos ou diagramas, mas precisa ser falada nas reuniões do dia a dia e refletida diretamente no código-fonte, de modo que a intenção do negócio nunca se perca nas decisões técnicas. No domínio da doação sanguínea, essa linguagem se traduziria na adoção de termos como "doador", "solicitação de doação" e "tipo sanguíneo" diretamente no vocabulário técnico da equipe, nomes que qualquer especialista da área reconheceria imediatamente, sem necessidade de tradução.

Para organizar esse conhecimento e tornar a abordagem mais acessível na prática, autores como Vernon e Khan estruturam o DDD em três pilares fundamentais — e é justamente essa

divisão que orienta as seções seguintes.

2.3.1 Padrões de *design* estratégico

No DDD, o *design* estratégico fornece diretrizes para a organização do sistema em larga escala, permitindo a identificação de fronteiras claras e o alinhamento do *software* com os objetivos de negócio. O conceito central nesta fase é o Contexto Delimitado (*Bounded Context*), que atua como uma fronteira semântica dentro da qual um modelo específico é aplicável e consistente (Evans, 2003). A definição dessas fronteiras é essencial para evitar que o sistema degenere em uma estrutura técnica confusa e fortemente acoplada, frequentemente descrita na literatura como uma "Grande Bola de Lama" (*Big Ball of Mud*) (Vernon, 2013).

Para gerenciar a comunicação entre diferentes contextos, utiliza-se o Mapeamento de Contextos (*Context Mapping*), que define as relações técnicas e organizacionais entre as partes do sistema (Vernon, 2016). Dentre os padrões de mapeamento, destaca-se a Camada Anticorrupção (*Anticorruption Layer*), que funciona como um mecanismo defensivo para isolar o modelo de domínio principal de influências externas ou sistemas legados, preservando a integridade das regras internas (Evans, 2003).

No domínio da doação sanguínea, é comum delimitar ao menos dois contextos com preocupações distintas: um voltado à solicitação e à priorização de doações, centrado nas regras de compatibilidade sanguínea e de urgência, e outro voltado à gestão dos doadores, responsável pelas regras de elegibilidade, histórico de doações e comunicação com os candidatos. Uma separação dessa natureza favorece que alterações nas regras de um dos contextos — por exemplo, os critérios de elegibilidade de um doador — não impactem diretamente a lógica do outro.

2.3.2 Padrões de *design* tático

Enquanto o *design* estratégico lida com as fronteiras de larga escala, os padrões de *design* tático fornecem as ferramentas para a modelagem detalhada dentro de um Contexto Delimitado, garantindo que os objetos do sistema possuam comportamento rico e mantenham a consistência dos dados (Evans, 2003).

O primeiro bloco de construção são as Entidades: objetos definidos por uma identidade única que persiste ao longo do tempo, independentemente de mudanças em seus atributos (Vernon, 2013). No domínio da doação sanguínea, um solicitante ou um centro de coleta são exemplos de entidades — um solicitante continua sendo o mesmo solicitante mesmo que seu contato ou endereço seja atualizado, pois o que o identifica é sua identidade no domínio, não seus dados.

Em contrapartida, os Objetos de Valor (*Value Objects*) são imutáveis e definidos exclusivamente por seus atributos, sem identidade própria (Evans, 2003). Conceitos como tipo sanguíneo, endereço ou contato se enquadrariam nessa categoria: dois doadores com o mesmo tipo sanguíneo compartilham o mesmo valor, e não faz sentido distingui-los por identidade —

o que importa é o valor em si. Essa imutabilidade traz segurança ao modelo, pois garante que tais conceitos não sejam alterados de forma inadvertida.

Os Agregados representam agrupamentos de entidades e objetos de valor tratados como uma unidade para fins de consistência e mudança de dados. Cada agregado possui uma Raiz do Agregado (*Aggregate Root*), responsável por garantir que todas as regras de negócio e invariantes sejam satisfeitas antes de qualquer transação ser concluída (Vernon, 2013). No domínio da doação sanguínea, uma solicitação de doação — com seu ciclo de vida completo, desde a criação até o atendimento — ou o histórico de doações de um doador são exemplos plausíveis de agregados: nenhum elemento interno poderia ser manipulado diretamente por objetos externos, devendo toda interação passar obrigatoriamente pela raiz do agregado, o que preserva a integridade do modelo.

Por fim, os Eventos de Domínio registram ocorrências significativas para o negócio no passado, permitindo a comunicação desacoplada entre diferentes partes do sistema (Vernon, 2016). No contexto da doação sanguínea, a confirmação de uma doação ou a manifestação de interesse de um candidato são exemplos de eventos capazes de disparar notificações ou atualizar o estado de uma solicitação, sem que os componentes envolvidos precisem se conhecer diretamente.

2.3.3 Arquitetura e isolamento do domínio

Para que o modelo de domínio permaneça livre de contaminações tecnológicas, o DDD recomenda arquiteturas que isolem a lógica de negócio das preocupações de infraestrutura. Padrões como a Arquitetura em Camadas ou a Arquitetura de Portas e Adaptadores — também conhecida como Arquitetura Hexagonal — garantem que a camada de domínio permaneça pura e independente de detalhes técnicos, como bancos de dados ou interfaces de usuário (Evans, 2003). Esse isolamento facilita a manutenibilidade, a testabilidade e a evolução contínua do conhecimento sobre o negócio dentro do *software* (Vernon, 2016).

Em um sistema modelado sob essa perspectiva, esse isolamento se refletiria na separação clara entre as classes de domínio — responsáveis por expressar as regras de negócio da doação sanguínea — e os serviços responsáveis por coordenar operações de infraestrutura, como persistência e envio de notificações. Dessa forma, a camada de domínio permaneceria como uma abstração independente de tecnologia, sobre a qual o restante da aplicação é construído.

2.4 Lógica de Recomendação em Sistemas de Saúde

Sistemas de recomendação são ferramentas computacionais desenhadas para sugerir itens, ações ou correspondências que atendam a uma finalidade específica, baseando-se em algoritmos de filtragem. Tradicionalmente, distinguem-se abordagens como a filtragem heurística e colaborativa, que utilizam o comportamento histórico de múltiplos usuários para inferir

interesses, e a filtragem baseada em regras ou conteúdo (*Rule-based matching*), que avalia características predefinidas e fixas das entidades para calcular a elegibilidade (Ricci; Rokach; Shapira, 2015).

Em cenários comerciais, como no *e-commerce*, essas recomendações possuem caráter estritamente probabilístico: o objetivo é prever preferências subjetivas para maximizar a conversão de vendas, admitindo-se uma margem de erro sem consequências críticas. Contudo, a aplicação dessa lógica no domínio hemoterápico exige uma mudança de paradigma. A recomendação de doadores para uma solicitação de sangue não pode operar por probabilidade ou preferência de consumo; ela deve ser estritamente determinística e clínica, balizada pelas regulamentações sanitárias rigorosas vigentes, como a Portaria de Consolidação do Ministério da Saúde (Brasil. Ministério da Saúde, 2017). Um falso positivo na recomendação de um doador inapto compromete diretamente a segurança transfusional.

Por essa razão, a lógica de recomendação em hemocentros adota o modelo de filtragem baseada em regras absolutas aliado a um ranqueamento multicritério. O algoritmo atua como um funil: primeiramente, aplica filtros clínicos eliminatórios, avaliando a compatibilidade direta de antígenos do sistema ABO e do fator Rh. Em seguida, verifica-se o critério temporal, assegurando que o candidato respeitou a janela imunológica e o intervalo legal desde a sua última doação. Apenas os doadores validados nessas etapas avançam para a fase de recomendação ativa, na qual são ranqueados e priorizados com base em atributos como proximidade geográfica em relação ao polo receptor, proximidade da data-limite do pedido e identificação de tipagens em níveis críticos de estoque.

2.5 Controle de Concorrência em Sistemas Distribuídos

Em sistemas de informação baseados na Web, o gerenciamento de acessos simultâneos a dados compartilhados é um desafio inerente à arquitetura distribuída. Quando múltiplos clientes tentam modificar o mesmo recurso concomitantemente, o sistema fica sujeito a anomalias de transação. Exemplos clássicos dessas falhas ocorrem quando dois doadores tentam agendar a triagem para o mesmo horário (*Race Conditions*) ou quando dois operadores do hemocentro atualizam o *status* de uma mesma bolsa de sangue simultaneamente, fazendo com que o salvamento de um anule inadvertidamente a alteração do outro, fenômeno conhecido como atualizações perdidas (*Lost Updates*) (Kleppmann, 2017).

Para contornar esses cenários, os bancos de dados relacionais oferecem historicamente o Controle de Concorrência Pessimista (*Pessimistic Locking*), operacionalizado por comandos como o `SELECT FOR UPDATE`. Nessa abordagem, a primeira transação que acessa o registro adquire um bloqueio exclusivo no banco, impedindo que outras o leiam ou alterem até a finalização da operação. Apesar de garantir consistência absoluta, o bloqueio pessimista apresenta graves desvantagens em ambientes Web: ele onera as conexões do banco de dados, reduz drasticamente a taxa de transferência e o desempenho geral da aplicação, e aumenta a probabilidade

de *deadlocks* (impasses irreversíveis entre processos). Devido à natureza *stateless* das APIs RESTful modernas, segurar bloqueios de banco de dados por requisições HTTP prolongadas é uma prática desencorajada (Kleppmann, 2017).

Diante disso, arquiteturas Web escaláveis adotam o Controle de Concorrência Otimista (*Optimistic Concurrency Control* - OCC). O modelo otimista parte da premissa de que os conflitos de edição são raros e, portanto, permite a leitura irrestrita sem impor bloqueios à base de dados. Para garantir a integridade na escrita, o OCC adiciona um atributo de controle de versão à entidade (frequentemente mapeado via anotação `@Version` em *frameworks* ORM como o JPA/Hibernate, utilizando um número sequencial ou um *timestamp*).

No fluxo otimista, o cliente lê o registro e sua respectiva versão atual. No momento de submeter a atualização, o sistema verifica se a versão enviada pelo cliente ainda corresponde à versão persistida no banco. Caso outro usuário já tenha modificado aquele registro, a versão no banco terá sido incrementada. Nessa situação, a ferramenta de persistência recusa a operação e dispara uma exceção de conflito (como a `OptimisticLockException`). A API captura essa exceção e, de forma elegante, traduz o erro para uma resposta HTTP apropriada (tipicamente o código HTTP 409 *Conflict*), instruindo a aplicação cliente de que o recurso sofreu alterações recentes e necessita ser atualizado antes de uma nova submissão. Essa estratégia assegura a consistência referencial sem sacrificar a escalabilidade e a performance do sistema distribuído.

3 Trabalhos relacionados

Este capítulo apresenta o estudo de trabalhos relacionados, com o propósito de mapear o estado da arte referente a sistemas de apoio à doação de sangue. A investigação busca compreender como a literatura acadêmica tem abordado a captação de doadores e o uso de algoritmos de recomendação nesse cenário. Ao analisar as soluções tecnológicas já publicadas, torna-se possível identificar lacunas, justificar as escolhas arquiteturais adotadas nesta pesquisa e evidenciar a contribuição da API desenvolvida em comparação ao contexto tecnológico atual.

3.1 Protocolo de pesquisa

Para garantir o rigor científico na busca e seleção dos estudos, adotou-se o método de Mapeamento Sistemático da Literatura. Segundo Petersen *et al.* (2008), essa abordagem fornece uma estrutura objetiva para descobrir, categorizar e estruturar o conhecimento existente sobre um tópico específico dentro da engenharia de software. Este protocolo define os passos executados durante a pesquisa bibliográfica, assegurando que o processo seja replicável e livre de vieses de seleção. O enquadramento metodológico geral da pesquisa — abordagem qualitativa, caráter exploratório-descritivo e instrumentos — é apresentado no Capítulo 4. As subseções a seguir descrevem as questões norteadoras, as bases de dados consultadas e os parâmetros aplicados para a filtragem do material.

3.1.1 Objetivo e questões de pesquisa

O objetivo deste mapeamento sistemático da literatura é identificar, analisar e classificar as soluções tecnológicas existentes que apoiam a doação de sangue, com ênfase em sistemas que utilizam abordagens de recomendação de possíveis doadores. Desse modo, a principal questão de pesquisa que se pretende responder é:

1. Quais são os sistemas/aplicativos que auxiliam na captação de doadores de sangue?

Além disso, visa responder subquestões a partir dos trabalhos selecionados, tais como:

1.1) Há sistemas que utilizam algoritmos de recomendação para sugerir ações ou contatos entre doadores e instituições?

1.2) Quais são as principais funcionalidades e estratégias de incentivo implementadas em sistemas computacionais para promover a doação de sangue?

3.1.2 Instrumentos

Para identificar os trabalhos foi utilizado o Scopus. De acordo com Elsevier (2024), Scopus é um banco de dados de resumos e citações de literatura revisada por pares, que relatam resultados da pesquisa mundial em áreas como: ciência, tecnologia, medicina, artes e humanidades. O Scopus abrange mais de 2,4 bilhões de referências citadas desde 1970 (Elsevier, 2025).

3.1.3 Critérios de seleção e procedimentos

Foram realizadas buscas por referenciais bibliográficos, utilizando indexação de dados em bases científicas como CAPES Periódicos (http://www.periodicos.capes.gov.br/) e Scopus, com descritores como “doação”; “sangue”; “sistema”; “aplicação”; “captação” e “recomendação”. Os critérios de inclusão consideraram publicações entre os anos de 2015 e 2025, em português e inglês, limitando a área temática a medicina, engenharia e ciência da computação, com foco em artigo, revisão e artigos de conferência que apresentassem fundamentação teórica sólida e aplicabilidade prática. A expressão de busca utilizada no Scopus foi definida utilizando a técnica PICO (*Population*, *Intervention*, *Comparison*, *Outcome*). Garrard (2020) recomenda a estruturação de buscas de revisões sistemáticas da literatura a partir de facetas bem definidas para obter respostas precisas. A Tabela 1 apresenta a distribuição das palavras-chave da questão de pesquisa nas facetas PICO.

Tabela 1 – Classificação das palavras-chave

Questão	**P**	**I**	**C**	**O**	**C**
a.1) Quais são os sistemas/aplicativos que auxiliam na captação de doadores de sangue?	doador* OR donor*	“sangue” OR “blood”	—	“sistema” OR “system” OR “app” OR “application”	“captação” OR “recruitment” OR recomend*

Fonte: Elaborado pelos próprios autores (2025).

Já A Tabela 2 representa a *query* (expressão de busca) que foi utilizada para a busca dos trabalhos relacionados na base de dados do Scopus.

Tabela 2 – Expressão de busca usada no Scopus

```
( TITLE-ABS-KEY ( doador* OR donor* ) AND TITLE-ABS-KEY ( sangue OR
blood ) AND TITLE-ABS-KEY ( Sistema OR system OR app OR application
) AND TITLE-ABS-KEY ( Captação OR recruitment OR recomend* ) ) AND
PUBYEAR > 2014 AND PUBYEAR < 2026 AND ( LIMIT-TO ( SUBJAREA , "COMP")
OR LIMIT-TO ( SUBJAREA , "ENGI") OR LIMIT-TO ( SUBJAREA , "MEDI")
) AND ( LIMIT-TO ( LANGUAGE , "English") OR LIMIT-TO ( LANGUAGE ,
"Portuguese") ) AND ( LIMIT-TO ( DOCTYPE , "ar") OR LIMIT-TO ( DOCTYPE
, "re") OR LIMIT-TO ( DOCTYPE , "cp") )
```

Fonte: Elaborado pelos próprios autores (2025).

Os critérios de inclusão (CI) e exclusão (CE) utilizados para selecionar os artigos são apresentados nA Tabela 3.

Tabela 3 – Critérios de inclusão e exclusão

ID	Critério
CI1	Texto completo disponível em PDF ou HTML via acesso institucional.
CI2	O artigo contém informação relevante para responder às questões postuladas.
CE1	Artigo duplicado.
CE2	O artigo não contém informação relevante para responder às questões postuladas.

Fonte: Elaborado pelos próprios autores (2025).

Na primeira etapa foi verificada a disponibilidade de texto completo, a partir do download via Scopus. Depois, a seleção dos artigos com texto completo envolveu a interpretação do título e resumo para verificar se atende ao critério CI2. Simultaneamente, aplicaram-se os critérios de exclusão (CE1 e CE2) de forma rigorosa, descartando as publicações que não se alinhavam ao escopo tecnológico da pesquisa ou que apresentavam duplicidade na base de dados.

3.2 Caracterização da pesquisa

A pesquisa foi realizada em julho de 2025 e retornou 147 documentos. A Figura 1 apresenta a distribuição de artigos por ano. Observa-se que a maioria dos artigos encontrados são dos anos de 2020 e 2021.

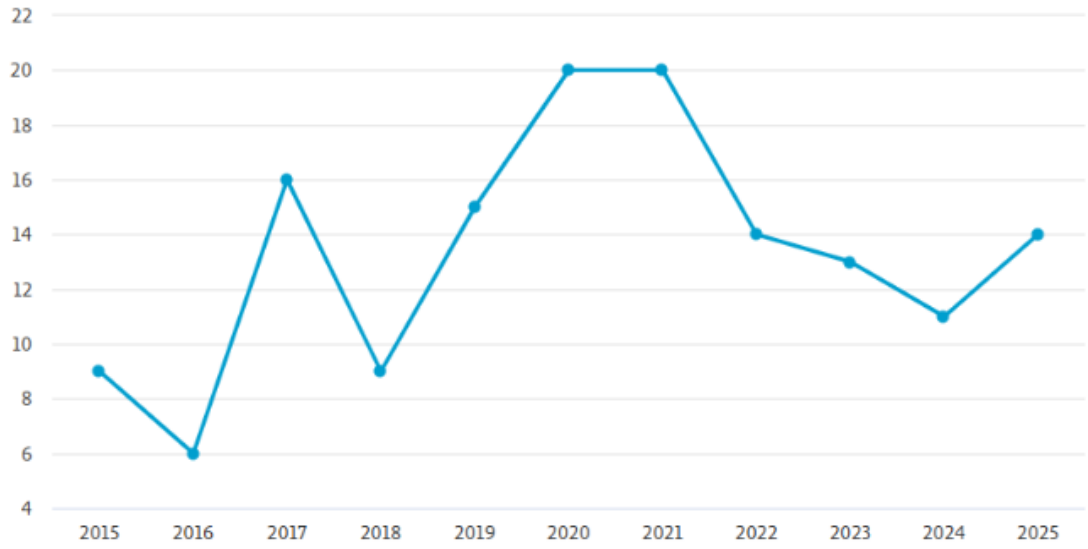


Figura 1 – Distribuição de artigos por ano

Fonte: Scopus (2025).

A Figura 2 apresenta a distribuição de artigos por país, cujo país com maior quantidade de publicações é os Estados Unidos.

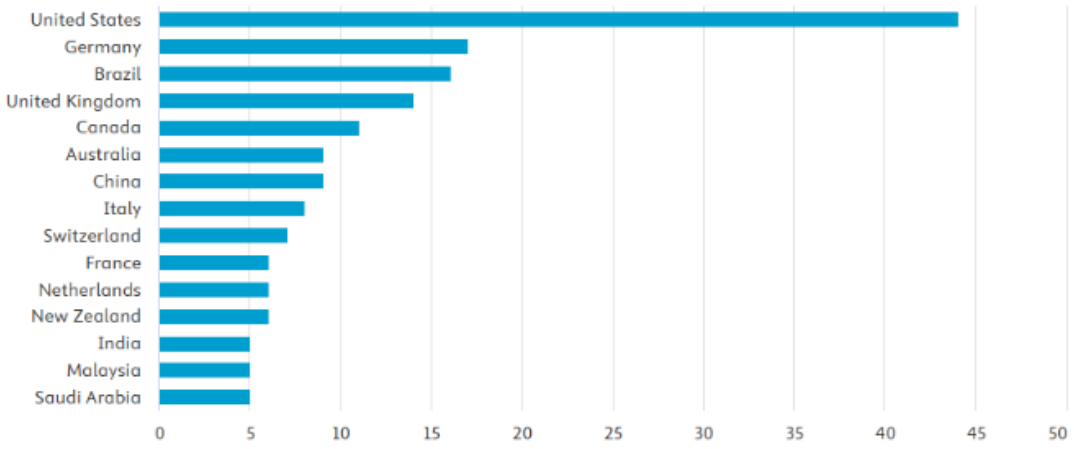


Figura 2 – Distribuição de artigos por país

Fonte: Scopus (2025).

Após a seleção, os materiais foram organizados por categorias temáticas e analisados com base em sua contribuição para a construção do referencial teórico e para o delineamento metodológico da pesquisa. Desse modo, foi utilizada a estratégia da análise de títulos e resumos dos artigos selecionados, para que os resultados possam ser auditáveis, replicáveis e úteis no suporte a pesquisas relacionadas a sistemas de captação de doadores de sangue, com o fim de que hospitais e hemocentros tenham estoques suficientes para atender às necessidades de pacientes que precisam de transfusões sanguíneas em diversas situações.

Sendo assim, a lista de artigos selecionados é apresentada nA Tabela 4.

Tabela 4 – Lista de artigos selecionados

Título	**Fonte**
Blood donation support application: Contributions from experts on the tool's functionality	Da Silva; Praça Brasil; De Vasconcelos Filho; Brasil; Paiva; De Oliveira; Dos Santos (2021)
Effective methods for reactivating inactive blood donors: A stratified randomised controlled study	Ou-Yang; Bei; Liang; He; Chen; Fu (2020)
Evaluation of the Wateen App in the Blood-Donation Process in Saudi Arabia	Alessa (2022)
Mobile applications for encouraging blood donation: A systematic review and case study	Li; Valero; Keyser; Ukuku; Zheng (2023)

Fonte: Elaborado pelos próprios autores (2025).

3.3 Resultados e discussões

A análise dos artigos selecionados permitiu identificar um conjunto relevante de soluções tecnológicas voltadas para a captação, o incentivo e a retenção de doadores de sangue, respondendo diretamente aos questionamentos levantados no protocolo de pesquisa. O levantamento revelou uma predominância de aplicativos móveis dedicados a facilitar a interação entre a população e os centros de saúde. Entre as plataformas destacadas na literatura, evidencia-se o Do-eSangue, desenvolvido com o intuito de apoiar a captação e fidelização ao integrar-se diretamente com a base de dados dos hemocentros, permitindo agendamentos e o acesso a informações personalizadas (Da Silva *et al.*, 2021). No cenário internacional, o Wateen, iniciativa do Ministério da Saúde da Arábia Saudita, atua de forma semelhante no recrutamento e agendamento via dispositivos móveis (Alessa, 2022). Outras iniciativas relevantes incluem o Blood Hero, que explora técnicas de incentivo social para alavancar os estoques (Li *et al.*, 2023), e o Blood Note, focado em promover a doação por meio de testes preliminares de elegibilidade realizados diretamente pelo celular do usuário (Li *et al.*, 2023).

No que tange à aplicação de algoritmos de recomendação ou mecanismos inteligentes de sugestão de ações, os estudos apontam para o uso de filtros dinâmicos e cruzamento de dados como alternativas eficientes de conexão. O aplicativo Do-eSangue, por exemplo, realiza o disparo de convites segmentados com base no tipo sanguíneo, na localização geográfica e na necessidade de fenótipos raros. Embora o estudo não utilize a nomenclatura estrita de "algoritmos de recomendação", essa filtragem avançada e baseada em critérios específicos de demanda atua, na prática, como uma recomendação direcionada de doadores (Da Silva *et al.*, 2021). Paralelamente, o Wateen demonstra um mecanismo ágil de correspondência ao notificar

usuários específicos quando há demanda hospitalar por sua tipagem sanguínea, permitindo ainda que o próprio doador compartilhe a solicitação com sua rede de contatos, criando um fluxo orgânico e eficiente de recomendações (Alessa, 2022).

Além da infraestrutura técnica de conexão, os trabalhos mapeados evidenciam diversas funcionalidades e estratégias de incentivo essenciais para a promoção contínua da doação. A comunicação personalizada e direcionada surge como um fator crítico nesse cenário. Ou-Yang *et al.* (2020) demonstraram que, para reativar doadores inativos, o contato direto via chamadas telefônicas apresenta resultados significativamente superiores ao uso de SMS, pois viabiliza um diálogo individualizado capaz de resolver barreiras pessoais e motivar o retorno. Aliado a isso, o uso estratégico de redes sociais e a disponibilização de informações abrangentes integradas aos sistemas ajudam a combater mitos, esclarecer dúvidas frequentes e detalhar os critérios de elegibilidade, aumentando a confiança da população nos serviços de saúde.

Outro ponto de forte convergência entre as pesquisas é a adoção da gamificação e de recursos de fidelização. A implementação de sistemas de pontuação, medalhas virtuais (*badges*) e programas de fidelidade que garantem preferência no atendimento servem como um forte apelo de reconhecimento social do voluntário, incentivando a repetição do ato solidário (Li *et al.*, 2023).

Por fim, a literatura destaca que a otimização da experiência prática do doador é indispensável. Funcionalidades como o agendamento simplificado reduzem drasticamente o tempo de espera e as filas nos hemocentros, fatores que impactam diretamente na satisfação geral do usuário (Alessa, 2022). Quando essas plataformas permitem que o voluntário acompanhe seu histórico de doações e acesse laudos de exames de forma unificada, consolida-se um sentimento de propósito e cuidado com a própria saúde, evidenciando o potencial da tecnologia como ferramenta de transformação na hemoterapia.

3.4 Considerações

O mapeamento sistemático da literatura apresentado neste capítulo permitiu traçar um panorama claro sobre o uso da tecnologia como aliada na captação e retenção de doadores de sangue. Os resultados evidenciam que a adoção de sistemas computacionais — especialmente aplicativos móveis — deixou de ser uma inovação distante para se tornar uma necessidade estratégica na gestão hemoterápica. Funcionalidades como o envio de notificações segmentadas, o agendamento prévio e a aplicação de técnicas de gamificação provaram ser eficazes para engajar a população e reduzir os índices de desabastecimento nos estoques.

Apesar das contribuições valiosas encontradas na literatura, nota-se que grande parte dos estudos concentra sua análise nas interfaces com o usuário final e nas estratégias de marketing social, abordando de forma superficial a engenharia de software e a arquitetura estrutural que sustentam essas soluções. Existe uma lacuna no detalhamento de como as complexas regras de

negócio desse domínio — que envolvem critérios médicos de elegibilidade, tipagem sanguínea e urgência hospitalar — são modeladas e processadas nos bastidores das aplicações.

Nesse sentido, os achados desta revisão validam a proposta central deste Trabalho de Conclusão de Curso. Ao constatar que a filtragem e a recomendação direcionada são os motores do sucesso das ferramentas atuais, reforça-se a relevância de uma RESTful API especializada, estruturada sobre DDD e princípios SOLID conforme discutido no Capítulo 2. O Capítulo 5 apresenta o estudo de caso, detalhando como os requisitos, a modelagem do domínio e a implementação da solução tratam, na prática, as lacunas de engenharia de software identificadas nesta revisão.

4 Métodos e recursos

Neste capítulo são descritos os procedimentos metodológicos adotados para o alcance dos objetivos propostos, abordando a abordagem de pesquisa escolhida, os instrumentos utilizados na coleta e análise de informações, o inventário das ferramentas tecnológicas empregadas e a aplicação prática dessas tecnologias no desenvolvimento da API.

4.1 Visão geral

Para o desenvolvimento da pesquisa adotou-se a abordagem qualitativa, que se destaca por enfatizar a análise de dados não numéricos, como documentos, normativas e literatura especializada, buscando compreender o significado e a complexidade dos fenômenos investigados.

> A pesquisa qualitativa trabalha com o universo de significados, motivos, aspirações, crenças, valores e atitudes, o que corresponde a um espaço mais profundo das relações, dos processos e dos fenômenos que não pode ser reduzido à operacionalização de variáveis. (Minayo, 2001, p. 21)

Para a condução do trabalho, foram traçadas etapas sequenciais, ilustradas na Figura 3 e na Figura 4.

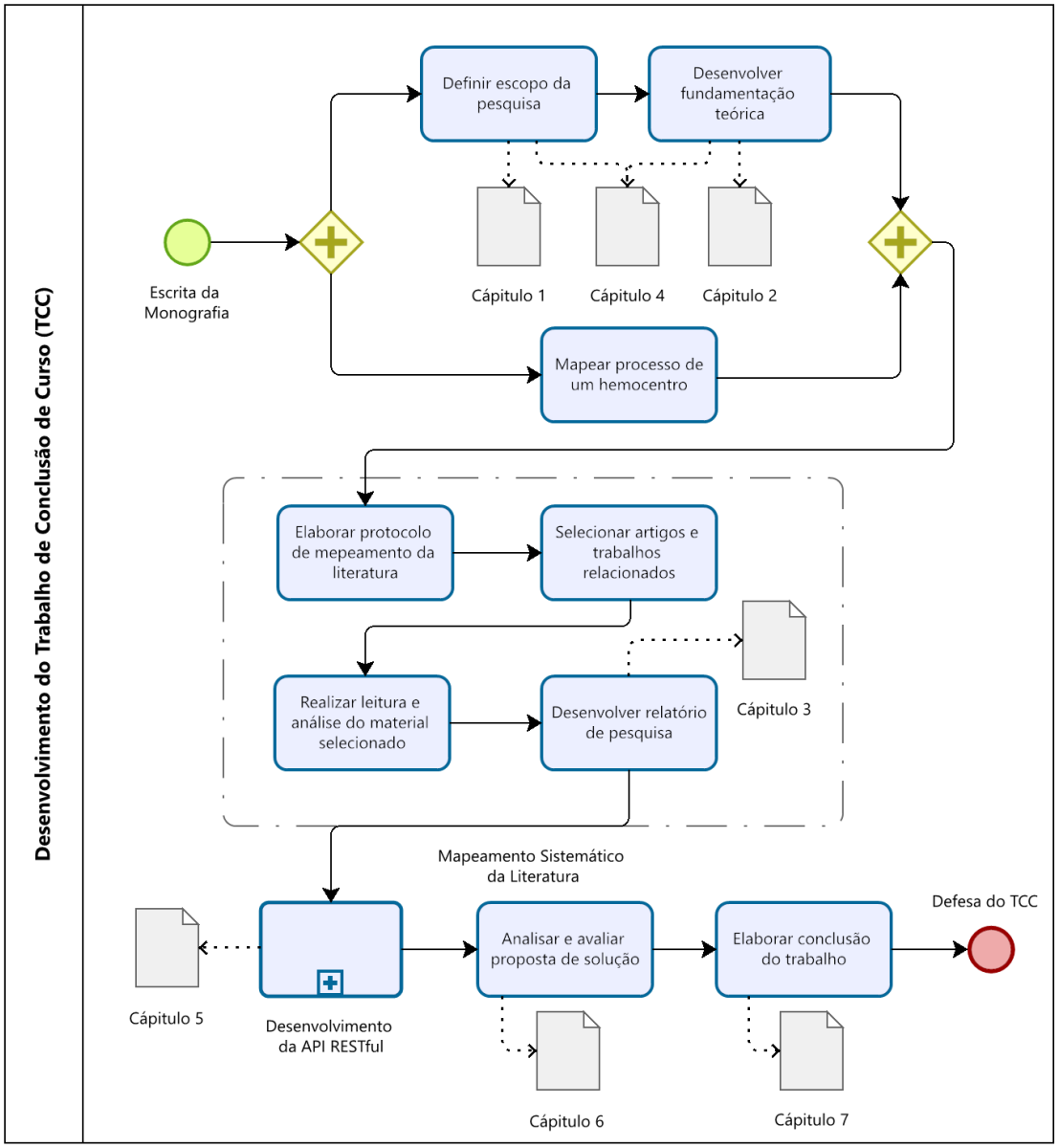


Figura 3 – Diagrama de processos do desenvolvimento do TCC

Fonte: Elaborado pelos próprios autores (2025).

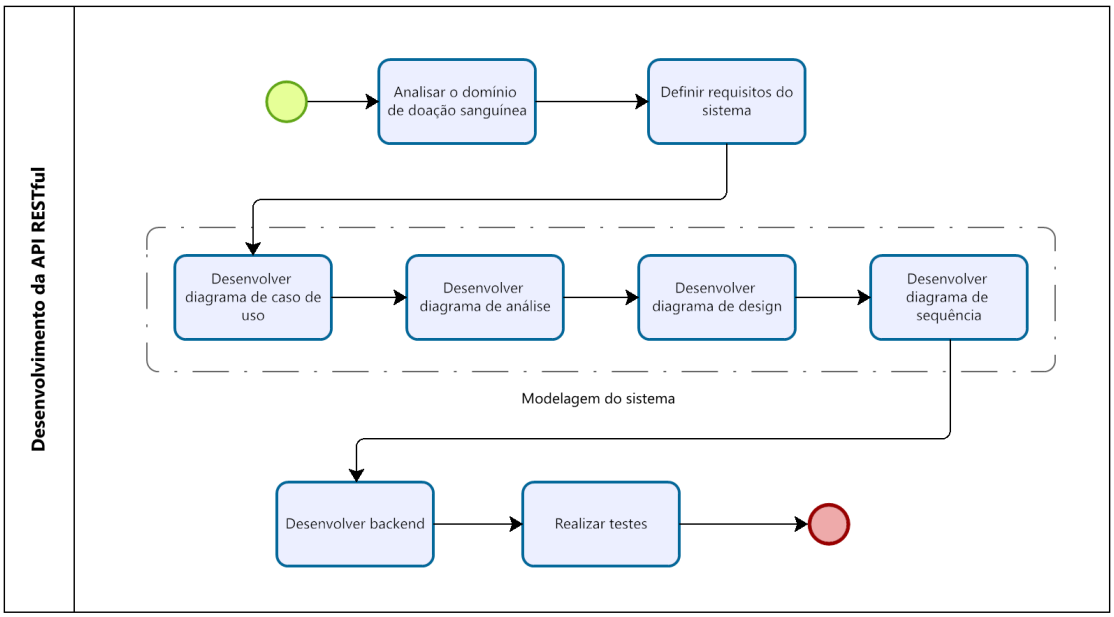


Figura 4 – Diagrama de processos do Desenvolvimento da API

Fonte: Elaborado pelos próprios autores (2025).

Inicialmente, definiu-se o escopo da pesquisa, o problema, as justificativas e os objetivos que fundamentam o Capítulo 1. Em seguida, conduziu-se o levantamento bibliográfico sobre OO, SOLID e DDD, aprofundados no Capítulo 2.

Paralelamente, realizou-se o mapeamento do processo de um hemocentro por meio de pesquisa documental e bibliográfica, buscando compreender o fluxo de trabalho, as regras de negócio e as necessidades do domínio da doação de sangue. Elaborou-se também um protocolo de mapeamento sistemático da literatura, cujos resultados são discutidos no Capítulo 3.

Com base no conhecimento adquirido nessas etapas, realizou-se a análise do domínio sob a ótica do DDD e a definição dos requisitos funcionais e não funcionais. Na sequência, elaborou-se o projeto da solução e desenvolveu-se a API, culminando na avaliação estrutural e funcional validada por testes automatizados.

4.2 Instrumentos metodológicos da pesquisa

Para a coleta e análise dos dados, foram utilizados instrumentos compatíveis com os objetivos e a abordagem metodológica da pesquisa, orientados pela necessidade de obter informações confiáveis e contextualizadas sobre o cenário da doação de sangue.

4.2.1 Pesquisa exploratória-descritiva

A presente pesquisa caracteriza-se como exploratória-descritiva, abordagem que, segundo Gil (2008), proporciona maior familiaridade com o problema investigado e, simulta-

neamente, descreve as características do fenômeno estudado. Segundo Prodanov e Freitas (2013), essa combinação metodológica é especialmente adequada para estudos que envolvem questões sociais e de saúde pública, permitindo uma análise contextualizada e orientada para a intervenção.

O contexto de análise adotado foi o das normativas e do cenário de atuação dos hemocentros, com atenção às demandas da região de Campos dos Goytacazes, local de moradia e estudo dos pesquisadores. Essa escolha favoreceu maior familiaridade com os aspectos sociais, culturais e institucionais que envolvem a temática, além de facilitar o acesso a dados públicos e normativas das unidades hospitalares e hemocentros locais.

Segundo Pressman e Maxim (2016), o esforço inicial de compreensão do domínio e das necessidades reais dos *stakeholders* é indispensável para criar uma base sólida de requisitos e evitar o desenvolvimento de sistemas inadequados à realidade operacional. Nesse sentido, utilizou-se a análise documental e bibliográfica como técnica de coleta de dados, por permitir profundidade na obtenção de informações normativas e procedimentais a partir de manuais do Ministério da Saúde, portarias vigentes e literatura acadêmica.

Segundo Marconi e Lakatos (2003), a pesquisa documental é composta por etapas fundamentais — como a seleção, leitura crítica e categorização dos dados — que garantem a sistematização e a confiabilidade das informações obtidas. Sommerville (2011) reforça que a análise de documentos atua como instrumento de captura e construção do conhecimento, sendo essencial para a engenharia de requisitos em sistemas críticos, como os voltados à saúde pública. O levantamento realizado a partir de documentos oficiais e estudos de caso teve como objetivo compreender os fatores que influenciam a decisão de doar sangue e subsidiar os requisitos da API proposta.

4.3 Recursos e Ferramentas Tecnológicas

Para o desenvolvimento da RESTful API, estabeleceu-se um ecossistema tecnológico fundamentado em ferramentas consolidadas no mercado corporativo, selecionadas por sua robustez, suporte a padrões arquiteturais e vasta documentação comunitária. A linguagem base adotada foi o Java, em sua versão 21 (*Long Term Support* — LTS), escolhida por sua forte tipagem estática e pelo suporte a recursos modernos como *Records*, ideais para a construção de Objetos de Valor e DTOs imutáveis (Deitel; Deitel, 2017). Em conjunto com a linguagem, empregou-se o *framework* Spring Boot 3, que atuou como o contêiner de inversão de controle, simplificando significativamente a configuração de servidores *web* embutidos e a orquestração de dependências do projeto (Walls, 2019).

No que tange ao armazenamento, o mecanismo de persistência selecionado foi o banco de dados não relacional (NoSQL) orientado a documentos MongoDB, justificado por sua flexibilidade na representação estrutural de agregados complexos, suporte nativo a índices geoespaciais e controle de concorrência otimista. Dada a flexibilidade do MongoDB em não exigir um

esquema rígido (*schema-less*), a evolução estrutural e o mapeamento dos documentos foram gerenciados de forma declarativa pela própria camada de aplicação, utilizando os recursos do Spring Data MongoDB.

Para assegurar a qualidade do *software*, adotaram-se as bibliotecas JUnit 5 e Mockito como peças fundamentais. O JUnit 5 forneceu o motor principal de execução para a suíte de testes, enquanto o Mockito viabilizou a criação de objetos simulados (*mocks*), permitindo testar as regras de domínio em total isolamento da infraestrutura externa. A interface de comunicação da aplicação, por sua vez, foi mapeada e descrita utilizando as especificações OpenAPI associadas ao Swagger, ferramentas que expuseram os contratos (*endpoints*) da RESTful API de forma padronizada e visual. Por fim, o processo de codificação foi conduzido no Ambiente de Desenvolvimento Integrado (IDE) Visual Studio Code, operando em sincronia com o Git, que gerenciou o histórico e o controle distribuído do código-fonte, mitigando perdas e viabilizando a paralelização do trabalho (Chacon; Straub, 2014).

4.4 Procedimentos Metodológicos e Aplicação Prática

Esta seção descreve os processos operacionais estabelecidos para traduzir os requisitos teóricos em código funcional, conectando as ferramentas elencadas na Seção 4.3 às diretrizes metodológicas do projeto.

4.4.1 Modelagem guiada pelo *Domain-Driven Design* (DDD)

O desenvolvimento não teve início direto na codificação de banco de dados ou de telas, mas sim na destilação do conhecimento do domínio hemoterápico. Sob a ótica do *design* estratégico do DDD, as discussões iniciais de levantamento de requisitos conduziram ao mapeamento dos contextos delimitados (*Bounded Contexts*). Identificou-se a separação funcional entre o núcleo de captação e ranqueamento das doações e as fronteiras auxiliares de perfis e autenticação.

No aspecto do *design* tático, o mapeamento de agregados foi o procedimento central. As regras biológicas extraídas dos manuais da área da saúde guiaram a criação de classes de Domínio (*Entities* e *Value Objects*). O fluxo de vida de uma solicitação de doação foi estritamente encapsulado sob a Raiz do Agregado `DonationRequest`. Para assegurar a Linguagem Ubíqua, todos os pacotes, classes e métodos foram nomeados utilizando termos diretamente mapeados do domínio (ex: `BloodType`, `Requester`, `EligibilityRule`).

4.4.2 Evolução estrutural e persistência de dados

Com o domínio estabelecido, a persistência foi aplicada respeitando as características de um banco de dados orientado a documentos. Diferentemente de bancos relacionais que exigem ferramentas de versionamento de esquema estrito, o processo operacional de evolução do banco

no MongoDB ocorreu de forma fluida por meio do mapeamento objeto-documento (*Object-Document Mapping* — ODM) provido pelo Spring Data MongoDB. As coleções e os índices de performance — com destaque para o índice geoespacial `2dsphere`, crítico para o cálculo de proximidade dos doadores — foram declarados via código e validados na inicialização da aplicação, assegurando que as regras de persistência refletissem o estado dos agregados de forma ágil e alinhada ao paradigma NoSQL.

4.4.3 Integração de testes automatizados e isolamento de regras

A cultura de qualidade foi incorporada ao método de trabalho desde as etapas primárias. A estratégia operacional consistiu em validar exaustivamente a camada de domínio por meio do JUnit 5, isolando os testes das camadas de entrada (controladores REST) e de saída (bancos de dados).

Para validar fluxos de serviços (Casos de Uso) sem acessar a infraestrutura real, o Mockito foi empregado no procedimento de injetar dependências simuladas, mimetizando repositórios de banco de dados. Esse isolamento permitiu que a equipe refatorasse lógicas complexas — como a validação de compatibilidade sanguínea e o cálculo do teto proporcional de bolsas — com a garantia de que regressões seriam detectadas em milissegundos pela suíte de testes.

4.4.4 Prototipação visual e documentação de *endpoints*

Por se tratar de uma API de *backend*, sem interface de usuário final desenvolvida na pesquisa, o procedimento de documentação das rotas assumiu um papel crítico. Operacionalmente, as classes controladoras da aplicação foram decoradas com anotações do padrão OpenAPI. Conforme as rotas eram desenvolvidas (ex: rotas `POST` para criação de solicitações e `GET` para matching de doadores), a biblioteca do Swagger operava a leitura destas assinaturas. O resultado prático foi a disponibilização automática de uma interface gráfica interativa (Swagger UI), capaz de atuar simultaneamente como contrato documentado para futuras aplicações cliente (*front-end*) e como ambiente de testes ad hoc durante o ciclo de desenvolvimento.

5 RESTful API proposta: requisitos, análise, design e implementação

5.1 Visão geral da solução

Este capítulo apresenta a solução tecnológica desenvolvida para apoiar o processo de captação e gerenciamento de solicitações de doação de sangue. A proposta materializa-se na construção de uma RESTful API focada exclusivamente na camada de *backend*, responsável pelo gerenciamento de participantes (pessoas e organizações), solicitações de doação, registro do histórico de doadores e recomendação de solicitações compatíveis.

O desenvolvimento foi orientado pelos princípios do DDD e pelos princípios SOLID, já fundamentados no Capítulo 2. Essa abordagem permitiu que a estrutura do software refletisse a realidade do domínio da hemoterapia, reduzindo o acoplamento entre os componentes e facilitando a evolução contínua do sistema. As seções seguintes detalham o levantamento de requisitos, a modelagem do sistema, a organização do domínio e da arquitetura, a implementação técnica e, por fim, a estratégia de recomendação e de cumprimento de metas, que constitui o núcleo funcional da API.

5.2 Levantamento de requisitos

A extração de requisitos ocorreu a partir da análise documental e bibliográfica descrita no Capítulo 4, alinhando as necessidades dos diferentes atores envolvidos no processo de doação de sangue às capacidades técnicas do sistema proposto.

Os requisitos funcionais representam as operações que a API disponibiliza para atender às rotinas de cadastro, gerenciamento de solicitações, recomendação e controle de doações. A Tabela 5 sintetiza as funcionalidades identificadas e implementadas, refletindo diretamente os recursos (*endpoints*) expostos pela API.

Tabela 5 – Requisitos funcionais

Código	Descrição
RF01	Permitir o cadastro de pessoas no sistema.
RF02	Permitir o cadastro de organizações no sistema.
RF03	Permitir que uma pessoa cadastrada registre um perfil de doador.
RF04	Permitir que uma pessoa ou organização registre um perfil de requisitante.
RF05	Permitir que uma organização cadastrada registre um perfil de banco de sangue (hemocentro).
RF06	Permitir a autenticação de usuários por meio de credenciais válidas, emitindo um JWT.
RF07	Permitir a atualização dos dados cadastrais de um participante (*party*).
RF08	Permitir a atualização do perfil e das preferências de recomendação do doador.
RF09	Permitir que requisitantes criem solicitações de doação de sangue.
RF10	Permitir a atualização dos dados de uma solicitação (meta de bolsas e/ou data-limite).
RF11	Permitir o cancelamento de uma solicitação de doação.
RF12	Permitir a consulta das solicitações de doação de um participante.
RF13	Permitir a consulta das solicitações de doação recomendadas para um doador.
RF14	Permitir a notificação de doadores potenciais elegíveis para uma solicitação.
RF15	Permitir o registro de uma doação, informando a data de intenção (pendente) ou a data efetiva (concluída).
RF16	Permitir o reagendamento de uma doação pendente.
RF17	Permitir a confirmação (conclusão) de uma doação pendente.
RF18	Permitir a consulta do histórico de doações de um doador.
RF19	Permitir a consulta do resumo (*summary*) de um doador.

Fonte: Elaborado pelos próprios autores (2026).

Ressalta-se que a lista de requisitos funcionais apresentada não tem a pretensão de esgotar todas as rotinas administrativas de um hemocentro ou unidade hospitalar, mas de delimitar um conjunto mínimo e coeso de operações indispensáveis para validar os conceitos teóricos e arquiteturais propostos. Esse escopo abrange o ciclo fundamental da doação de sangue ao contemplar, de maneira integrada, a gestão da demanda (abertura e acompanhamento de solicitações por unidades de saúde e pacientes), a oferta (disponibilidade, elegibilidade e intenções de doação pelos voluntários) e o registro histórico das doações realizadas.

Além das funcionalidades diretas, foram definidos requisitos não funcionais para estabelecer os parâmetros de qualidade, segurança e arquitetura esperados. Considerando a adoção do DDD e da arquitetura em camadas, A Tabela 6 apresenta as diretrizes técnicas estabelecidas para a solução.

Tabela 6 – Requisitos não funcionais

Código	Descrição
RNF01	O sistema deve disponibilizar seus serviços por meio de uma RESTful API utilizando o protocolo HTTP.
RNF02	O sistema deve autenticar usuários por meio de JWT.
RNF03	O sistema deve persistir os dados em banco de dados NoSQL (MongoDB).
RNF04	O sistema deve permitir evolução e manutenção sem impacto significativo nas demais camadas da aplicação.
RNF05	O sistema deve seguir uma arquitetura em camadas orientada pelos princípios do DDD.
RNF06	O sistema deve aplicar os princípios SOLID para favorecer baixo acoplamento e alta coesão.
RNF07	A API deve retornar respostas em formato JSON.
RNF08	O sistema deve validar os dados recebidos antes do processamento das operações.
RNF09	O sistema deve controlar o acesso às funcionalidades protegidas por meio de autorização baseada em perfis (*roles*) de usuário.
RNF10	O sistema deve preservar a consistência de escritas concorrentes sobre todas as entidades agregadas, evitando a perda de atualizações.

Fonte: Elaborado pelos próprios autores (2026).

Diferentemente dos requisitos funcionais, que especificam os serviços e as ações que o sistema deve executar, os requisitos não funcionais definem as qualidades, restrições e propriedades estruturais do *software*. Nesse sentido, os itens elencados estabelecem parâmetros de segurança (como autenticação sem estado e autorização por papéis), manutenibilidade e baixo acoplamento (assegurados pela divisão em camadas orientada ao DDD e ao SOLID), integridade de dados (por meio do controle de concorrência otimista) e padronização das comunicações (via RESTful API e formato JSON), sustentando a robustez técnica da solução desenvolvida.

5.3 Modelagem do sistema

Após a consolidação dos requisitos, procedeu-se com a modelagem do sistema para descrever as interações entre os atores e a API, permitindo visualizar o comportamento esperado da solução sob a perspectiva estrutural e de uso.

5.3.1 Casos de uso

A modelagem de casos de uso, em notação UML, sintetiza as operações disponíveis para os diferentes perfis de acesso. A Tabela 7 apresenta o mapeamento dos casos de uso e seus respectivos atores, incluindo tanto as operações representadas nos diagramas a seguir quanto operações complementares oferecidas pela API para os mesmos atores.

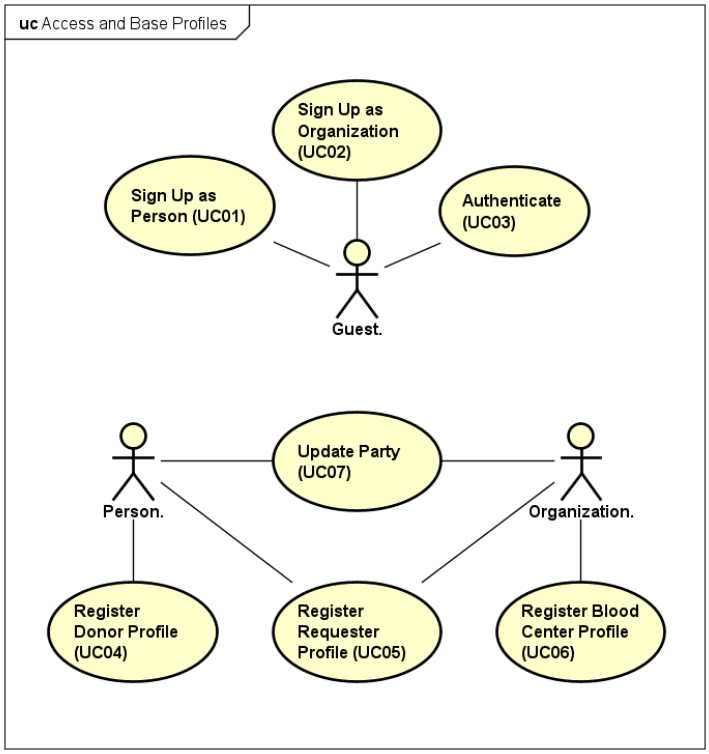
Tabela 7 – Lista de casos de uso

Código	**Caso de uso**	**Ator principal**
UC01	Cadastrar-se como pessoa (*Sign Up as Person*)	Guest
UC02	Cadastrar-se como organização (*Sign Up as Organization*)	Guest
UC03	Autenticar (*Authenticate*)	Guest
UC04	Registrar perfil de doador	Person
UC05	Registrar perfil de requisitante	Person / Organization
UC06	Registrar perfil de banco de sangue	Organization
UC07	Atualizar dados cadastrais (*Update Party*)	Person / Organization
UC08	Atualizar perfil e preferências do doador (*Update Donor*)	Donor
UC09	Registrar solicitação de doação (*Register Donation Request*)	Requester
UC10	Consultar recomendações de solicitações (*Consult Recommendations*)	Donor
UC11	Registrar doação (*Register Donation*)	Donor
UC12	Completar doação pendente (*Complete a Pending Donation*)	Donor
UC13	Consultar histórico de doações (*See Donation History*)	Donor
UC14	Gerenciar solicitações — atualizar dados, cancelar e consultar	Requester
UC15	Notificar doadores potenciais elegíveis	Requester
UC16	Reagendar doação pendente	Donor
UC17	Consultar resumo do doador (*Donor Summary*)	Donor

Fonte: Elaborado pelos próprios autores (2026).

A representação gráfica dessas interações é dividida de acordo com o nível de acesso dos atores. A Figura 5 apresenta o diagrama de caso de uso de acesso e perfis base (*Guest, Person and Organization*).

Figura 5 – Diagrama de caso de uso de acesso e perfis base

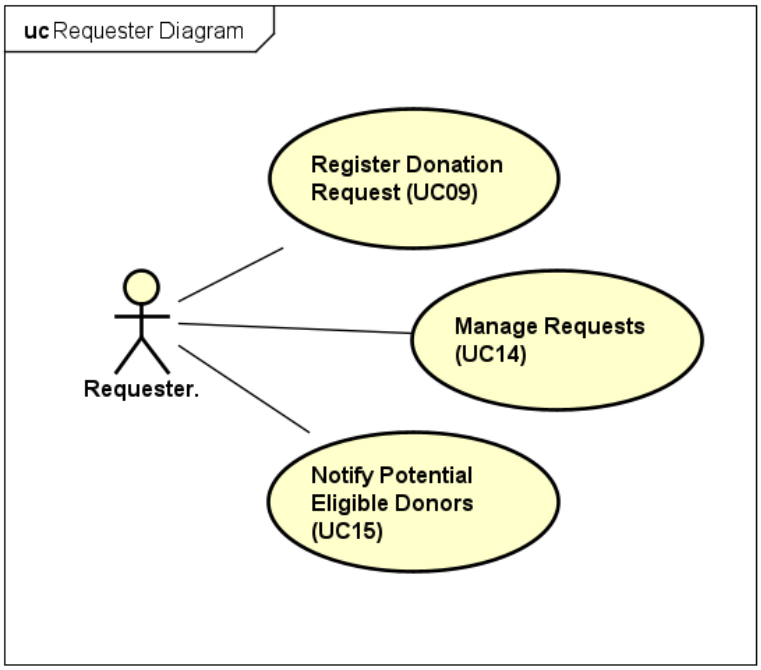


Fonte: Elaborado pelos próprios autores (2026).

O ator *Guest* representa utilizadores sem cadastro prévio, limitados às operações de criação de contas e autenticação (UC01, UC02 e UC03). Os atores *Person* e *Organization* representam os usuários recém-cadastrados, permitindo a atualização de dados da conta (UC07) e a definição de seus papéis específicos no sistema de domínio, como Doador (UC04), Requisitante (UC05) ou Banco de Sangue (UC06).

Já a Figura 6 representa o diagrama de caso de uso do ator (*Requester*).

Figura 6 – Diagrama de caso de uso de requisitante

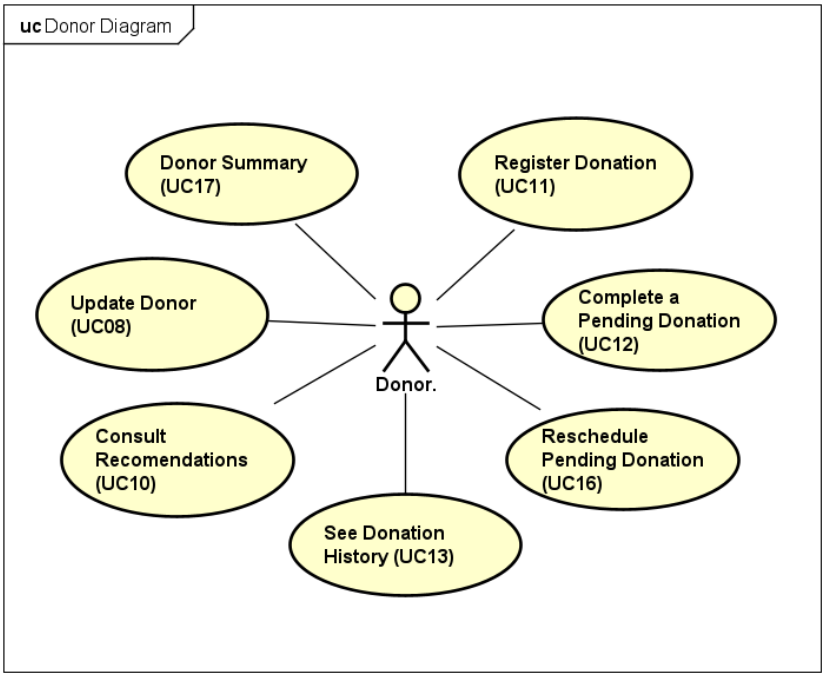


Fonte: Elaborado pelos próprios autores (2026).

O ator *Requester* concentra as permissões relacionadas à criação de solicitações de doação (UC09) e à sua gestão (UC14), além do disparo de notificações a doadores potenciais quando uma solicitação necessita de maior alcance (UC15).

Por fim, a Figura 7 representa o diagrama de caso de uso do ator (*Donor*).

Figura 7 – Diagrama de caso de uso de doador



Fonte: Elaborado pelos próprios autores (2026).

O ator *Donor* possui acesso ao núcleo do processo de doação, incluindo a atualização de seu perfil clínico (UC08), a consulta às solicitações recomendadas (UC10), o registro unificado de doações (UC11), a conclusão (UC12) e o reagendamento de doações pendentes (UC16), a consulta ao próprio histórico hemoterápico (UC13) e a emissão de seu resumo (UC17). Vale ressaltar que, no UC11, a doação nasce pendente quando o doador informa a data de intenção ou concluída quando informa a data efetiva — um único caso de uso, sem métodos concorrentes de instanciação no domínio.

5.3.1.1 Detalhamento dos casos de uso

Devido à abrangência da API e visando manter a concisão documental, optou-se por detalhar apenas os casos de uso críticos para o núcleo da aplicação, adotando a estrutura formal de especificação textual proposta por Cockburn (2001). Essa abordagem documenta o fluxo de eventos, as restrições operacionais e as pré-condições de segurança exigidas, justificando arquiteturalmente a ausência de relacionamentos visuais de dependência de autenticação nos diagramas UML. A Tabela 8 sintetiza essas especificações.

Tabela 8 – Especificação dos casos de uso críticos

UC09: Registrar solicitação de doação	
Ator principal	Requisitante (*Requester*)
Pré-condição	Usuário autenticado com perfil de Requisitante ativo.
Fluxo principal	1. O Requisitante solicita a criação de uma nova campanha. 2. Informa dados da necessidade (tipo sanguíneo, bolsas, prazo, urgência e hemocentro). 3. O sistema valida as regras de negócio e os invariantes de domínio. 4. O sistema persiste a solicitação no estado ativo. 5. O sistema retorna a confirmação com o identificador único gerado.
Cenário alternativo	**3a.** Falha de validação (prazo retroativo ou meta nula): rejeita a operação e retorna mensagem de erro estruturada.
UC10: Consultar recomendações de solicitações	
Ator principal	Doador (*Donor*)
Pré-condição	Usuário autenticado com perfil de Doador ativo.
Fluxo principal	1. O Doador solicita a consulta de campanhas recomendadas. 2. O sistema avalia a elegibilidade temporal com base na última doação. 3. O sistema recupera solicitações ativas e avalia a compatibilidade tipológica. 4. O sistema aplica o filtro geoespacial pela distância máxima configurada. 5. O sistema ordena por urgência, proximidade e prazo, retornando a listagem.
Cenário alternativo	**2a.** Doador temporariamente inelegível: encerra o processamento e retorna listagem vazia.
UC12: Completar doação pendente	
Ator principal	Doador (*Donor*)
Pré-condição	Usuário autenticado com perfil de Doador ativo.
Fluxo principal	1. O Doador submete a data de efetivação de intenção previamente registrada. 2. O sistema recupera a doação pendente associada ao identificador. 3. O sistema valida a consistência cronológica em relação à data corrente. 4. O sistema transiciona o estado do agregado para concluído. 5. O sistema atualiza o registro do doador, recalculando a janela de descanso regulamentar.
Cenário alternativo	**3a.** Data inconsistente (data futura): rejeita a atualização e mantém a doação pendente.

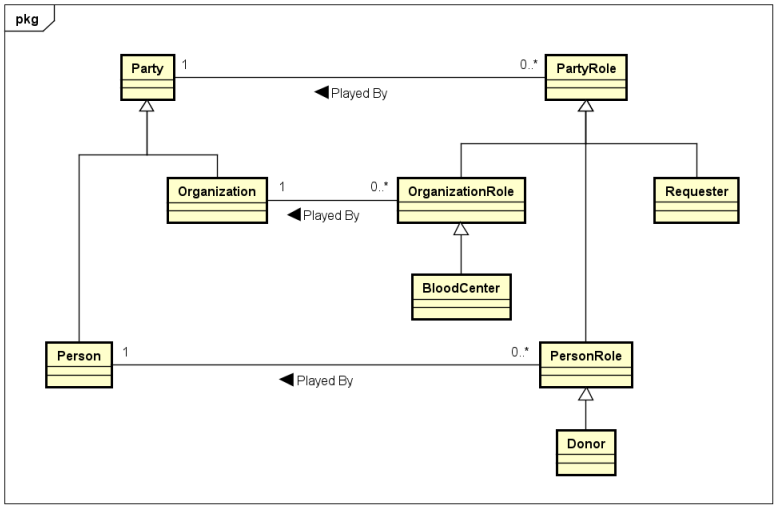
Fonte: Elaborado pelos próprios autores (2026).

5.3.2 Diagramas de classes

Para representar a estrutura estática da aplicação e evidenciar a aplicação do DDD na prática — cujos conceitos de entidade, agregado e objeto de valor já foram apresentados no Capítulo 2 —, a modelagem de classes foi concebida em duas perspectivas distintas: de análise e de design. A perspectiva de análise mapeia o vocabulário do negócio e as relações semânticas do mundo real, omitindo métodos e tipos de dados para facilitar a validação com especialistas. Em contrapartida, a perspectiva de design atua como uma representação estrutural do código-fonte construído, detalhando os limites transacionais, o encapsulamento e as diretrizes arquiteturais.

A Figura 8 apresenta o diagrama de classes de análise focado nos participantes e em seus papéis (*Roles*).

Figura 8 – Diagrama de análise dos participantes

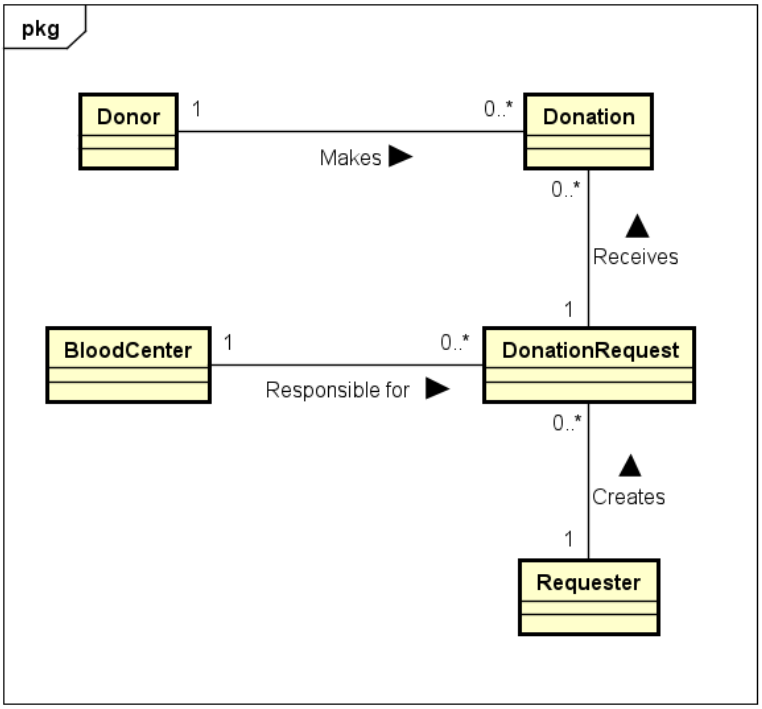


Fonte: Elaborado pelos próprios autores (2026).

Observa-se a utilização da abstração central `Party`, que atua como classe base para `Person` (Pessoa) e `Organization` (Organização). Essa modelagem reflete a flexibilidade do domínio, permitindo que as entidades assumam múltiplos papéis, como `Donor` (Doador) e `Requester` (Requisitante), eliminando a duplicidade de dados no sistema.

Dando prosseguimento à representação do negócio, a Figura 9 ilustra o diagrama de classes de análise do processo de doação.

Figura 9 – Diagrama de análise do processo de doação

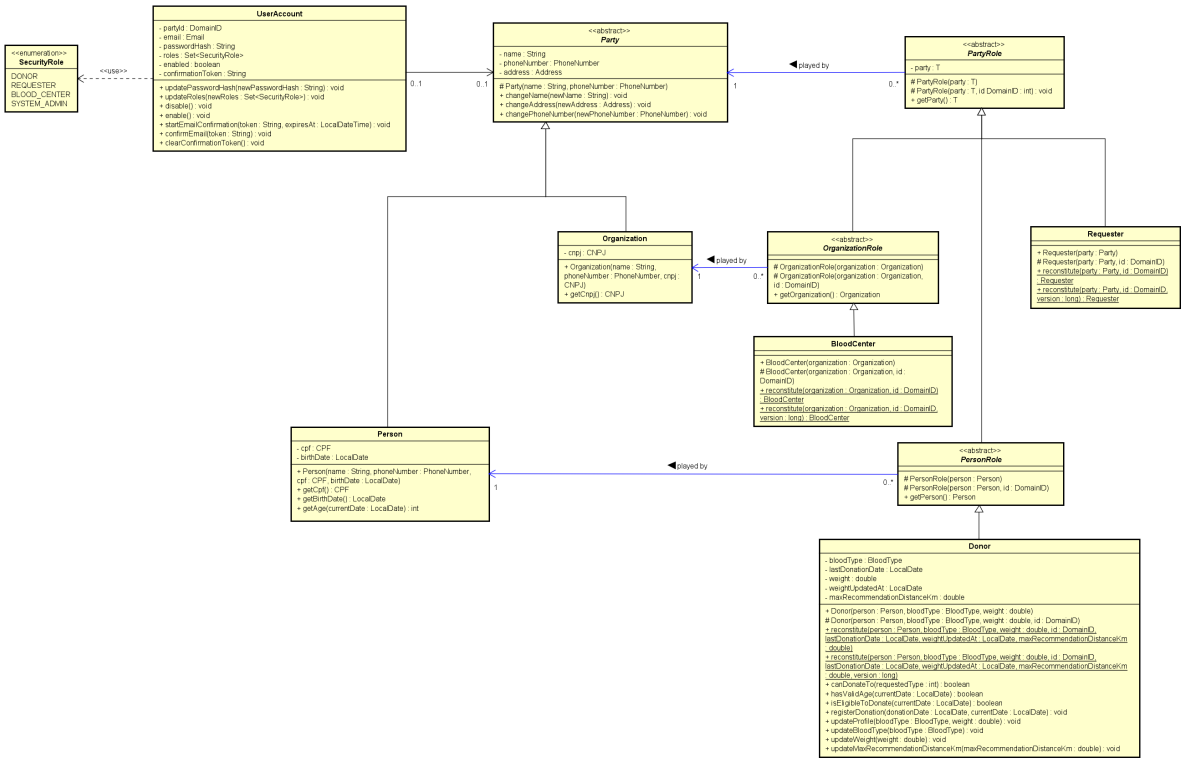


Fonte: Elaborado pelos próprios autores (2026).

Neste nível de abstração, evidencia-se a associação semântica entre a solicitação (`DonationRequest`) e o ato de doar (`Donation`). O fluxo natural demonstra que um `Requester` origina a solicitação, que é vinculada a um `BloodCenter` e, posteriormente, recebe as doações efetivadas por um `Donor`.

Elevando o rigor para o design de software orientado a objetos, a modelagem de design reflete com fidelidade os limites de encapsulamento, os tipos dos atributos, as visibilidades e os contratos de instanciação e reconstituição. A Figura 10 detalha o diagrama de classes de design de `Party` e da respectiva hierarquia de papéis (`PartyRole`).

Figura 10 – Diagrama de design: participantes, papéis e controle de acesso



Fonte: Elaborado pelos próprios autores (2026).

Na perspectiva de design de participantes e papéis, destacam-se decisões estruturais essenciais:

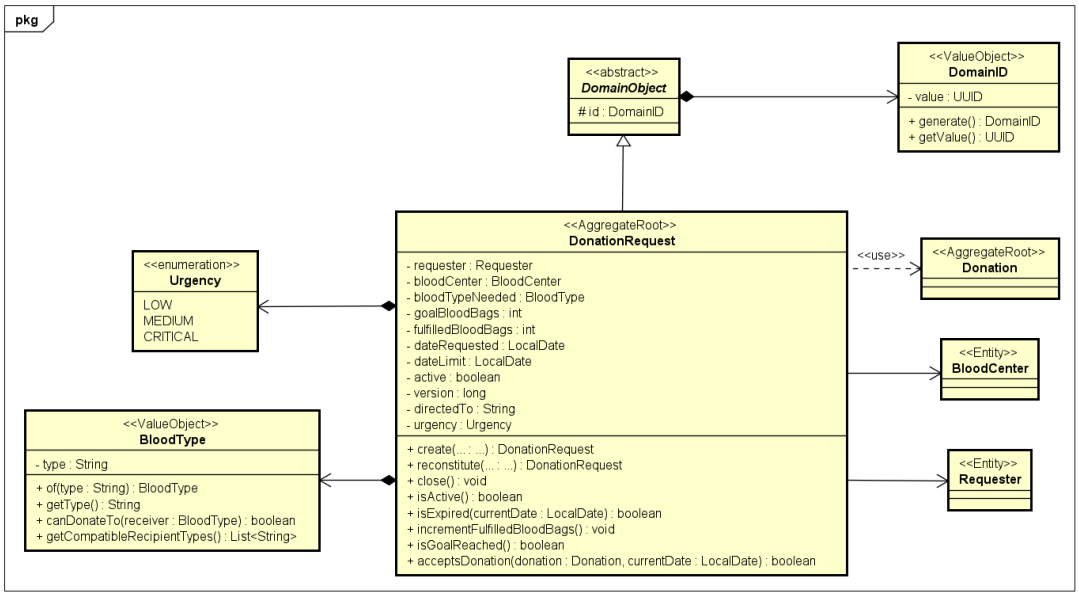
- **Segregação entre Identidade e Acesso:** Os dados cadastrais do participante (`Party`) são estritamente isolados da sua conta de acesso (`UserAccount`), associada de forma opcional (0..1 ↔ 0..1). A entidade `UserAccount` concentra as credenciais, o estado da conta (`enabled`) e os perfis de segurança (`SecurityRole`), evitando que regras de autenticação e credenciais poluam as entidades do domínio.
- **Hierarquia Base com Encapsulamento Protegido:** A superclasse abstrata `Party` define atributos protegidos para nome, telefone e endereço, sendo estendida pelas classes concretas `Person` (que introduz CPF, data de nascimento e o cálculo de idade pelo método `getAge`) e `Organization` (que introduz o CNPJ).
- **Parametrização e Tipagem Forte de Papéis (*Party-Role*):** A classe genérica abstrata `PartyRole<T>` impõe que todo papel mantenha uma referência obrigatória e imutável ao seu participante gerador através do construtor protegido `#PartyRole(party:T)`. A partir dessa base, definem-se os limites estritos de cada papel:
  - `Requester`: estende `PartyRole<Party>`, viabilizando polimorficamente que tanto pessoas físicas (`Person`) quanto jurídicas (`Organization`) abram solicitações de doação;

  - `BloodCenter`: especializa `OrganizationRole`, garantindo em tempo de compilação que apenas organizações possam atuar como hemocentros;
  - `Donor`: especializa `PersonRole`, assegurando que apenas pessoas físicas atuem como doadores voluntários. Essa classe concentra os métodos de negócio fundamentais da hemoterapia, como `canDonateTo` (compatibilidade sanguínea), `isEligibleToDonate` (janela temporal de descanso e idade regulamentar) e `registerDonation`.

- **Contratos Explícitos de Reconstituição:** As classes de papéis disponibilizam tanto construtores para novos registros quanto métodos estáticos `reconstitute(...)` com controle de identificador (`DomainID`) e versão de concorrência (`version`), permitindo que o repositório remonte agregados a partir do banco de dados sem acionar validações exclusivas da criação primária.

Dando prosseguimento aos agregados centrais do sistema, a Figura 11 detalha o diagrama de design do agregado `DonationRequest`.

Figura 11 – Diagrama de design: agregado DonationRequest

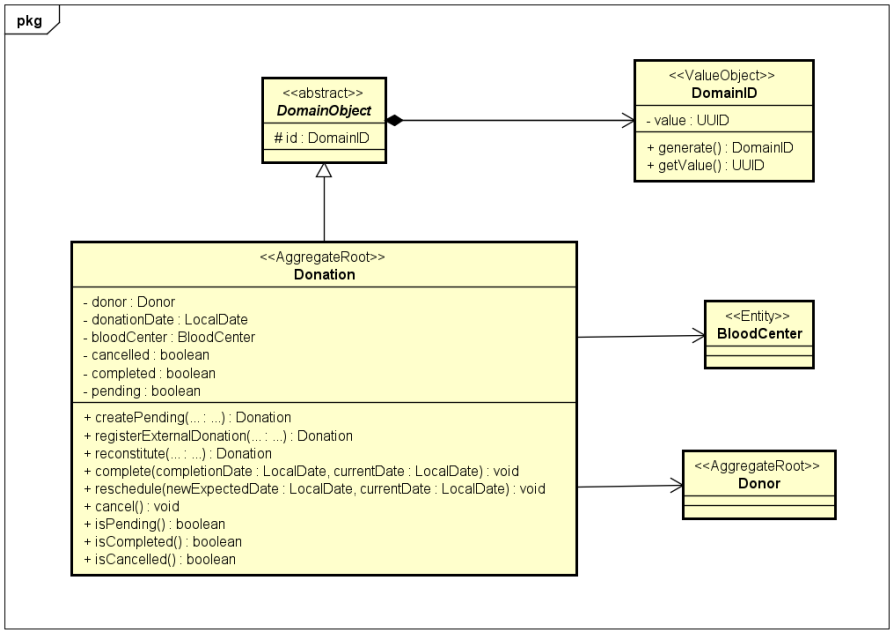


Fonte: Elaborado pelos próprios autores (2026).

A classe é modelada como uma raiz de agregado («`AggregateRoot`»), atuando como a única porta de entrada para a proteção de invariantes, como as datas-limite e a urgência da solicitação. Observa-se a aplicação da herança por meio da superclasse abstrata `DomainObject`, responsável por compor os identificadores únicos globais («`ValueObject`» `DomainID`). Destaca-se também a dependência tracejada («`use`») com a classe `Donation`, a qual evidencia o baixo acoplamento da arquitetura: a requisição não armazena doações em memória, apenas as avalia de forma transitória por meio de parâmetros no método `acceptsDonation()`.

Por fim, a Figura 12 apresenta o diagrama de design do agregado `Donation`.

Figura 12 – Diagrama de design: agregado Donation



Fonte: Elaborado pelos próprios autores (2026).

Tratada também como uma raiz de agregado autônoma, a classe concentra um único *static factory method* `create` e o ciclo de vida pendente–concluída–cancelada. O estado não é armazenado como flags independentes: deriva das datas de intenção (`intendedDate`), de efetivação (`donationDate`) e de cancelamento (`cancelledAt`). Completar uma doação pendente preenche a data efetiva sem apagar a intenção; reagendar e cancelar permanecem transições sobre o mesmo agregado. A navegabilidade unidirecional explícita demonstra que a doação possui a referência imediata do `Donor` e do `BloodCenter` envolvidos, assegurando a consistência da operação sem a necessidade de carregar listas e metadados completos de uma solicitação simultaneamente.

5.4 Domínio e arquitetura

A partir das interações mapeadas, estruturou-se o núcleo da aplicação: o modelo de domínio e a organização das camadas que o protegem de dependências tecnológicas externas.

5.4.1 Linguagem ubíqua e organização em camadas

Conforme estabelecido nos fundamentos do DDD (Capítulo 2), a modelagem foi construída sobre uma linguagem ubíqua, garantindo que termos como `DonationRequest`, `Donor`, `Requester` e `BloodCenter` transitem com a mesma semântica tanto nas discussões de projeto quanto no código-fonte final. Essa padronização reduz ambiguidades entre a gestão de perfis de participantes e o ciclo de vida das doações.

A organização estrutural do código adota o padrão orientado a entidades de domínio (*Package-by-Entity* / *Package-by-Feature*), promovendo alta coesão e baixo acoplamento entre os conceitos do sistema. Em vez de uma divisão horizontal global agrupando classes exclusivamente por camada técnica, o sistema é particionado em pacotes de alto nível que representam diretamente os agregados e entidades de domínio (`request`, `donation`, `party`, `role` e `auth`), complementados por um pacote compartilhado (`shared`) para conceitos transversais (*Shared Kernel*).

Internamente, cada pacote de entidade é estruturado nos subpacotes que materializam as camadas arquiteturais clássicas, garantindo a separação estrita de responsabilidades:

- `domain`: entidades, raízes de agregado, objetos de valor, regras de negócio e interfaces de portas de repositórios e serviços de domínio, totalmente livres de acoplamento com frameworks ou tecnologias externas;
- `application`: orquestração dos casos de uso, coordenando o fluxo de dados entre as portas de repositório e os modelos de domínio;
- `infrastructure`: implementações concretas dos repositórios (Spring Data MongoDB), esquemas de persistência de banco de dados, filtros de segurança e clientes de serviços externos;
- `presentation`: controladores REST (*controllers*) e objetos de transferência de dados (DTOs) responsáveis pela exposição HTTP da API, recepção de requisições e validação sintática primária.

O fluxo de dados segue os princípios da arquitetura limpa: a camada de apresentação invoca a camada de aplicação, que opera sobre o domínio (`presentation` → `application` → `domain`). A camada de infraestrutura, por sua vez, implementa as interfaces definidas no domínio (`infrastructure` → `domain`), garantindo a aplicação do Princípio da Inversão de Dependência (DIP) entre os módulos de alto e baixo nível (Martin, 2008).

5.4.2 Entidades, agregados e objetos de valor

A estrutura central do domínio foi modelada de forma hierárquica e orientada a objetos. A Tabela 9 sintetiza as principais entidades e agregados responsáveis por garantir a consistência das transações no sistema.

Tabela 9 – Entidades e suas responsabilidades

Entidade	**Responsabilidade**
Party	Abstrai características comuns dos participantes do sistema.
Person	Representa pessoas físicas cadastradas.
Organization	Representa instituições cadastradas.
Donor	Representa o perfil de doador e suas regras de elegibilidade.
Requester	Representa o perfil requisitante.
BloodCenter	Representa o perfil de banco de sangue vinculado a uma organização.
DonationRequest	Representa uma solicitação de doação de sangue, incluindo sua meta e seu progresso.
Donation	Representa uma doação registrada no sistema, cujo estado (pendente, concluída ou cancelada) deriva das datas de intenção, de efetivação e de cancelamento.

Fonte: Elaborado pelos próprios autores (2026).

A entidade `Party` foi adotada como abstração de mais alto nível da hierarquia, aplicando herança para derivar `Person` e `Organization`. A partir dessas bases, o sistema utiliza polimorfismo para permitir que os participantes assumam papéis (*Roles*) específicos, como `Donor`, `Requester` ou `BloodCenter`, dependendo de suas ações no sistema.

As entidades `DonationRequest` e `Donation` operam como raízes de agregados (*Aggregate Roots*), concentrando regras de negócio — como a validação de datas-limite e a verificação de compatibilidade sanguínea — e protegendo o estado interno dos objetos por meio do encapsulamento. Adicionalmente, conceitos imutáveis sem identidade própria, como `Address` (Endereço), `BloodType` (Tipo Sanguíneo) e `DomainID`, foram modelados como objetos de valor, na linha proposta por Evans (2003) e detalhada por Vernon (2013), aumentando a segurança e a previsibilidade do modelo.

5.5 Endpoints e recursos da RESTful API

A arquitetura da API foi projetada para refletir diretamente os agrupamentos lógicos do domínio (*Package-by-Entity*). Dessa forma, os *endpoints* foram estruturados e organizados em torno dos principais agregados e entidades do sistema, promovendo alta coesão e previsibilidade no consumo dos recursos. Nas tabelas a seguir, apresentam-se as rotas mapeadas para cada módulo da aplicação, detalhando os métodos HTTP e suas respectivas finalidades.

A Tabela 10 consolida os *endpoints* de acesso público e gerenciamento de identidade, responsáveis por garantir a segurança e a entrada no sistema.

Tabela 10 – Autenticação e Acesso Público (Auth)

Método HTTP	**Descrição**	**URI**
POST	Autenticar Usuário (Login)	`/auth/login`
GET	Confirmar E-mail (via link)	`/auth/confirm-email?token=`
POST	Confirmar E-mail (via payload)	`/auth/confirm-email`
POST	Reenviar Confirmação de E-mail	`/auth/resend-confirmation`

Fonte: Elaborado pelos próprios autores (2026).

Este módulo atua como a fronteira de segurança da aplicação. A rota de autenticação é encarregada de validar credenciais e emitir o *JSON Web Token* (JWT), exigido para o consumo das rotas privadas. Além disso, as rotas de confirmação de e-mail garantem a verificação em duas etapas, assegurando a validade e a titularidade dos canais de comunicação cadastrados pelos usuários.

Na sequência, A Tabela 11 apresenta as rotas destinadas ao cadastro base dos participantes do sistema, abstraídos conceitualmente na arquitetura como `Party`.

Tabela 11 – Participantes (Party)

Método HTTP	**Descrição**	**URI**
POST	Registrar Pessoa	`/parties/persons`
POST	Registrar Organização	`/parties/organizations`
PATCH	Atualizar Dados Cadastrais da Parte	`/parties`

Fonte: Elaborado pelos próprios autores (2026).

Esses *endpoints* operam no nível mais fundamental do cadastro, permitindo o registro e a manutenção de atributos comuns a pessoas físicas e organizações, como credenciais de acesso, dados de contato e localização. A separação dessas rotas reflete a modelagem do domínio, em que a identidade básica do participante existe independentemente da função específica que ele virá a desempenhar no processo de doação de sangue.

Para gerenciar essas funções específicas, A Tabela 12 detalha as operações que permitem aos participantes base assumirem papéis (*Roles*) dentro do ecossistema do banco de sangue.

Tabela 12 – Papéis e Perfis (Role)

Método HTTP	**Descrição**	**URI**
POST	Registrar perfil de Doador	`/donors`
PATCH	Atualizar perfil de Doador	`/donors`
GET	Acessar informações (Resumo) do Doador	`/donors/{person_id}/summary`
POST	Registrar perfil de Requisitante	`/requesters`
POST	Registrar perfil de Hemocentro	`/blood-centers`
GET	Buscar Hemocentro(s)	`/blood-centers/search`

Fonte: Elaborado pelos próprios autores (2026).

A estruturação destas requisições evidencia a flexibilidade da modelagem orientada a objetos adotada. Através dessas rotas, uma entidade do tipo `Person` pode se registrar como um Doador ou Requisitante, enquanto uma `Organization` pode assumir o papel de Hemocentro ou Requisitante. Adicionalmente, as rotas de consulta exclusivas deste pacote expõem dados sumarizados e públicos atrelados a esses perfis, facilitando a busca por hemocentros e a visualização de resumos clínicos.

A Tabela 13, por sua vez, mapeia as rotas responsáveis pelo gerenciamento das campanhas de doação, concentrando as regras de negócio mais complexas da aplicação.

Tabela 13 – Solicitações de Doação (Request)

Método HTTP	**Descrição**	**URI**
POST	Criar Solicitação de Doação	`/donation-requests`
PATCH	Atualizar Solicitação de Doação	`/donation-requests`
GET	Buscar Solicitações por PartyId	`/donation-requests/{party_id}`
DELETE	Cancelar Solicitação de Doação	`/donation-requests/{request_id}`
GET	Obter Solicitações Recomendadas	`/donation-requests/recommendations`
POST	Enviar notificação para doadores elegíveis	`/donation-requests/{request_id}/notify`

Fonte: Elaborado pelos próprios autores (2026).

Este agrupamento expõe as operações do agregado central `DonationRequest`. Ele abrange desde as transações de manutenção do ciclo de vida da solicitação (como criação, atualização de metas e cancelamento) até o acionamento dos motores heurísticos de compatibilidade.

Destacam-se a rota de recomendação, desenhada para fornecer ao doador as campanhas mais aderentes ao seu perfil biológico e geográfico, e o *endpoint* de notificação ativa, que delega ao sistema o engajamento de candidatos elegíveis.

Por fim, A Tabela 14 expõe as rotas encarregadas do registro e da manutenção do histórico do ato hemoterápico propriamente dito.

Tabela 14 – Doações (Donation)

Método HTTP	**Descrição**	**URI**
POST	Criar Doação (Pendente/Concluída)	`/donations`
PATCH	Completar Doação Pendente	`/donations/complete`
PATCH	Reagendar Doação Pendente	`/donations/reschedule`
GET	Acessar histórico de doações do doador	`/donors/{person_id}/donations`

Fonte: Elaborado pelos próprios autores (2026).

O design destas rotas reflete a máquina de estados interna do agregado `Donation`. Em vez de um único ponto de atualização genérico para manipulação de dados, a API oferece semânticas declarativas e claras para transicionar a doação de uma intenção agendada (pendente) para uma efetivação (concluída) ou para um reagendamento. Tais rotas alimentam diretamente o histórico clínico dos doadores, afetando dinamicamente o cálculo de suas janelas de inaptidão temporal para doações futuras.

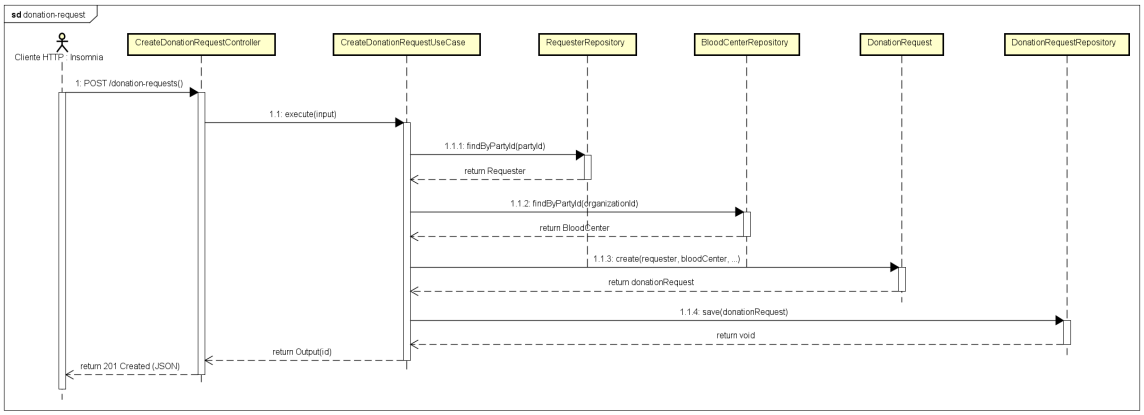
5.6 Implementação e fluxo de dados

A implementação técnica da API orienta-se pelos princípios da arquitetura em camadas e do *Domain-Driven Design* (DDD). Para fins de exemplificar a aplicação prática desses conceitos, esta seção demonstra o fluxo completo de uma entidade — o registro de uma **Solicitação de Doação** (`DonationRequest`) —, acompanhando a requisição desde a sua chegada no controlador REST até a sua persistência no banco de dados com controle de concorrência. Esse rastreamento horizontal ilustra o papel de cada camada arquitetural e as transformações sofridas pelos dados.

5.6.1 Visão geral do fluxo de execução

Para compreender a orquestração entre as camadas da arquitetura antes da análise direta do código-fonte, a modelagem temporal das requisições faz-se necessária. A Figura 13 apresenta o diagrama de sequência UML detalhando o fluxo completo de criação de uma solicitação de doação, mapeando o trajeto da informação desde a interceptação da chamada HTTP até a persistência no banco de dados.

Figura 13 – Diagrama de sequência: DonationRequest



Fonte: Elaborado pelos próprios autores (2026).

O diagrama ilustra a estrita separação de responsabilidades estabelecida pela arquitetura em camadas e pelo *Domain-Driven Design*. O fluxo é iniciado pelo cliente HTTP, que aciona o controlador da camada de apresentação. Este converte os dados de entrada e delega a operação para a camada de aplicação por meio do método `execute(input)` do caso de uso correspondente. No núcleo da aplicação, o caso de uso coordena a recuperação dos atores envolvidos (`Requester` e `BloodCenter`) por meio de suas respectivas interfaces de repositório. Com as dependências estruturais carregadas, o caso de uso invoca o *static factory method* `create(...)` diretamente na raiz do agregado `DonationRequest`, responsável por proteger suas invariantes e instanciar o objeto de domínio. Por fim, a nova entidade é persistida na camada de infraestrutura e o fluxo de resposta transita de volta de forma decrescente, culminando no retorno de sucesso (HTTP 201) ao cliente. Essa representação comprova o isolamento do domínio, que permanece alheio aos detalhes do protocolo web e da tecnologia de banco de dados.

As subseções a seguir detalham a implementação técnica e o código de cada componente participante desse fluxo.

5.6.2 Camada de Apresentação: REST Controller e DTOs

A camada de apresentação atua como o ponto de entrada da aplicação, isolando o núcleo do sistema dos detalhes do protocolo HTTP. O `CreateDonationRequestController` é responsável por interceptar as requisições, executar validações sintáticas básicas e converter o *payload* da requisição (DTO) para o modelo de entrada do caso de uso.

Listagem 5.1 – Exemplo de REST Controller da Solicitação de Doação

```
1 @RestController
2 @RequestMapping("/donation-requests")
3 public class CreateDonationRequestController {
4
5   private final CreateDonationRequestUseCase useCase;
6
```

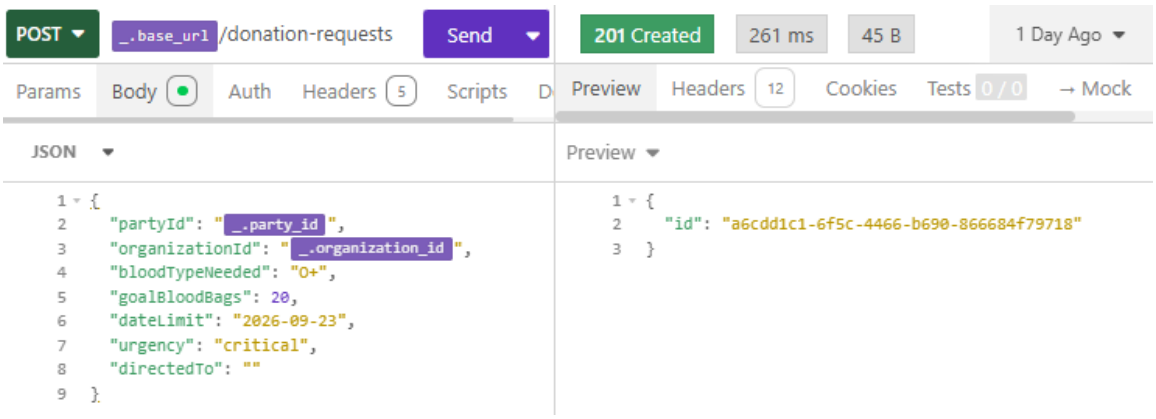
```
7   public CreateDonationRequestController(CreateDonationRequestUseCase useCase) {
8     this.useCase = useCase;
9   }
10
11  @PostMapping
12  public ResponseEntity<CreateDonationRequestResponseDto> create(
13      @RequestBody CreateDonationRequestDto payload) {
14
15    // Validacoes sintaticas primarias
16    requireNonNull(payload.goalBloodBags(), "goalBloodBags cannot be null");
17    requireSamePartyOrAdmin(payload.partyId()); // Autorizacao
18
19    // Conversao do DTO para o objeto de entrada (Input) do Caso de Uso
20    var output = useCase.execute(new Input(
21        payload.partyId(),
22        payload.organizationId(),
23        payload.bloodTypeNeeded(),
24        payload.goalBloodBags(),
25        payload.dateLimit(),
26        payload.urgency(),
27        payload.directedTo()));
28
29    return ResponseEntity
30        .status(HttpStatus.CREATED)
31        .body(CreateDonationRequestResponseDto.from(output));
32  }
33 }
```

A Figura 14 ilustra na prática a execução deste controlador por meio do cliente HTTP Insomnia. Observa-se a requisição enviada para a URI `/donation-requests` utilizando o verbo POST, contendo no corpo da requisição (*Request Body*) o JSON de entrada com os dados preenchidos pelo requisitante. Para ilustrar o formato trafegado, a Listagem 5.2 apresenta o JSON enviado na requisição HTTP POST.

Listagem 5.2 – JSON de entrada na requisição HTTP POST

```
1 {
2   "partyId": "party-12345",
3   "organizationId": "org-67890",
4   "bloodTypeNeeded": "A_POSITIVE",
5   "goalBloodBags": 10,
6   "dateLimit": "2026-12-31",
7   "urgency": "HIGH",
8   "directedTo": "Joao Silva"
9 }
```

Figura 14 – Execução do fluxo de criação de solicitação de doação via cliente HTTP Insomnia



Fonte: Elaborado pelos próprios autores (2026).

A resposta gerada pelo sistema comprova o sucesso da operação, retornando o código de estado HTTP `201Created`. O corpo da resposta (*Response Body*) devolve ao cliente um novo JSON contendo o identificador único (`id`) gerado pelo domínio para a nova solicitação, atestando que a requisição transpassou as camadas de validação e foi persistida com sucesso.

5.6.3 Camada de Aplicação: Casos de Uso e o Padrão *execute*

A camada de aplicação orquestra o fluxo de dados coordenando chamadas aos repositórios e aos serviços de domínio. O `CreateDonationRequestUseCase` carrega as entidades associadas (`Requester` e `BloodCenter`), invoca o *static factory method* da entidade e instrui o repositório a persistir o novo agregado. Esta camada desconhece qualquer dependência de infraestrutura, utilizando exclusivamente portas de interfaces.

Para padronizar a orquestração de negócios, todos os casos de uso do sistema adotam o padrão formal estruturado na assinatura:

```
Output execute(Input input)
```

Essa padronização fundamenta-se nos seguintes princípios arquiteturais:

- **Princípio da Responsabilidade Única (SRP) e *Command Pattern*:** Cada classe de caso de uso representa uma única operação de negócio do sistema. A presença de um único ponto de entrada público (`execute`) impede que a classe se transforme em um serviço inflado com múltiplos métodos de ciclos de vida distintos, mantendo a responsabilidade estrita e coesa.

- **Isolamento da Camada de Apresentação via DTOs Imutáveis (*Records*):** Os objetos `Input` e `Output` são concebidos como *records* imutáveis puros da linguagem Java, desprovidos de quaisquer dependências ou anotações de frameworks Web e bibliotecas de serialização. Essa separação garante que a lógica de aplicação permaneça agnóstica

ao protocolo ou mecanismo de entrega, cabendo aos controladores da camada de apresentação mapear o corpo das requisições HTTP para os registros de entrada do caso de uso.

- **Proteção contra Vazamento de Domínio:** O `Output` expõe estritamente os atributos necessários para a resposta ao cliente (como identificadores de recursos gerados), impedindo que entidades e agregados ricos de domínio vazem acidentalmente para camadas externas.
- **Determinismo Temporal em Testes:** O método `execute` admite uma sobrecarga que recebe a data corrente (`LocalDatecurrentDate`), viabilizando testes unitários puramente determinísticos sobre regras de expiração e prazos, sem ancoragem rígida ao relógio do sistema operacional.

A Listagem 5.3 exemplifica o caso de uso de criação com injeção explícita de dependências via construtor e declaração formal de seus contratos de entrada e saída.

Listagem 5.3 – Exemplo de Caso de Uso com Padrão Output execute(Input)

```
1  @Service
2  public class CreateDonationRequestUseCase {
3
4    private final DonationRequestRepositoryInterface donationRequestRepository;
5    private final RequesterRepositoryInterface requesterRepository;
6    private final BloodCenterRepositoryInterface bloodCenterRepository;
7
8    public CreateDonationRequestUseCase(
9        DonationRequestRepositoryInterface donationRequestRepository,
10       RequesterRepositoryInterface requesterRepository,
11       BloodCenterRepositoryInterface bloodCenterRepository) {
12     this.donationRequestRepository = donationRequestRepository;
13     this.requesterRepository = requesterRepository;
14     this.bloodCenterRepository = bloodCenterRepository;
15   }
16
17   public Output execute(Input input, LocalDate currentDate) {
18     DomainID partyId = DomainIdParser.parse(input.partyId(), "partyId");
19     DomainID organizationId = DomainIdParser.parse(input.organizationId(), "organizationId")
       ;
20
21     Requester requester = requesterRepository.findByPartyId(partyId)
22         .orElseThrow(() -> new NotFoundException("Requester role not found"));
23     BloodCenter bloodCenter = bloodCenterRepository.findByPartyId(organizationId)
24         .orElseThrow(() -> new NotFoundException("Blood center role not found"));
25
26     DonationRequest request = DonationRequest.create(
27         requester, bloodCenter, parseBloodType(input.bloodTypeNeeded()),
28         input.goalBloodBags(), input.dateLimit(),
29         currentDate, parseUrgency(input.urgency()), input.directedTo());
30
31     donationRequestRepository.save(request);
32     return Output.from(request);
```

```
33    }
34
35    public record Input(
36        String partyId, String organizationId, String bloodTypeNeeded,
37        Integer goalBloodBags, LocalDate dateLimit, String urgency, String directedTo) {}
38
39    public record Output(String id) {
40      public static Output from(DonationRequest request) {
41        return new Output(request.getId().getValue().toString());
42      }
43    }
44  }
```

5.6.4 Camada de Domínio: Agregado, Invariantes no Construtor e *Static Factory Method*

O coração do *software* reside na camada de domínio. A classe `DonationRequest` atua como a raiz do agregado, protegendo os seus invariantes e garantindo que nenhuma solicitação seja instanciada em um estado inválido.

Nesse contexto, cabe distinguir com rigor o padrão adotado das fábricas clássicas do GoF (*Factory Method* polimórfico e *Abstract Factory*) ou de serviços de fábrica dedicados do DDD (*Domain Factories*). O modelo implementa o padrão idiomático ***Static Factory Method***, formalizado por Joshua Bloch (2018, Item 1):

- **Expressividade Semântica:** Diferencia de forma explícita as intenções de criação no ciclo de vida do agregado. O método `DonationRequest.create(...)` expressa a criação de uma nova solicitação no sistema (gerando um novo `DomainID` e definindo o estado inicial ativo), enquanto o método `DonationRequest.reconstitute(...)` expressa a reconstituição de um agregado pré-existente carregado a partir do banco de dados, preservando o identificador e a versão de persistência originais.

- **Proteção de Invariantes no Construtor:** As invariantes de consistência e regras estruturais essenciais (como proibir metas zeradas, garantir campos obrigatórios não nulos e impedir datas limite anteriores à data de criação) residem obrigatoriamente no **construtor** da classe. Em Java, o construtor atua como o ponto de estrangulamento obrigatório de instanciação (*chokepoint of integrity*). Se as validações de invariantes permanecessem soltas apenas no método estático `create`, o construtor ficaria desprotegido contra chamadas que pudessem gerar um objeto inconsistente. Portanto, o construtor impõe as regras de integridade do agregado antes de qualquer atribuição de estado, cabendo ao *static factory method* apenas a orquestração semântica dos parâmetros.

A Listagem 5.4 ilustra a proteção das invariantes no construtor do agregado e sua instanciação coordenada pelo *static factory method*.

Listagem 5.4 – Construtor com Invariantes e *Static Factory Method* do Agregado

```
1  public class DonationRequest extends DomainObject {
2    private Requester requester;
3    private BloodCenter bloodCenter;
4    private BloodType bloodTypeNeeded;
5    private int goalBloodBags;
6    private LocalDate dateRequested;
7    private LocalDate dateLimit;
8    private boolean active;
9    private Urgency urgency;
10   private String directedTo;
11
12   public DonationRequest(
13       Requester requester, BloodCenter bloodCenter, BloodType bloodTypeNeeded,
14       int goalBloodBags, LocalDate dateLimit, LocalDate currentDate,
15       Urgency urgency, String directedTo) {
16     // Validacoes de integridade e invariantes de negocio
17     if (goalBloodBags <= 0)
18       throw new IllegalArgumentException("Goal blood bags must be greater than zero");
19     if (dateLimit == null || currentDate == null || dateLimit.isBefore(currentDate))
20       throw new IllegalArgumentException("Limit date invalid");
21     // ... validacoes complementares de nulidade (requester, bloodCenter, bloodType) ...
22
23     this.id = DomainID.generate();
24     this.requester = requester;
25     this.bloodCenter = bloodCenter;
26     this.bloodTypeNeeded = bloodTypeNeeded;
27     this.goalBloodBags = goalBloodBags;
28     this.dateRequested = currentDate;
29     this.dateLimit = dateLimit;
30     this.active = true;
31     this.urgency = urgency;
32     this.directedTo = directedTo;
33   }
34
35   // Static Factory Method idiomatico (Bloch, 2018)
36   public static DonationRequest create(
37       Requester requester, BloodCenter bloodCenter, BloodType bloodTypeNeeded,
38       int goalBloodBags, LocalDate dateLimit, LocalDate currentDate,
39       Urgency urgency, String directedTo) {
40     return new DonationRequest(
41         requester, bloodCenter, bloodTypeNeeded,
42         goalBloodBags, dateLimit, currentDate, urgency, directedTo);
43   }
44 }
```

5.6.5 Camada de Infraestrutura: Mapeamento de Esquemas e Persistência

Na extremidade da arquitetura, a camada de infraestrutura traduz o modelo conceitual de domínio para as estruturas de persistência orientadas a documentos do MongoDB. Essa conversão é desempenhada por classes de esquema (*schemas*), que mapeiam agregados ricos em documentos BSON. Destacam-se nesse mapeamento duas responsabilidades essenciais: a

proteção contra acessos simultâneos concorrentes via anotação `@Version` — suportando o Controle de Concorrência Otimista (OCC), detalhado na Seção 5.8 — e a indexação espacial para viabilizar consultas eficientes de proximidade geográfica. A Listagem 5.5 apresenta a definição de `DonationRequestSchema`, evidenciando o mapeamento do agregado, a versão de persistência e o atributo de localização com índice esférico.

Listagem 5.5 – Mapeamento do Esquema com Concorrência Otimista e Índice Geoespacial

```
@Document(collection = "donation_requests")
public class DonationRequestSchema {

  @Id
  private String id;
  @Version
  private Long version; // Controle de Concorrencia Otimista (OCC)
  private String requesterId;
  private String organizationId;
  // ... atributos de negocio (bloodTypeNeeded, goalBloodBags, datas, active, urgency) ...

  @GeoSpatialIndexed(type = GeoSpatialIndexType.GEO_2DSPHERE)
  private double[] location; // Coordenadas [longitude, latitude] para busca espacial

  public DonationRequestSchema(DonationRequest donationRequest) {
    this.id = donationRequest.getId().getValue().toString();
    this.version = donationRequest.getVersion();
    // ... mapeamento dos atributos do agregado ...

    var address = donationRequest.getBloodCenter().getOrganization().getAddress();
    if (address != null && address.hasCoordinates()) {
      this.location = new double[] { address.getLongitude(), address.getLatitude() };
    }
  }
}
```

5.6.5.1 Consultas de Proximidade com Indexação Geoespacial

A captação eficiente de doadores demanda não apenas a verificação biológica de compatibilidade sanguínea, mas também a avaliação da viabilidade logística do deslocamento do voluntário até o hemocentro responsável pela coleta. Para permitir que o sistema filtre rapidamente demandas situadas dentro do raio de interesse do doador, sem incorrer na sobrecarga de calcular distâncias em memória sobre toda a base de dados, a arquitetura delega o processamento espacial ao próprio motor de banco de dados.

Isso é feito por meio dos índices geoespaciais nativos do MongoDB (do tipo `2dsphere`), que permitem mapear coordenadas geográficas diretamente nos documentos armazenados. Na aplicação, as coordenadas associadas à solicitação são indexadas para facilitar a busca.

Assim, a camada de repositório consegue executar consultas geográficas de alta performance. Com o auxílio do Spring Data, a aplicação traduz a busca por proximidade para operadores nativos do banco (como o `$near`). O banco de dados realiza, então, a triagem

espacial e biológica em uma única operação atômica e indexada, retornando à aplicação apenas as solicitações geograficamente próximas e tipologicamente compatíveis, otimizando drasticamente o tempo de resposta e o consumo de recursos.

5.6.5.2 Composição versus Herança na Arquitetura do Repositório

Um ponto central no desenho da persistência é a decisão entre herança e composição na implementação dos repositórios:

- **Princípio de Inversão de Dependência (DIP) e Arquitetura Hexagonal:** O núcleo de domínio deve ser estritamente agnóstico a ferramentas e tecnologias de banco de dados. A interface `DonationRequestRepositoryInterface` reside puramente no Domínio e expressa um contrato orientado à coleção concebida pela linguagem ubíqua (`save`, `findById`, `findActiveRequestsForDonor`), sem qualquer dependência ou anotação do Spring Data (`org.springframework.data...`).

- **O Problema da Herança Direta:** Caso a interface de repositório de domínio utilizasse herança estendendo diretamente as interfaces do Spring Data (por exemplo, `interfaceDonationRequestRepositoryInterfaceextendsMongoRepository<...>`), as abstrações do framework de persistência vazariam para o núcleo da aplicação. Isso tornaria o domínio refém do Spring Data e das convenções de consulta específicas do MongoDB.

- **A Escolha por Composição (*Adapter Pattern*):** Aplicando a máxima clássica do GoF (*"Favor object composition over class inheritance"*), a classe concreta `DonationRequestRepositoryImpl` implementa a interface do domínio e **compõe** internamente tanto a interface do Spring Data (`DonationRequestMongoRepository`) quanto o carregador de entidades (`PersistenceGraphLoader`). Essa abordagem viabiliza:

  1. O isolamento completo das regras de negócio contra modificações na tecnologia de persistência;
  2. A conversão bidirecional controlada entre os agregados de domínio e os esquemas de documentos (`DonationRequestSchema`);
  3. A interceptação e tratamento centralizado de conflitos de concorrência através do componente utilitário `OptimisticConcurrency`;
  4. A alta testabilidade da aplicação por meio de testes unitários rápidos utilizando dublês de teste (*mocks*) da interface pura do domínio.

A Listagem 5.6 apresenta o contrato puro da interface de domínio, enquanto a Listagem 5.7 detalha a classe de implementação concreta com composição, demonstrando a operação de escrita com barreira de concorrência otimista e as operações de consulta e reconstituição.

Listagem 5.6 – Interface de Domínio do Repositório (Porta)

```
1  package bloodmatch.request.domain;
2
3  import bloodmatch.shared.domain.valueObjects.Address;
4  import bloodmatch.shared.domain.valueObjects.BloodType;
5  import bloodmatch.shared.domain.valueObjects.DomainID;
6  import java.time.LocalDate;
7  import java.util.List;
8  import java.util.Optional;
9
10 public interface DonationRequestRepositoryInterface {
11
12   void save(DonationRequest request);
13
14   Optional<DonationRequest> findById(DomainID id);
15
16   List<DonationRequest> findActiveRequests();
17
18   List<DonationRequest> findActiveRequestsForDonor(
19       BloodType donorBloodType,
20       Address donorAddress,
21       double maxDistanceInKm,
22       LocalDate currentDate);
23
24   List<DonationRequest> findByRequesterPartyId(DomainID requesterPartyId);
25
26   void deleteById(DomainID id);
27 }
```

Listagem 5.7 – Implementação do Repositório via Composição e Controle de Concorrência Otimista

```
1  @Repository
2  public class DonationRequestRepositoryImpl implements DonationRequestRepositoryInterface {
3
4    private final DonationRequestMongoRepository mongoRepository; // Composicao
5    private final PersistenceGraphLoader graphLoader;              // Composicao
6
7    public DonationRequestRepositoryImpl(
8        DonationRequestMongoRepository mongoRepository,
9        PersistenceGraphLoader graphLoader) {
10     this.mongoRepository = mongoRepository;
11     this.graphLoader = graphLoader;
12   }
13
14   @Override
15   public void save(DonationRequest request) {
16     DonationRequestSchema schema = new DonationRequestSchema(request);
17     OptimisticConcurrency.save(
18         () -> mongoRepository.save(schema),
19         "Donation request");
20   }
21
22   @Override
23   public List<DonationRequest> findActiveRequestsForDonor(
```

```
24        BloodType donorBloodType, Address donorAddress,
25        double maxDistanceInKm, LocalDate currentDate) {
26      if (donorAddress == null || !donorAddress.hasCoordinates())
27        return List.of();
28
29      Point donorLocation = new Point(donorAddress.getLongitude(), donorAddress.getLatitude())
        ;
30      Distance maxDistance = new Distance(maxDistanceInKm, Metrics.KILOMETERS);
31
32      List<DonationRequestSchema> schemas = mongoRepository
33          .findByActiveTrueAndDateLimitGreaterThanEqualAndBloodTypeNeededInAndLocationNear(
34              currentDate, donorBloodType.getCompatibleRecipientTypes(),
35              donorLocation, maxDistance);
36
37      return graphLoader.toDonationRequests(schemas);
38    }
39
40    // ... operacoes complementares findById, findByRequesterPartyId e deleteById delegadas
        via composicao
41  }
```

5.6.6 Segurança e autenticação

O controle de acesso é implementado por meio de autenticação baseada em JWT. O endpoint público de login (`/auth/login`) valida as credenciais do usuário e emite um *token* assinado, que passa a ser exigido nas demais chamadas autenticadas por meio de um filtro (`JwtAuthenticationFilter`) na camada `infra/security`. Esse filtro intercepta as requisições antes de chegarem aos *controllers*, valida o *token* e popula o contexto de segurança com a identidade do participante autenticado, permitindo que os casos de uso apliquem regras de autorização — como a verificação de que apenas o próprio requisitante pode gerenciar ou notificar doadores sobre uma solicitação de sua autoria.

5.6.7 Serviços externos e documentação da API

A notificação de doadores é abstraída por uma interface de domínio (`NotificationServiceInterface`), permitindo múltiplas implementações na camada de infraestrutura — como uma implementação por e-mail e uma implementação simplificada para ambiente de desenvolvimento — sem que o caso de uso que dispara a notificação precise conhecer o mecanismo concreto de envio. Essa inversão de dependência, alinhada aos princípios SOLID discutidos no Capítulo 2, evita que o domínio fique acoplado a bibliotecas de terceiros.

Para a documentação e a exploração interativa dos recursos da API, foi integrada a biblioteca *springdoc-openapi*, que gera automaticamente a especificação OpenAPI a partir dos *controllers* e disponibiliza uma interface de teste dos *endpoints*. Essa documentação viva reduz o custo de manutenção da documentação técnica e facilita a validação manual dos fluxos descritos neste capítulo.

5.7 Recomendação de solicitações e cumprimento de metas

Esta seção detalha o mecanismo central da API: a recomendação de solicitações compatíveis a doadores, a notificação de doadores potenciais e o acompanhamento do cumprimento de metas de cada solicitação. O cumprimento de metas não é um contador incrementado a cada doação, tampouco um campo persistido: é o resultado de um algoritmo de alocação FIFO calculado em memória a partir de um *snapshot* do banco no momento da leitura.

5.7.1 Recomendação de solicitações para o doador

O caminho de leitura é implementado pelo `GetRecommendedRequestsUseCase`, acionado pelo *endpoint* `GET /donation-requests/recommendations`. Dado um doador autenticado, o caso de uso primeiro verifica sua elegibilidade para doar na data corrente por meio do método `Donor.isEligibleToDonate(...)`, considerando faixa etária válida e intervalo mínimo desde a última doação. Em seguida, busca no repositório as solicitações ativas compatíveis com o tipo sanguíneo do doador (avaliadas segundo as regras de compatibilidade de `BloodType.canDonateTo`), dentro da distância máxima de recomendação configurada por ele (RF21) e ainda não expiradas, aproveitando o índice geoespacial `2dsphere` configurado no esquema de persistência e detalhado na Seção 5.6.5.1 e na Listagem 5.7.

As solicitações candidatas cuja meta já foi atingida no *snapshot* de preenchimento são descartadas, e as demais são ordenadas por proximidade geográfica, urgência e proximidade da data-limite, de modo que o doador visualize primeiro as solicitações mais próximas, mais urgentes e mais próximas de expirar. O resultado retornado inclui, para cada solicitação recomendada, a distância estimada, a urgência, a meta de bolsas e o total já cumprido — este último obtido do `DonationRequestFulfillmentService` no momento da leitura, sem campo persistido.

A Listagem 5.8 apresenta a implementação completa do caso de uso, ilustrando a recepção do `Input`, a verificação de elegibilidade do doador, a consulta espacial ao repositório, o cálculo de preenchimento via `fulfillmentService.fill` e a ordenação multi-critério em memória.

Listagem 5.8 – Caso de Uso de Recomendação de Solicitações ao Doador

```
1  @Service
2  public class GetRecommendedRequestsUseCase {
3
4    private final DonorRepositoryInterface donorRepository;
5    private final DonationRequestRepositoryInterface donationRequestRepository;
6    private final DonationRequestFulfillmentService fulfillmentService;
7
8    public GetRecommendedRequestsUseCase(
9        DonorRepositoryInterface donorRepository,
10       DonationRequestRepositoryInterface donationRequestRepository,
11       DonationRequestFulfillmentService fulfillmentService) {
12     this.donorRepository = donorRepository;
13     this.donationRequestRepository = donationRequestRepository;
14     this.fulfillmentService = fulfillmentService;
15   }
```

```
16
17   public List<OutputItem> execute(Input input, LocalDate currentDate) {
18     DomainID personId = DomainIdParser.parse(input.personId(), "personId");
19     Donor donor = donorRepository.findByPartyId(personId)
20         .orElseThrow(() -> new NotFoundException("Donor role not found"));
21
22     if (!input.includeNonEligible() && !donor.isEligibleToDonate(currentDate)) {
23       return List.of();
24     }
25
26     // Consulta espacial com indice 2dsphere filtrando raio e compatibilidade
27     List<DonationRequest> candidateRequests = donationRequestRepository.
       findActiveRequestsForDonor(
28         donor.getBloodType(), donor.getPerson().getAddress(),
29         donor.getMaxRecommendationDistanceKm(), currentDate);
30
31     // Snapshot em memoria de preenchimento de metas sob demanda
32     Map<DomainID, DonationRequestFulfillmentStatusRecord> snapshot =
33         fulfillmentService.fill(candidateRequests, currentDate);
34
35     // Descarte de metas atingidas e ordenacao multi-criterio
36     return candidateRequests.stream()
37         .filter(request -> !statusOf(snapshot, request).goalReached())
38         .sorted(Comparator
39             .comparing((DonationRequest request) -> distanceKm(donor, request),
40                 Comparator.nullsLast(Comparator.naturalOrder()))
41             .thenComparing(DonationRequest::getUrgency, Comparator.reverseOrder())
42             .thenComparing(DonationRequest::getDateLimit))
43         .map(request -> toOutput(request, donor, snapshot))
44         .toList();
45   }
46
47   // ... metodos auxiliares toOutput, distanceKm e declaracao dos records Input e OutputItem
48 }
```

5.7.2 Notificação de doadores potenciais

Complementarmente à recomendação, o requisitante pode disparar notificações ativas por meio do *endpoint* `POST /donation-requests/{id}/notify`, implementado pelo `NotifyPotentialDonorsUseCase`. Esse caso de uso valida que o solicitante autenticado é o dono da solicitação, que a solicitação ainda não expirou e que sua meta ainda não foi atingida; em seguida, delega ao `FindEligibleDonorsUseCase` (por meio do serviço de domínio `DonorMatchingService`) a identificação dos doadores elegíveis — considerando compatibilidade sanguínea, elegibilidade temporal e proximidade geográfica — para, então, notificar cada doador elegível com conta ativa através do serviço de notificação descrito na Seção 5.5.3. Esse fluxo materializa, na arquitetura da API, o comportamento observado na literatura (Capítulo 3) de notificar diretamente candidatos compatíveis em vez de aguardar passivamente novas doações.

A Listagem 5.9 detalha a implementação de `NotifyPotentialDonorsUseCase`, ressaltando a verificação de propriedade via `PartyOwnership`, o bloqueio de envio caso a meta já

tenha sido cumprida no *snapshot* de preenchimento e o despacho de notificações aos usuários elegíveis.

Listagem 5.9 – Caso de Uso de Notificação Ativa de Doadores Potenciais

```
@Service
public class NotifyPotentialDonorsUseCase {

  // Dependencias: repositorios, matchingService, notificationService e fulfillmentService
  public NotifyPotentialDonorsUseCase(...) { /* injecao via construtor */ }

  public Output execute(Input input, LocalDate currentDate) {
    DomainID requestId = DomainIdParser.parse(input.requestId(), "requestId");
    DonationRequest request = requestRepository.findById(requestId)
        .orElseThrow(() -> new NotFoundException("Donation request not found"));

    // Validacao de propriedade e expiracao
    PartyOwnership.requireSameParty(request.getRequester().getParty().getId(), input.
    actorPartyId());
    if (currentDate.isAfter(request.getDateLimit())) {
      throw new ValidationException("Cannot notify donors. The donation request has already
    expired.");
    }

    // Bloqueio de notificacao caso a meta ja tenha sido alcancada no snapshot
    DonationRequestFulfillmentStatusRecord status = fulfillmentService.fill(request.
    getBloodCenter(), currentDate)
        .getOrDefault(request.getId(), new DonationRequestFulfillmentStatusRecord(0, false))
    ;
    if (status.goalReached()) {
      throw new ValidationException("Cannot notify donors. The goal for this request has
    already been reached.");
    }

    // Identificacao de doadores compativeis e despacho de notificacoes ativas
    List<Donor> eligibleDonors = matchingService.findEligibleDonors(request, donorRepository.
    findAll(), currentDate);
    for (Donor donor : eligibleDonors) {
      userAccountRepository.findByPartyId(donor.getPerson().getId())
          .filter(UserAccount::isEnabled)
          .ifPresent(account -> notificationService.notifyDonorAboutRequest(donor, account,
    request));
    }

    return Output.success();
  }

  public record Input(String requestId, String actorPartyId) {}
  public record Output(String message) { /* ... */ }
}
```

5.7.3 Snapshot do cumprimento de metas

Não existe vínculo direto entre uma doação e uma solicitação específica: cada doação concluída em um hemocentro entra em um agrupamento (*pool*) de doações daquela organização e é distribuída entre as solicitações ativas do mesmo hemocentro pelo `DonationRequestFulfillmentService` (Listagem 5.10). Os casos de uso de leitura invocam `fill` com a data de referência `asOfDate` — em geral a data corrente. O serviço recarrega as solicitações ativas e ainda não expiradas dos hemocentros envolvidos (`findActiveRequestsByOrganizationIds`), de modo que a alocação considere o pool inteiro do centro, e não apenas o recorte da recomendação ou da listagem do requisitante. As doações concluídas são lidas na janela `[data da solicitação ativa mais antiga, asOfDate]`. A alocação ocorre em memória e isolada por hemocentro, com solicitações ordenadas da mais antiga para a mais nova e doações ordenadas pela data efetiva.

A política de distribuição combina a ordem *First In, First Out* (FIFO) com um teto proporcional ao prazo restante de cada solicitação. O teto, calculado pelo método `DonationRequest.proportionalGoalAt`, libera a meta no mesmo ritmo em que a janela `[dateRequested,dateLimit]` é consumida: decorridos 3 de 10 dias, apenas $3/10$ da meta está disponível na data de referência. A distribuição ocorre, então, em duas passagens sobre o mesmo pool. Na primeira passagem, cada doação (com saldo inicial unitário contínuo de 1,0 bolsa) é distribuída fracionariamente entre as solicitações compatíveis (`acceptsDonation`) até o respectivo teto proporcional; atingido o teto de um pedido, o saldo remanescente da doação avança na fila FIFO para as próximas solicitações compatíveis. Na segunda passagem, as frações de doações que nenhuma solicitação pôde absorver retornam na mesma ordem FIFO, agora alocadas até a meta cheia de cada solicitação.

O propósito do teto é redistribuir a escassez em favor de quem tem menos tempo. Sem ele, a solicitação mais antiga do hemocentro consome o pool integralmente, ainda que disponha de semanas de prazo, enquanto uma solicitação prestes a expirar permanece zerada. Como a segunda passagem devolve as sobras, a política não reduz o total de bolsas alocadas — altera somente qual solicitação recebe cada bolsa quando o pool é insuficiente; havendo doações em quantidade suficiente para todas as metas, o resultado coincide com o da alocação FIFO simples.

Para ilustrar o funcionamento prático dessa estratégia sob uma ótica estritamente matemática contínua, a Figura 15 esquematiza um cenário hipotético em um mesmo hemocentro, avaliado na data de referência (`asOfDate`) de 24/07/2026.

Figura 15 – Cenário ilustrativo de alocação contínua restrita por teto proporcional



Fonte: Elaborado pelos próprios autores (2026).

No cenário apresentado, duas solicitações ativas concorrem pelo mesmo *pool* de doações do hemocentro:

- **Solicitação R_1 (mais antiga na FIFO):** registrada em 14/07 com data-limite em 03/08 (janela de 20 dias) e meta cheia de 1,0 bolsa. Em 24/07, decorreram 10 dias da janela (50% do prazo), resultando em um teto proporcional calculado de $1{,}0 \times 0{,}5 = \mathbf{0{,}5000}$ bolsa;
- **Solicitação R_2 (mais recente na FIFO):** registrada em 21/07 com data-limite em 31/07 (janela de 10 dias) e meta cheia de 2,0 bolsas. Em 24/07, decorreram 3 dias da janela (30% do prazo), resultando em um teto proporcional calculado de $2{,}0 \times 0{,}3 = \mathbf{0{,}6000}$ bolsa.

O hemocentro dispõe de duas doações concluídas e compatíveis em seu *pool*: D_1 (efetivada em 22/07) e D_2 (efetivada em 23/07), cada qual com saldo contínuo unitário de 1,0 bolsa, totalizando 2,0 bolsas a distribuir.

Por se tratar de um cálculo em memória avaliado sob demanda pelo `DonationRequestFulfillmentService`, o modelo adota métrica contínua em `BigDecimal`. Tanto os tetos quanto os quantitativos alocados (`fulfilledBloodBags`) operam com frações exatas de bolsas, eliminando distorções de arredondamento discreto.

A distribuição do *pool* ocorre sequencialmente em duas passagens:

1ª Passagem: Distribuição limitada ao teto proporcional (equidade temporal)
As doações fluem pela ordem FIFO das solicitações, com absorção limitada estritamente aos tetos calculados para a data de referência:

- **Doação D_1 (1,0 bolsa):** a fila inicia pela solicitação mais antiga (R_1). Como seu teto proporcional é de 0,5000 bolsa, R_1 absorve 0,5 bolsa e atinge seu limite da primeira fase. O saldo remanescente de 0,5 bolsa avança na FIFO para R_2 (cujo teto de 0,6000 comporta a absorção), esgotando integralmente D_1;
- **Doação D_2 (1,0 bolsa):** R_1 já atingiu seu teto proporcional, tendo sua absorção temporariamente represada. A busca avança para R_2, que necessita de 0,1 bolsa para atingir seu teto (0,6000) e absorve essa fração. O saldo não absorvido de D_2 (0,9 bolsa) não encontra mais capacidades sob teto na fila e é encaminhado à segunda passagem.

Ao término da primeira passagem, foram alocadas 0,5000 bolsa para R_1 e 0,6000 bolsa para R_2, restando uma sobra de 0,9000 bolsa.

2ª Passagem: Distribuição das sobras até a meta cheia (FIFO irrestrita)
Com os tetos temporais suspensos, a sobra de 0,9 bolsa retorna ao início da fila FIFO visando suprir as metas nominais completas:

- R_1 necessita de 0,5 bolsa para alcançar sua meta total de 1,0 bolsa. Absorve 0,5 bolsa da sobra e conclui integralmente seu pedido (1,0/1,0 bolsa, `goalReached=true`);
- A fração restante da sobra (0,4 bolsa) avança para R_2, elevando seu quantitativo final para 1,0 bolsa (50% da meta cheia de 2,0 bolsas solicitadas).

Ao término do processo, todo o *pool* é aproveitado com zero desperdício: R_1 atinge 100% da meta (1,0/1,0 bolsa) e R_2 alcança 50% (1,0/2,0 bolsas). O benefício da estratégia reflete-se na garantia de atendimento proporcional: caso houvesse apenas a doação D_1 de 1,0 bolsa no hemocentro, ela seria partilhada em 0,5 bolsa para R_1 e 0,5 bolsa para R_2, impedindo que a solicitação mais antiga monopolizasse a oferta enquanto o pedido mais recente permanecesse totalmente desassistido.

O progresso (`fulfilledBloodBags`) não é persistido. Os casos de uso de leitura que precisam exibir o preenchimento — listagem de solicitações do requisitante, recomendação ao doador e bloqueio de notificação quando a meta já foi atingida — invocam o mesmo serviço de domínio e recebem o mapa calculado na hora. Concluir uma doação apenas persiste a doação; a leitura seguinte já vê o *pool* atualizado. Como o cálculo não regrava solicitações, não há condição de corrida entre escritas de contador: cada leitura monta o próprio *snapshot*. Assume-se, como contrapartida, que o progresso exibido de uma solicitação de prazo folgado

pode decrescer entre duas leituras: à medida que os tetos das solicitações concorrentes crescem com o tempo, bolsas antes atribuídas a ela na segunda passagem migram para solicitações mais próximas do vencimento.

A Listagem 5.10 apresenta o algoritmo central do serviço de domínio `DonationRequestFulfillmentService`, evidenciando a distribuição do *pool* em duas passagens.

Listagem 5.10 – Serviço de Domínio para Cumprimento de Metas das Solicitações

```
@Service
public class DonationRequestFulfillmentService {

  // Dependencias e ordenadores (OLDEST_REQUEST_FIRST, OLDEST_DONATION_FIRST)
  public DonationRequestFulfillmentService(
      DonationRequestRepositoryInterface requestRepo,
      DonationRepositoryInterface donationRepo) { /* ... */ }

  public Map<DomainID, DonationRequestFulfillmentStatusRecord> fill(
      BloodCenter bloodCenter, LocalDate asOfDate) {
    // Carrega solicitacoes ativas e doacoes concluidas do hemocentro na janela temporal
    List<DonationRequest> requests = requestRepository.findActiveRequestsByOrganizationIds(
        List.of(bloodCenter.getOrganization().getId()), asOfDate);
    if (requests.isEmpty()) return Map.of();

    return allocateLoaded(requests, asOfDate);
  }

  // Algoritmo em duas passagens: 1a proporcional (equidade), 2a FIFO das sobras (meta cheia)
  private static Map<DomainID, DonationRequestFulfillmentStatusRecord>
    allocateProportionalThenFifo(
      List<DonationRequest> requests, List<Donation> donations, LocalDate asOfDate) {
    List<DonationRequest> requestsOldestFirst = sorted(requests, OLDEST_REQUEST_FIRST);
    List<Donation> donationsOldestFirst = sorted(
        donations.stream().filter(Donation::isCompleted).toList(), OLDEST_DONATION_FIRST);

    Map<DomainID, BigDecimal> bags = initBagsMap(requestsOldestFirst);
    List<DonationPortion> portions = donationsOldestFirst.stream()
        .map(d -> new DonationPortion(d, BigDecimal.ONE)).toList();

    // 1a PASSAGEM: distribuicao limitada ao teto proporcional (equidade temporal)
    List<DonationPortion> leftovers = distribute(
        requestsOldestFirst, portions, bags,
        proportionalLimits(requestsOldestFirst, asOfDate), asOfDate);

    // 2a PASSAGEM: distribuicao das sobras ate a meta cheia (FIFO irrestrita)
    distribute(
        requestsOldestFirst, leftovers, bags,
        fullGoalLimits(requestsOldestFirst), asOfDate);

    return buildSnapshot(requestsOldestFirst, bags);
  }

  private static List<DonationPortion> distribute(
      List<DonationRequest> requestsOldestFirst, List<DonationPortion> portions,
      Map<DomainID, BigDecimal> bags, Map<DomainID, BigDecimal> limits, LocalDate asOfDate)
```

```
      {
46    List<DonationPortion> unallocated = new ArrayList<>();
47    for (DonationPortion portion : portions) {
48      for (DonationRequest request : requestsOldestFirst) {
49        if (portion.getRemainingQuantity().compareTo(BigDecimal.ZERO) <= 0) break;
50
51        if (request.acceptsDonation(portion.getDonation(), asOfDate)) {
52          BigDecimal given = bags.get(request.getId());
53          BigDecimal limit = limits.get(request.getId());
54          BigDecimal capacity = limit.subtract(given);
55          if (capacity.compareTo(BigDecimal.ZERO) > 0) {
56            BigDecimal allocation = portion.getRemainingQuantity().min(capacity);
57            bags.put(request.getId(), given.add(allocation));
58            portion.deduct(allocation);
59          }
60        }
61      }
62      if (portion.getRemainingQuantity().compareTo(BigDecimal.ZERO) > 0) {
63        unallocated.add(portion);
64      }
65    }
66    return unallocated;
67  }
68
69  // ... metodos auxiliares de agrupamento, limites (proportionalLimits, fullGoalLimits) e
      DonationPortion
70 }
```

Para orquestrar o consumo desse serviço de domínio pela camada de aplicação, o caso de uso `GetDonationRequestsByPartyIdUseCase` recupera as solicitações associadas a um determinado requisitante e aciona a computação do preenchimento das metas sob demanda no momento da leitura, sem persistir contadores transitórios no banco de dados. A Listagem 5.11 detalha a implementação completa da classe, evidenciando a chamada a `fulfillmentService.fill(userRequests,currentDate)`, a ordenação cronológica das solicitações e a projeção dos dados no registro imutável de saída (`OutputItem`), o qual computa as bolsas remanescentes (`remainingBloodBags`) e sinaliza se a meta foi alcançada.

Listagem 5.11 – Caso de Uso de Consulta de Solicitações com Snapshot de Cumprimento de Metas

```
1  @Service
2  public class GetDonationRequestsByPartyIdUseCase {
3
4    private final DonationRequestRepositoryInterface donationRequestRepository;
5    private final DonationRequestFulfillmentService fulfillmentService;
6
7    public GetDonationRequestsByPartyIdUseCase(
8        DonationRequestRepositoryInterface donationRequestRepository,
9        DonationRequestFulfillmentService fulfillmentService) {
10     this.donationRequestRepository = donationRequestRepository;
11     this.fulfillmentService = fulfillmentService;
12   }
13
```

```
14   public List<OutputItem> execute(Input input, LocalDate currentDate) {
15     DomainID partyId = DomainIdParser.parse(input.partyId(), "partyId");
16     List<DonationRequest> userRequests = donationRequestRepository.findByRequesterPartyId(
       partyId);
17
18     // Calculo do snapshot de metas em memoria para as solicitacoes do participante
19     Map<DomainID, DonationRequestFulfillmentStatusRecord> snapshot =
20         fulfillmentService.fill(userRequests, currentDate);
21
22     return userRequests.stream()
23         .sorted(Comparator.comparing(DonationRequest::getDateRequested).reversed()
24             .thenComparing(request -> request.getId().getValue()))
25         .map(request -> toOutput(request, currentDate, snapshot))
26         .toList();
27   }
28
29   private OutputItem toOutput(
30       DonationRequest request, LocalDate currentDate,
31       Map<DomainID, DonationRequestFulfillmentStatusRecord> snapshot) {
32     var status = snapshot.getOrDefault(request.getId(), new
       DonationRequestFulfillmentStatusRecord(0, false));
33     BigDecimal fulfilled = status.fulfilledBloodBags();
34     BigDecimal remaining = BigDecimal.valueOf(request.getGoalBloodBags()).subtract(fulfilled
       ).max(BigDecimal.ZERO);
35
36     return new OutputItem(
37         request.getId().getValue().toString(), request.getVersion(),
38         request.getBloodTypeNeeded().getType(), request.getDateRequested(), request.
       getDateLimit(),
39         request.isActive(), request.isExpired(currentDate), request.getBloodCenter().
       getOrganization().getName(),
40         request.getUrgency().name(), request.getGoalBloodBags(), fulfilled, remaining,
       status.goalReached());
41   }
42
43   public record Input(String partyId) {}
44   public record OutputItem(...) { /* projecao imutavel dos dados da solicitacao */ }
45 }
```

5.8 Controle Otimista de Concorrência

Em sistemas Web distribuídos concebidos segundo o estilo arquitetural RESTful, as comunicações entre cliente e servidor são estritamente desprovidas de estado (*stateless*), não existindo transações de longa duração ou sessões ativas com bloqueio de recursos. No contexto da hemoterapia, as solicitações de doação de sangue configuram recursos dinâmicos e altamente concorrentes: um operador de hemocentro ou requisitante pode atualizar a meta de bolsas necessárias (`goalBloodBags`), estender a data-limite (`dateLimit`) ou ajustar a urgência enquanto, paralelamente, outros usuários ou serviços automatizados consultam ou tentam modificar a mesma entidade.

Caso duas requisições de escrita concorrentes operem sobre o mesmo registro sem um

mecanismo de coordenação transacional, instala-se o problema clássico da **perda de atualização** (*lost update problem*). Nessa situação, a requisição que finalizar a gravação por último sobrescreverá cegamente o estado armazenado no banco de dados, eliminando as alterações consolidadas pela requisição precedente sem que qualquer um dos atores seja notificado da anomalia.

A adoção de bloqueios pessimistas (*pessimistic locking*) — mediante a retenção de travas exclusivas nas linhas ou documentos durante toda a jornada de edição do usuário — é incompatível com a natureza desconectada da Web. Esse bloqueio gera contenção severa, degrada a vazão (*throughput*) do servidor, eleva o tempo de resposta e introduz risco crítico de impasses (*deadlocks*) enquanto o usuário interage com a interface visual.

Para solucionar esse desafio de forma escalável, o sistema implementa formalmente o **Controle de Concorrência Otimista** (*Optimistic Concurrency Control* — OCC), fundamentado nos princípios seminais de Kung e Robinson (1981) e nos padrões de arquitetura corporativa formulados por Fowler (2002). O OCC parte da premissa de que os conflitos entre transações são exceções: as operações de leitura e computação em memória transcorrem de modo livre e não bloqueante, concentrando a checagem de consistência estritamente no momento da persistência.

5.8.1 Caso de uso de atualização de solicitação

Para materializar essa estratégia, o sistema disponibiliza o caso de uso de edição `UpdateDonationRequestUseCase`, acionado pela rota HTTP `PATCH /donation-requests/{id}`. A Listagem 5.12 detalha a implementação completa da classe.

Listagem 5.12 – Caso de Uso de Atualização com Verificação Precoce de Concorrência

```
1  @Service
2  public class UpdateDonationRequestUseCase {
3
4    private final DonationRequestRepositoryInterface donationRequestRepository;
5
6    public UpdateDonationRequestUseCase(DonationRequestRepositoryInterface repo) {
7      this.donationRequestRepository = repo;
8    }
9
10   @Transactional
11   public Output execute(Input input) {
12     DomainID requestId = DomainIdParser.parse(input.requestId(), "requestId");
13     DonationRequest request = donationRequestRepository.findById(requestId)
14         .orElseThrow(() -> new NotFoundException("Donation request not found"));
15
16     // 1o MOMENTO DE CHECAGEM: Nivel de Aplicacao (Pre-condition Fail-Fast)
17     if (!Objects.equals(request.getVersion(), input.version())) {
18       throw new ConcurrencyException(
19           "Resource version conflict: expected " + input.version() + " but found " + request.
       getVersion());
20     }
21
```

```
22      PartyOwnership.requireSameParty(request.getRequester().getParty().getId(), input.
        actorPartyId());
23
24      if (input.goalBloodBags() != null) request.setGoalBloodBags(input.goalBloodBags());
25      if (input.dateLimit() != null) request.setDateLimit(input.dateLimit());
26
27      // 2o MOMENTO DE CHECAGEM: Nivel de Persistencia / Banco de Dados
28      donationRequestRepository.save(request);
29      return Output.from(request);
30    }
31
32    public record Input(
33        String requestId, Long version, Integer goalBloodBags,
34        LocalDate dateLimit, String actorPartyId) {}
35
36    public record Output(String id, Long version, Integer goalBloodBags, LocalDate dateLimit)
        { /* ... */ }
37  }
```

5.8.2 Os dois momentos de checagem de versão

A garantia de integridade transacional na API estrutura-se sobre uma defesa em profundidade dividida em **dois momentos complementares de checagem de versão**:

1. **Primeiro Momento: Validação Lógica na Camada de Aplicação (*Fail-Fast Check*):** O primeiro momento ocorre explicitamente no corpo do caso de uso (na validação de versão da Listagem 5.12), imediatamente após a recuperação do agregado de domínio do repositório (`findById`). Quando o cliente consome a API (por exemplo, via consulta prévia `GET /donation-requests/{id}`), a resposta serializa o número de versão persistido (`version`). Ao emitir a instrução de alteração, o cliente é obrigado a devolver esse valor no corpo da requisição (`Input.version`).

   O caso de uso compara a versão enviada pelo cliente com a versão atual carregada do banco de dados:

   ```
   1  if (!Objects.equals(request.getVersion(), input.version())) {
   2    throw new ConcurrencyException("Resource version conflict: expected " + input.version()
          + " but found " + request.getVersion());
   3  }
   4
   ```

   Caso haja divergência (por exemplo, o cliente submete `version = 1`, porém outro usuário já havia atualizado o recurso e gravado `version = 2`), a operação é abortada instantaneamente, disparando uma `ConcurrencyException`. Essa barreira lógica antecipada proporciona benefícios cruciais:

   - ***Fail-Fast* de Domínio:** evita o processamento computacional inútil de regras de negócio, verificações de propriedade (`PartyOwnership`) e validações sobre dados sabida-

mente defasados;

- **Preservação do Agregado em Memória:** nenhum método mutador de estado (`setGoalBloodBags`, `setDateLimit`) chega a ser invocado se a versão recebida for anacrônica;
- **Resposta Semântica Padronizada:** a exceção é capturada na camada de apresentação pelo manipulador global `RestExceptionHandler`, que devolve ao cliente a resposta HTTP `409Conflict`, informando que o registro foi alterado concorrentemente e orientando o recarregamento dos dados.

2. **Segundo Momento: Validação Atômica na Camada de Persistência (*Atomic Write OCC*):** Embora o primeiro momento proteja com eficácia contra submissões de dados desatualizados, ele é inerentemente incapaz de sanar condições de corrida estritas (*race conditions*), nas quais duas requisições paralelas chegam ao servidor de forma quase simultânea, ambas lendo a mesma versão válida (por exemplo, `version = 1`). Nesse caso, ambas as requisições ultrapassariam com sucesso a checagem da camada de aplicação.

   Para neutralizar essa vulnerabilidade física, o segundo momento atua diretamente na instrução de escrita no banco de dados. No esquema documental `DonationRequestSchema` (Listagem 5.5), o atributo `version` é decorado com a anotação `@Version` do Spring Data MongoDB. Ao disparar a operação `donationRequestRepository.save(request)`, o motor do Spring Data traduz a atualização para uma instrução atômica condicional no MongoDB:

```
db.donation_requests.updateOne(
  { "_id": "req-123", "version": 1 },
  {
    "$set": { "goalBloodBags": 15, "dateLimit": ISODate("2026-10-01") },
    "$inc": { "version": 1 }
  }
)

```

   A primeira requisição a atingir o MongoDB atualiza o documento e incrementa atômica e irreversivelmente o contador de versão para `2`. No instante seguinte, quando a segunda requisição concorrente executa o seu comando buscando pelo filtro `"version": 1`, a consulta não encontra nenhum registro correspondente. Zero documentos são afetados, e o Spring Data detecta o conflito físico, disparando a exceção `OptimisticLockingFailureException`.

   Para garantir o desacoplamento arquitetural e impedir que tipos de exceção do ecossistema Spring Data vazem para as camadas superiores, o adaptador de persistência utiliza a classe utilitária `OptimisticConcurrency` (Listagem 5.13). Esse utilitário intercepta a falha e a converte de forma transparente para a exceção padrão da aplicação (`ConcurrencyException`):

Listagem 5.13 – Isolamento de Concorrência Otimista na Infraestrutura

```
1  package bloodmatch.shared.infrastructure.persistence.repository;
2
3  import bloodmatch.shared.application.exception.ConcurrencyException;
4  import org.springframework.dao.OptimisticLockingFailureException;
5
6  public final class OptimisticConcurrency {
7
8    private OptimisticConcurrency() {}
9
10   public static void save(Runnable action, String entityName) {
11     try {
12       action.run();
13     } catch (OptimisticLockingFailureException e) {
14       throw new ConcurrencyException(
15           entityName + " was changed by another operation. Reload it and try again.", e);
16     }
17   }
18 }
```

Essa abordagem em dois momentos conjuga a agilidade do *fail-fast* preventivo na aplicação com a garantia matemática de isolamento transacional atômico no motor do banco de dados, consolidando uma arquitetura imune a corrupções de estado por concorrência.

6 Avaliação da proposta de solução

Este capítulo apresenta a avaliação da solução desenvolvida, tratada como passo metodológico — e não como objetivo de entrega —, conforme o Capítulo 4. A avaliação combina a consolidação do sistema entregue — descrito no Capítulo 5 — com a execução de uma suíte de testes automatizados unitários e de integração, seguida de uma análise qualitativa da arquitetura quanto à manutenibilidade.

6.1 Sistema entregue

A solução implementada materializa-se em uma RESTful API em Java 17 com Spring Boot, persistência em MongoDB e autenticação e autorização baseadas em JWT, organizada por entidades de domínio (`request`, `donation`, `party`, `role`, `auth` e `shared`) com subpacotes em camadas inspiradas no DDD — `domain`, `application`, `infrastructure` e `presentation` —, conforme detalhado no Capítulo 5. Do ponto de vista funcional, a API cobre integralmente o conjunto de requisitos dA Tabela 5. Essa cobertura inicia-se pelo cadastro estruturado de pessoas e organizações e a respectiva atribuição de papéis de doador, requisitante e hemocentro. A partir dessa base, o sistema contempla o gerenciamento completo do ciclo de vida das solicitações de doação de sangue, permitindo a abertura de demandas, o acompanhamento de seu estado, a atualização dinâmica de metas e prazos, o cancelamento justificado e o disparo de notificações a voluntários elegíveis.

O fluxo de doações é estruturado por um registro unificado, capaz de diferenciar a intenção de doar — que gera um agendamento pendente — da doação já efetivada, viabilizando ainda o reagendamento, a confirmação de conclusão e a recuperação do histórico hemoterápico individual. No núcleo de inteligência da aplicação, destaca-se o mecanismo de recomendação de solicitações compatíveis, o qual pondera tipagem sanguínea, elegibilidade temporal e proximidade geográfica. O acompanhamento dessas demandas ocorre por meio de um algoritmo de cumprimento de metas calculado por *snapshot* em memória: as doações concluídas são alocadas às solicitações do mesmo hemocentro segundo uma ordenação FIFO modulada por um teto proporcional ao prazo restante, dispensando contadores persistidos no banco de dados. Complementarmente, o sistema fornece recursos autenticados para a busca de hemocentros por nome, recuperando os identificadores necessários às operações de escrita.

A documentação interativa OpenAPI (*springdoc*) apoia a exploração e a validação manual dos *endpoints*. Para os fins desta monografia, a avaliação formal concentrou-se na RESTful API, objeto central do estudo de caso.

6.2 Plano e método de avaliação

A avaliação foi conduzida em duas frentes complementares, coerentes com o Capítulo 4.

A primeira frente, funcional, apoia-se em testes automatizados executados com Maven Surefire e JUnit 5. Os testes unitários isolam regras de domínio e casos de uso com dependências mockadas (repositórios e serviços). Os testes de integração exercitam a configuração Spring Boot e a cadeia de segurança HTTP com `MockMvc`, verificando respostas `401`/`403`/`200` conforme a presença e o papel do JWT.

A segunda frente, estrutural, avalia qualitativamente a manutenibilidade à luz dos princípios SOLID e do DDD (Capítulo 2), observando se a separação em camadas isola o domínio de detalhes de infraestrutura e se a suíte de testes cobre as regras críticas do negócio — em especial compatibilidade sanguínea, elegibilidade, recomendação e alocação de doações às metas.

O critério de sucesso funcional adotado foi a execução integral da suíte sem falhas (*failures*) nem erros (*errors*). O critério de manutenibilidade foi a evidência de que as regras centrais permanecem testáveis no domínio/aplicação, sem necessidade de acoplar cada cenário a um banco de dados real.

6.3 Organização e estratégia da suíte de testes

A suíte de testes reside em `src/test/java` e foi concebida sob os princípios da pirâmide de testes, priorizando testes unitários rápidos, determinísticos e isolados na base arquitetural, complementados por testes de integração focados na camada de segurança HTTP. Ao todo, a suíte conta com 50 classes de teste e 323 casos de teste automatizados, espelhando a modularidade dos contextos delimitados (*bounded contexts*) do DDD.

A Tabela 15 sintetiza a distribuição da suíte pelas camadas arquiteturais. Evidencia-se uma concentração preponderante na base (camadas de Domínio e Aplicação, totalizando 308 dos 323 casos), viabilizando tempos de execução da ordem de 15 segundos para a suíte integral, sem a necessidade de acoplamento a bancos de dados externos.

Tabela 15 – Composição consolidada da suíte por camada arquitetural

Camada	**Classes de teste (exemplos de foco)**	**Casos**
Domínio	Compatibilidade sanguínea; agregados de doação e solicitação; teto proporcional de metas; *matching* de doadores; alocação FIFO; invariantes de conta de usuário; objetos de valor e entidades básicas	154
Aplicação	Autenticação e confirmação de e-mail; criação, cancelamento e listagem de solicitações; recomendação; registro, reagendamento e conclusão de doações; cadastro e atualização de papéis e participantes; busca de hemocentros; elegibilidade de doadores; notificação	154
Infraestrutura / segurança	Emissão e validação de JWT; autorização HTTP com `MockMvc`	14
Smoke	Carregamento integral do contexto Spring Boot	1
Total	50 classes de teste	**323**

Fonte: Elaborado pelos próprios autores (2026), a partir da execução Maven Surefire e JaCoCo.

A distribuição da suíte pelos contextos delimitados e seus respectivos níveis de cobertura são detalhados na Tabela 16. A relação exaustiva de todas as 50 classes de teste, acompanhada dos cenários e comportamentos específicos verificados em cada uma, encontra-se documentada no Apêndice A.

Tabela 16 – Distribuição e cobertura da suíte por contexto delimitado

Contexto	Classes	Casos	Linhas	Branches	Principais comportamentos validados
`request`	12	123	99,83%	96,98%	Teto proporcional, alocação FIFO, descarte por meta atingida, geolocalização e cancelamento justificado.
`shared`	14	62	99,60%	94,59%	Matriz de compatibilidade ABO/Rh, CPF, CNPJ, políticas de intervalo de doação e objetos de valor.
`role`	9	48	100,0%	96,88%	Aptidão física e peso do doador, *matching* euclidiano, busca de hemocentros e hierarquia de papéis.
`auth`	6	45	100,0%	98,63%	Confirmação de e-mail por token, autenticação, emissão de JWT e autorização HTTP via `MockMvc`.
`donation`	5	38	100,0%	100,0%	Registro de intenção e efetivação, reagendamento, transições de estado da doação e histórico.
`party`	3	16	100,0%	82,14%	Integridade cadastral de pessoas e organizações e controle de posse (*party ownership*).
Smoke	1	1	100,0%	n/a	Inicialização correta de beans e injeção de dependências no contexto Spring Boot.
Total	**50**	**323**	**99,89%**	**95,80%**	**100% de sucesso em todos os cenários**

Fonte: Elaborado pelos próprios autores (2026), com base em relatórios do Maven Surefire e JaCoCo.

6.3.1 Verificação formal das regras críticas de negócio

Para assegurar que o sistema atenda aos requisitos funcionais com confiabilidade e sem risco de regressão, a elaboração da suíte concentrou-se na formalização de casos de teste sobre quatro eixos críticos do domínio:

6.3.1.1 Cumprimento dinâmico de metas: teto proporcional e ordenação FIFO

O mecanismo de alocação de doações às solicitações abertas constitui o algoritmo mais sofisticado do sistema, distribuído em quatro classes de teste no pacote `request`:

- O `DonationRequestFulfillmentFifoTest` (26 casos) valida a ordenação temporal das demandas de um mesmo hemocentro, a restrição de compatibilidade tipológica entre doador e solicitação, o respeito à janela entre abertura e data-limite e o esgotamento progressivo das metas;
- O `DonationRequestFulfillmentProportionalTest` (5 casos) cobre o teto proporcional: o represamento de solicitações de prazo folgado em favor daquelas prestes a vencer, a liberação total do teto no dia limite, o retorno das bolsas excedentes em segunda passagem FIFO e a invariante de conservação do total de bolsas alocadas;

- O `DonationRequestTest` (22 casos) exercita o cálculo isolado da fórmula do teto no agregado, o uso de precisão com `BigDecimal` e o contorno de janela de um único dia;
- O `DonationRequestFulfillmentServiceTest` (9 casos) orquestra a obtenção de solicitações ativas e a consolidação do *snapshot* em memória.

6.3.1.2 Compatibilidade sanguínea e invariantes biológicas

A compatibilidade transfusional é modelada no objeto de valor `BloodType` (`shared`), desacoplada de persistência ou bibliotecas de infraestrutura. A Listagem 6.1 ilustra a abordagem adotada nos testes unitários de domínio, evidenciando a verificação exaustiva da matriz ABO/Rh pelo método `canDonateTo` — atestando o comportamento do tipo O^- como doador universal, a restrição de doadores O^+ apenas para receptores Rh positivo e a recepção estrita de AB^+.

Listagem 6.1 – Teste Unitário de Compatibilidade Sanguínea e Invariantes do Objeto de Valor

```
1  package bloodmatch.shared.domain.valueObjects;
2
3  import org.junit.jupiter.api.Test;
4  import static org.junit.jupiter.api.Assertions.*;
5
6  class BloodTypeTest {
7
8    @Test
9    void shouldCreateValidBloodType() {
10     BloodType type = BloodType.of("A+");
11     assertEquals("A+", type.getType());
12   }
13
14   @Test
15   void shouldThrowExceptionForInvalidBloodType() {
16     assertThrows(IllegalArgumentException.class, () -> BloodType.of("X"));
17   }
18
19   @Test
20   void oNegativeShouldDonateToEveryone() {
21     BloodType donor = BloodType.of("O-");
22
23     assertTrue(donor.canDonateTo(BloodType.of("O-")));
24     assertTrue(donor.canDonateTo(BloodType.of("O+")));
25     assertTrue(donor.canDonateTo(BloodType.of("A+")));
26     assertTrue(donor.canDonateTo(BloodType.of("B+")));
27     assertTrue(donor.canDonateTo(BloodType.of("AB+")));
28     // ... compativel com todos os demais receptores ABO/Rh
29   }
30
31   @Test
32   void oPositiveShouldDonateOnlyToPositiveTypes() {
33     BloodType donor = BloodType.of("O+");
34
35     assertTrue(donor.canDonateTo(BloodType.of("O+")));
36     assertTrue(donor.canDonateTo(BloodType.of("A+")));
37     assertFalse(donor.canDonateTo(BloodType.of("A-")));
38   }
```

```
39
40    // ... demais casos de teste cobrindo a matriz completa ABO/Rh e receptores nulos
41 }
```

6.3.1.3 Elegibilidade temporal, sanitária e motor de recomendação

As regras regulamentares de aptidão para doação sanguínea e o motor de recomendação dirigida são verificados nas camadas de domínio e aplicação:

- **Aptidão sanitária e intervalos mínimos:** No agregado `Donor` e na política `DonationPolicy`, valida-se a restrição de peso corporal ($\geq$ 50 kg) e o intervalo temporal regulamentar entre doações consecutivas (60 dias para homens e 90 dias para mulheres). Os testes unitários comprovam que doações recentes bloqueiam a efetivação de novos agendamentos e excluem automaticamente o voluntário das listas de doadores elegíveis a notificação;
- **Recomendação ponderada de solicitações:** A classe `GetRecommendedRequestsUseCaseTest` (9 casos) e o serviço `DonorMatchingServiceTest` (3 casos) exercitam os critérios de ordenação e descarte: solicitações cuja meta de bolsas já foi atingida são sumariamente eliminadas, tipos sanguíneos incompatíveis são filtrados e as demandas restantes são ordenadas de forma decrescente combinando proximidade física (distância euclidiana entre geocoordenadas), criticidade de urgência e tempo restante até a data-limite.

6.3.1.4 Segurança, isolamento de participantes e autorização HTTP

A proteção dos recursos da API contra acessos indevidos e privilégios cruzados é formalizada em duas frentes:

- **Emissão e integridade de credenciais JWT:** A classe `JwtTokenProviderTest` valida a geração do token assinado via HMAC, a correta extração dos *claims* de identificação (`sub`), papéis atribuídos e prazos de validade, atestando a rejeição mandatória de assinaturas adulteradas ou tokens expirados;
- **Cadeia de filtros de autorização com `MockMvc`:** O teste de integração `SecurityAuthorizationIntegrationTest` (11 casos) submete requisições simuladas à cadeia completa de segurança do Spring Security. A suíte comprova que requisições anônimas a *endpoints* protegidos retornam invariavelmente `401 Unauthorized`, requisições com papéis discordantes da operação pretendida retornam `403 Forbidden`, e requisições autenticadas com os papéis devidos (`DONOR` ou `REQUESTER`) obtêm `200 OK` ou `201 Created`, garantindo o isolamento entre participantes.

6.4 Resultados da execução e análise de cobertura

A suíte completa foi executada em ambiente local via Maven (`./mvnwtest`), utilizando Java 17.0.12 e Spring Boot 3.5.11. A execução obteve o resultado sintetizado na Tabela 17.

Tabela 17 – Resultado consolidado da execução da suíte automatizada

Indicador	**Valor**
Casos de teste executados	323
Falhas (*Failures*)	0
Erros (*Errors*)	0
Casos ignorados (*Skipped*)	0
Taxa de sucesso	100%
Cobertura de instruções / linhas (JaCoCo)	99,89% (1.769 / 1.771)
Cobertura de ramificações / *branches* (JaCoCo)	95,80% (798 / 833)
Tempo total de execução (aprox.)	15,2 s
Resultado Maven	`BUILD SUCCESS`

Fonte: Elaborado pelos próprios autores (2026).

Como pode ser observado na Tabela 17, a ausência de falhas aliada à expressiva cobertura de 99,89% de linhas e 95,80% de ramificações atesta que, para o conjunto abrangente de cenários formalizados na suíte, o comportamento observado coincide rigorosamente com o comportamento esperado das regras modeladas no Capítulo 5. Em particular, os resultados sustentam a corretude da estratégia de recomendação e da alocação de `fulfilledBloodBags` por *snapshot* — FIFO restrita pelo teto proporcional —, bem como o controle de acesso baseado em JWT e a proteção dos agregados de negócio.

No âmbito da engenharia de software, a distinção entre a cobertura de linhas (99,89%) e a cobertura de ramificações (95,80%) oferece uma leitura qualitativa relevante. Enquanto a cobertura de linhas atesta que praticamente todas as instruções executáveis foram percorridas, a cobertura de ramificações (*branch coverage*) avalia se cada estrutura condicional (`if`, `switch`, operadores ternários e filtros booleanos) foi testada tanto em seu desfecho verdadeiro quanto no falso. A retenção de 95,80% reflete um nível excepcional de rigor, sendo os 4,20% residuais correspondentes exclusivamente a salvaguardas defensivas de código (como checagens nulas redundantes exigidas por contratos de persistência ou métodos gerados de comparação estrutural).

Ressalta-se que a totalidade dos testes de domínio e de aplicação não depende de uma instância MongoDB ativa: repositórios e serviços externos são substituídos por *mocks*, o que favorece o determinismo, a repetibilidade e reduz o custo de execução para meros 15,2 segundos. Os testes que sobem o contexto Spring Boot (*smoke* e autorização HTTP) validam a montagem da aplicação e a barreira de segurança sem exigir, nos cenários cobertos, a persistência real de documentos.

6.5 Avaliação de manutenibilidade

Do ponto de vista estrutural, a avaliação qualitativa confirma que a arquitetura em camadas favorece a manutenibilidade e a sustentabilidade técnica da solução (RNF04–RNF06). Essa constatação apoia-se, em primeiro lugar, no rigoroso isolamento da camada de domínio, na qual as regras de tipagem sanguínea, cálculo de elegibilidade, verificação de aceitação (`acceptsDonation`), aplicação do teto proporcional e alocação de bolsas estão estritamente encapsuladas em entidades e serviços de domínio, operando sem qualquer acoplamento a bibliotecas do Spring MVC ou do MongoDB. A introdução do algoritmo de teto proporcional exemplifica essa flexibilidade, tendo sido incorporada como regra pura do agregado e de seu serviço de domínio sem provocar alterações em casos de uso, controladores ou esquemas de persistência.

Em consonância com esse isolamento, os casos de uso assumem exclusivamente o papel de orquestradores de fluxo, o que permite que a suíte de testes exercite cenários complexos de aplicação — como a conclusão de uma doação e a subsequente visualização do progresso recalculado — sem a interferência de detalhes de serialização HTTP. Adicionalmente, a aplicação do princípio de inversão de dependência por meio de oito interfaces de repositório e da interface de notificação permite desacoplar o núcleo de qualquer tecnologia concreta de infraestrutura, viabilizando o uso intensivo de objetos simulados (*mocks*) que garantem testes determinísticos e rápidos sem a necessidade de um banco de dados ativo. Por fim, a verificação de segurança é concentrada na borda da aplicação, assegurando que o controle de acesso e autorização seja validado nos pontos de entrada sem poluir as regras de negócio de alocação e recomendação. Em conjunto, esses elementos demonstram que a manutenibilidade é amplamente favorecida pela separação de responsabilidades e pela elevada testabilidade das regras críticas.

6.6 Sugestões de melhorias

A partir dos resultados obtidos na avaliação, delineiam-se aprimoramentos pertinentes para a evolução da API e de seus mecanismos de validação. No âmbito dos testes de integração, mostra-se recomendável expandir os cenários com persistência real em contêineres gerenciados (como Testcontainers com MongoDB), assegurando a validação automatizada de aspectos de infraestrutura como o controle de concorrência otimista (`@Version`) e consultas de agregação complexas na busca de hemocentros. No nível das interfaces externas, mostra-se oportuno formalizar testes automatizados de contrato a partir da especificação OpenAPI, bem como conduzir ensaios de carga e estresse dedicados a avaliar o desempenho das consultas geoespaciais e da reconstituição de agregados sob volumes elevados de dados e requisições concorrentes.

6.7 Considerações sobre a avaliação

Os resultados obtidos permitem concluir que o sistema implementado — verificado pela ampla suíte de 323 testes automatizados com 100% de sucesso e cobertura superior a 99% — materializa o estudo de caso do Capítulo 5 e evidencia o atendimento ao objetivo O3 (recomendação de solicitações compatíveis e notificação de candidatos elegíveis, cobertos por testes de matching, recomendação e notificação indireta via casos de uso). A análise de manutenibilidade, igualmente metodológica, aponta uma arquitetura testável em camadas. A Seção 7 retoma esses achados à luz da problemática apresentada na Introdução.

7 Considerações finais

Este capítulo retoma a problemática apresentada na Introdução à luz do sistema desenvolvido (Capítulo 5) e dos resultados da avaliação (Capítulo 6), discutindo o alcance dos objetivos, as principais contribuições do trabalho e direcionamentos para pesquisas e evoluções futuras.

7.1 Alcance dos objetivos

O objetivo geral — desenvolver um sistema de informação para gerenciamento de doações de sangue, permitindo o registro de solicitações de unidade de saúde e pacientes e, posterior, captação e recomendação de potenciais doadores de sangue — foi alcançado por meio da implementação descrita no Capítulo 5. A verificação automatizada relatada no Capítulo 6 constitui o passo metodológico de avaliação da proposta, e não uma entrega autônoma.

No que tange aos objetivos específicos estabelecidos, constata-se que todos foram plenamente atendidos no escopo do trabalho. O primeiro objetivo específico (O1) concretizou-se por meio da definição da linguagem ubíqua do projeto, estritamente alinhada ao processo de doação sanguínea, culminando na modelagem da hierarquia de participantes (`Party`, `Person` e `Organization`), na definição dinâmica de seus papéis (`Donor`, `Requester` e `BloodCenter`) e na estruturação dos agregados centrais `DonationRequest` e `Donation`, detalhados no Capítulo 5.

O segundo objetivo específico (O2) foi alcançado com a condução do mapeamento sistemático da literatura apresentado no Capítulo 3, o qual possibilitou identificar o estado da arte e as lacunas nos sistemas existentes, fornecendo subsídios teóricos e práticos que nortearam as decisões de projeto para recomendação, notificação ativa e gestão de demandas.

O terceiro objetivo específico (O3) materializou-se no projeto e desenvolvimento do sistema com base nos padrões e preceitos do paradigma *Domain-Driven Design* (DDD) e dos princípios SOLID, conforme fundamentado no Capítulo 2 e evidenciado na implementação do Capítulo 5, onde a modelagem tática — agregados, entidades, objetos de valor e serviços de domínio — reflete fielmente a realidade do negócio.

O quarto objetivo específico (O4) foi atingido por meio da adoção de uma arquitetura de software que promove a clara separação das camadas do sistema (domínio, aplicação, REST, infraestrutura), minimizando o acoplamento dos componentes. Essa organização é verificável na estrutura de pacotes e na inversão de dependências descrita no Capítulo 5.

O quinto objetivo específico (O5) foi cumprido ao prover a interface de acesso ao *backend* do sistema seguindo o estilo arquitetural REST com o protocolo HTTP, disponibilizando uma RESTful API de acesso, cujos *endpoints* estão documentados e apresentados no Capítulo 5.

Por sua vez, o sexto objetivo específico (O6) materializou-se na concepção e codificação dos serviços de *matching* e recomendação geoespacial compatível e na notificação dirigida a

voluntários elegíveis — por meio de casos de uso como `GetRecommendedRequestsUseCase`, `FindEligibleDonorsUseCase`, `DonorMatchingService` e `NotifyPotentialDonorsUseCase` —, sendo a eficácia e a corretude de tais componentes formalmente corroboradas pela execução da suíte de testes automatizados apresentada no Capítulo 6. Desse modo, evidencia-se o atendimento harmônico e integral dos objetivos propostos para a pesquisa.

Um aspecto crítico que merece destaque no alcance dos objetivos foi o papel desempenhado pelos 323 testes automatizados como rede de segurança durante a evolução do código. Ao longo do desenvolvimento, a lógica de alocação de bolsas de sangue (*fulfillment*) sofreu refatorações substanciais — em especial a introdução do teto proporcional ao prazo, que alterou o comportamento do algoritmo FIFO base. Em cada uma dessas alterações, a execução imediata da suíte detectou regressões pontuais, permitindo que fossem corrigidas antes de qualquer impacto nos demais componentes. Essa dinâmica evidencia uma das principais vantagens clínicas dos testes automatizados neste domínio: a garantia de que um doador clinicamente inapto — por intervalo insuficiente entre doações ou por peso abaixo do mínimo — jamais seja aprovado pelo sistema após qualquer mudança de código, visto que esses cenários estão explicitamente formalizados na suíte e são verificados a cada execução.

7.2 Principais contribuições

Do ponto de vista acadêmico, este trabalho oferece contribuições ao sistematizar o conhecimento sobre sistemas de captação de doadores por meio de um mapeamento sistemático da literatura, ao formalizar um estudo de caso de engenharia de *software* aplicando DDD e princípios SOLID a um domínio crítico de saúde pública com restrições biológicas e temporais, e ao fornecer uma metodologia explícita de avaliação estruturada em suíte de testes automatizados que torna verificáveis as regras de negócio essenciais.

No plano tecnológico, a principal contribuição consolida-se no desenvolvimento da RESTful API proposta. Destaca-se a concepção da estratégia de recomendação personalizada que combina proximidade e critérios clínicos, bem como o algoritmo de cumprimento de metas calculado por *snapshot* em memória. Esse algoritmo introduz uma abordagem de racionamento em que a alocação FIFO de bolsas é modulada por um teto proporcional ao tempo de vigência da solicitação, repartindo a escassez em favor das demandas mais urgentes e prevenindo concorrência em contadores persistidos. A solução é complementada por um modelo seguro de autenticação e autorização via JWT com validação de posse (*ownership*) dos recursos e por uma arquitetura em camadas desacopladas que preserva a pureza do domínio em relação a protocolos de transporte e esquemas de armazenamento, servindo de base arquitetural para aplicações correlatas na área da saúde.

7.3 Limitações

Apesar dos resultados positivos, reconhecem-se limitações. A avaliação funcional privilegiou testes unitários e de integração com *mocks*, sem baterias de carga em produção. O *snapshot* de preenchimento relê o pool do hemocentro a cada consulta que exibe progresso, o que privilegia simplicidade e consistência em detrimento de um contador materializado. Como consequência do teto proporcional sobre esse recálculo, o progresso exibido de uma solicitação de prazo folgado pode decrescer entre duas leituras, quando bolsas migram para solicitações mais próximas do vencimento — comportamento correto do ponto de vista da política de racionamento, mas cuja apresentação ao requisitante merece tratamento específico na interface. Além disso, o estudo de caso não inclui avaliação formal de usabilidade com usuários finais de hemocentros.

7.4 Trabalhos futuros

Como direcionamentos para trabalhos e pesquisas futuras, vislumbra-se aprimorar o algoritmo de recomendação com a introdução de critérios heurísticos e comportamentais adicionais, tais como a taxa de resposta histórica dos voluntários e padrões sazonais de doação. Do ponto de vista arquitetural, sugere-se evoluir o particionamento do sistema mediante a extração de módulos Maven ou microsserviços independentes para os contextos delimitados do DDD, reforçando os limites transacionais hoje estabelecidos por convenção de pacotes.

Sob a perspectiva da engenharia de testes, recomenda-se a expansão da suíte por meio da adoção de contêineres gerenciados para testes com banco real, da geração automatizada de relatórios de cobertura de código e da execução sistemática de testes de carga e de contrato da API. Por fim, no plano aplicado, mostra-se fundamental conduzir estudos empíricos integrados ao cotidiano operacional de hemocentros reais, permitindo avaliar a usabilidade e quantificar o impacto efetivo da plataforma na conversão de solicitações em doações realizadas.

7.5 Considerações finais

O descompasso entre demanda e oferta de sangue, contextualizado na Introdução, motiva soluções tecnológicas que aproximem requisitantes e doadores com critérios clinicamente relevantes — tipagem, elegibilidade e proximidade —, convergindo com as metas de acesso a serviços essenciais de saúde preconizadas pelo ODS 3 da ONU. Este trabalho respondeu a essa necessidade com uma RESTful API orientada ao domínio da doação sanguínea, validada por implementação e por 323 testes automatizados sem falhas (alcançando 99,89% de cobertura de linhas e 95,80% de ramificações no núcleo do sistema).

Conclui-se que a combinação de DDD, SOLID e testes automatizados favoreceu não apenas a entrega funcional dos requisitos, mas também a confiança na corretude das regras

críticas e a perspectiva de manutenção evolutiva da solução. Destacam-se, em especial, as vantagens práticas observadas pela adoção do DDD no projeto: o isolamento estrutural das entidades de domínio em relação a detalhes de infraestrutura e *frameworks*; a facilidade alcançada para a evolução contínua do sistema, permitindo a adição e alteração de funcionalidades sem causar regressão nas regras vitais de negócio; e a comunicação fluida propiciada pela Linguagem Ubíqua, que traduziu com naturalidade os termos do contexto clínico — como bolsa, triagem, lote e doador — diretamente para o código-fonte.

Do ponto de vista da confiabilidade clínica, a suíte de testes produziu resultados quantitativos concretos que reforçam a robustez do sistema: 323 casos de teste executados em aproximadamente 15,2 segundos, com taxa de sucesso de 100%, zero falhas e zero erros. Os 154 testes de domínio — que cobrem invariantes biológicas como a matriz ABO/Rh, o intervalo mínimo entre doações e a elegibilidade por peso, além de 154 testes de aplicação e 15 de infraestrutura e segurança — constituem uma barreira automatizada que impede que qualquer alteração de código viole silenciosamente as regras sanitárias fundamentais. A importância dessa barreira é proporcional ao risco: em sistemas de saúde pública, uma regra de elegibilidade incorreta pode resultar em uma doação clinicamente inapropriada, comprometendo tanto o doador quanto o receptor. A verificação automatizada transforma esse controle de uma responsabilidade puramente humana — sujeita a falhas de atenção — em uma garantia estrutural, executada deterministicamente a cada ciclo de desenvolvimento. Espera-se que os artefatos e os resultados aqui apresentados subsidiem trabalhos futuros e inspirem aprimoramentos em sistemas de apoio à captação de doadores de sangue.

Referências

ALESSA, T. Evaluation of the Wateen App in the Blood-Donation Process in Saudi Arabia. **Journal of Blood Medicine**, v. 13, p. 181–190, 2022.

BLOCH, Joshua. **Effective Java**. 3. ed. Boston: Addison-Wesley, 2018.

BOOCH, Grady. **Object-oriented analysis and design with applications**. 3. ed. Boston: Addison-Wesley, 2007.

BRASIL. MINISTÉRIO DA SAÚDE. **Portaria de Consolidação nº 5, de 28 de setembro de 2017. Consolidação das normas sobre as ações e os serviços de saúde do Sistema Único de Saúde**. Brasília: [*s. n.*], 2017. Diário Oficial da União.

CEARÁ. SECRETARIA DA SAÚDE. **Hemoce registra número recorde de doações em 2024, o maior nos últimos 30 anos**. Fortaleza: [*s. n.*], jan. 2025. Online.

CEARÁ. SECRETARIA DA SAÚDE. **Homens são os que mais doam sangue no Ceará entre os 2% da população doadora**. Fortaleza: [*s. n.*], ago. 2021. Online.

CENTRO DE HEMATOLOGIA E HEMOTERAPIA DO CEARÁ — HEMOCE. **Rede de atendimento**. Fortaleza: [*s. n.*], 2022. Online.

CHACON, Scott; STRAUB, Ben. **Pro Git**. [*S. l.*]: Apress, 2014.

COCKBURN, Alistair. **Writing Effective Use Cases**. [*S. l.*]: Addison-Wesley, 2001.

DA SILVA, J. R. *et al.* Blood donation support application: Contributions from experts on the tool's functionality. **Ciência & Saúde Coletiva**, v. 26, n. 2, p. 493–503, 2021.

DEITEL, Paul; DEITEL, Harvey. **Java: How to Program**. [*S. l.*]: Pearson, 2017.

ELSEVIER. **Scopus**. [S. l.]: Elsevier, 2024. Online.

ELSEVIER. **Scopus content coverage**. [*S. l.: s. n.*], 2025. Online. Mais de 2,4 bilhões de referências citadas desde 1970.

EVANS, Eric. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [*S. l.*]: Addison-Wesley, 2003.

GAMMA, Erich *et al.* **Design patterns: elements of reusable object-oriented software**. Reading: Addison-Wesley, 1994.

GARRARD, Judith. **Health Sciences Literature Review Made Easy: The Matrix Method**. 6. ed. Burlington: Jones & Bartlett Learning, 2020.

GIL, Antônio Carlos. **Métodos e técnicas de pesquisa social**. 6. ed. São Paulo: Atlas, 2008.

HEMORIO. **Doação de sangue no Brasil: desafios e perspectivas**. Rio de Janeiro, 2023.

KLEPPMANN, Martin. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol: O'Reilly Media, 2017.

LI, L. *et al.* Mobile applications for encouraging blood donation: A systematic review and case study. **Digital Health**, v. 9, 2023.

MARCONI, Marina de Andrade; LAKATOS, Eva Maria. **Fundamentos de metodologia científica**. 5. ed. São Paulo: Atlas, 2003.

MARTIN, Robert C. **Agile software development: principles, patterns, and practices**. Upper Saddle River: Prentice Hall, 2003.

MARTIN, Robert C. **Clean architecture: a craftsman's guide to software structure and design**. Boston: Prentice Hall, 2017.

MARTIN, Robert C. **Clean code: a handbook of agile software craftsmanship**. Upper Saddle River: Prentice Hall, 2008.

MARTIN, Robert C. **Design principles and design patterns**. [*S. l.*], 2000.

MINAYO, Maria Cecília de Souza. **Pesquisa social: teoria, método e criatividade**. 19. ed. Petrópolis: Vozes, 2001.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS. **Transformando nosso mundo: a Agenda 2030 para o Desenvolvimento Sustentável**. Nova York: [*s. n.*], 2015. Online.

ORGANIZAÇÃO MUNDIAL DA SAÚDE. **Blood safety and availability**. Geneva, 2020.

ORGANIZAÇÃO PAN-AMERICANA DA SAÚDE. **Supply of blood for transfusion in Latin American and Caribbean countries 2019-2020**. Washington, D.C., 2020.

PETERSEN, Kai *et al.* Systematic mapping studies in software engineering. *In:* 12TH International Conference on Evaluation and Assessment in Software Engineering (EASE). [*S. l.: s. n.*], 2008. p. 1–10.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

PRODANOV, Cleber Cristiano; FREITAS, Ernani Cesar de. **Metodologia do trabalho científico**. 2. ed. Novo Hamburgo: Feevale, 2013.

RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha. **Recommender Systems Handbook**. 2. ed. New York: Springer, 2015.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

VERNON, Vaughn. **Domain-driven design distilled**. Boston: Addison-Wesley, 2016.

VERNON, Vaughn. **Implementing Domain-Driven Design**. [*S. l.*]: Addison-Wesley, 2013.

WALLS, Craig. **Spring in Action**. [*S. l.*]: Manning, 2019.

OU-YANG, J. *et al.* Effective methods for reactivating inactive blood donors: A stratified randomised controlled study. **BMC Public Health**, v. 20, 2020.

Apêndices

APÊNDICE A – Matriz de Cobertura da Suíte de Testes Automatizados

Este apêndice apresenta a relação exaustiva das 50 classes de teste que compõem a suíte automatizada da RESTful API do *Blood Match*. Ao todo, são 323 casos de teste que protegem o núcleo de regras de domínio, a orquestração de casos de uso na camada de aplicação, os componentes de segurança e os pontos de entrada HTTP.

A Tabela 18 detalha cada classe de teste, seu respectivo contexto delimitado (*bounded context*), a camada arquitetural correspondente, os cenários e comportamentos formais verificados e o número de casos de teste implementados.

Tabela 18 – Matriz exaustiva de testes automatizados da RESTful API Blood Match

Classe de teste	**Contexto**	**Camada**	**Comportamentos verificados**	**Casos**
`BloodTypeTest`	shared	Domínio	Criação de tipo válido; normalização de caixa; rejeição de tipo nulo ou inválido; rejeição de receptor nulo; O^- como doador universal; compatibilidade completa da matriz ABO/Rh; igualdade e hash	16
`AddressTest`	shared	Domínio	Criação de endereço completo; geocodificação com coordenadas válidas; validação de campos obrigatórios e invariantes de localização	7
`CPFTest`	shared	Domínio	Validação de formato e cálculo dos dígitos verificadores de CPF; rejeição de nulos, tamanho incorreto e sequências repetidas	4
`CNPJTest`	shared	Domínio	Validação de formato e cálculo algorítmico de CNPJ; rejeição de documento inválido ou nulo	4

Tabela 18 – continuação

Classe de teste	Contexto	Camada	Comportamentos verificados	Casos
PhoneNumberTest	shared	Domínio	Formatação de número telefônico; rejeição de caracteres não numéricos e comprimento fora do padrão	4
EmailTest	shared	Domínio	Validação de formato RFC de e-mail; normalização para caracteres minúsculos; rejeição de strings malformadas	4
DomainIDTest	shared	Domínio	Geração aleatória de UUIDv4; conversão para string e validação de formato de identificador universal	4
DomainObjectTest	shared	Domínio	Atribuição de ID e controle de versão concorrente (version); rejeição de ID nulo	2
DonationPolicyTest	shared	Domínio	Políticas de doação e intervalos mínimos exigidos por gênero (60 dias homens, 90 dias mulheres)	1
DomainIdParserTest	shared	Aplicação	Conversão segura de UUID a partir de parâmetros de entrada; tratamento de erros de formatação	3
EmailConfirmationTokensTest	shared	Aplicação	Geração criptográfica de tokens e cálculo de janelas temporais de expiração	3
PartyOwnershipTest	shared	Aplicação	Verificação de posse de recurso por participante autenticado; bloqueio de acessos indevidos	3
TimeFormatsTest	shared	Aplicação	Formatação e *parsing* padronizado de horários no formato HH:mm	4
ExceptionsTest	shared	Aplicação	Hierarquia de exceções de aplicação; mensagens de erro e causas encadeadas	3

Tabela 18 – continuação

Classe de teste	Contexto	Camada	Comportamentos verificados	Casos
`UserAccountTest`	auth	Domínio	Criação de conta; fluxo de confirmação por token; bloqueio de conta inativa; expiração de token; regras de login	12
`AuthenticationServiceTest`	auth	Aplicação	Login com credenciais válidas; rejeição de senha incorreta; rejeição de conta desabilitada; geração e validação de JWT na resposta	6
`ConfirmEmailUseCaseTest`	auth	Aplicação	Confirmação bem-sucedida; token não encontrado; token expirado; reenvio antes da expiração; tratamento de exceções de domínio	6
`ResendConfirmationUseCaseTest`	auth	Aplicação	Reenvio de e-mail com novo token; rejeição de conta já confirmada; rejeição de token ainda válido; invalidação do token anterior	7
`JwtTokenProviderTest`	auth	Infraestr.	Geração de token com claims corretos (`sub`, papéis, expiração); validação de token legítimo; rejeição de assinatura adulterada	3
`SecurityAuthorizationIntegrationTest`	auth	Apresent.	Acesso anônimo retorna HTTP 401; token inválido retorna 401; papel incompatível retorna 403; papel autorizado retorna 200/2xx para *endpoints* protegidos	11
`PartyTest`	party	Domínio	Criação de pessoa física e jurídica; integridade cadastral; atualização de endereço e telefone	2

Tabela 18 – continuação

Classe de teste	Contexto	Camada	Comportamentos verificados	Casos
`RegisterPartyUseCaseTest`	party	Aplicação	Cadastro de pessoa física e jurídica; validações cadastrais completas; rejeição de e-mail duplicado	8
`UpdatePartyUseCaseTest`	party	Aplicação	Atualização cadastral de dados do participante; restrições de permissão e consistência	6
`DonorTest`	role	Domínio	Definição de *weightUpdatedAt*; atualização apenas sob variação de peso; histórico de doações; aptidão sanitária ($\geq$ 50 kg)	11
`DonorMatchingServiceTest`	role	Domínio	*Matching* de doadores compatíveis por tipo sanguíneo, elegibilidade temporal e proximidade física	3
`RoleHierarchyTest`	role	Domínio	Hierarquia de papéis (`PartyRole`); agregação com participantes e entidades base	3
`RegisterDonorUseCaseTest`	role	Aplicação	Registro de doador com dados válidos; validação de peso e tipo sanguíneo; rejeição de participante sem conta; detecção de duplicidade	8
`UpdateDonorUseCaseTest`	role	Aplicação	Atualização de peso e dados do doador; recálculo imediato de aptidão	6
`GetDonorSummaryUseCaseTest`	role	Aplicação	Retorno de resumo consolidado contendo elegibilidade e histórico do doador autenticado	4
`RegisterRequesterUseCaseTest`	role	Aplicação	Registro de requisitante com dados válidos; rejeição de participante sem conta; consistência de papéis	4

Tabela 18 – continuação

Classe de teste	Contexto	Camada	Comportamentos verificados	Casos
`RegisterBloodCenterUseCaseTest`	role	Aplicação	Cadastro de hemocentro vinculado à organização; validação de CNPJ e dados operacionais	5
`SearchBloodCentersUseCaseTest`	role	Aplicação	Busca por nome de hemocentros; delimitação de paginação; busca parcial insensível a maiúsculas; validação de argumentos	4
`DonationRequestTest`	request	Domínio	Regras do agregado; fórmula de teto proporcional; contorno de janela de um único dia; precisão via `BigDecimal`; cálculo de `acceptsDonation`; expiração	22
`DonationRequestFulfillmentFifoTest`	request	Domínio	Ordenação FIFO temporal por hemocentro; compatibilidade tipológica ABO/Rh; janela temporal válida; esgotamento de meta; isolamento entre hemocentros	26
`DonationRequestFulfillmentProportionalTest`	request	Domínio	Represamento de solicitação com prazo folgado; ausência de teto no dia da data-limite; devolução de sobras em segunda passagem FIFO; preservação do total de bolsas	5
`DonationRequestFulfillmentServiceTest`	request	Domínio	Alocação de doação compatível à solicitação; rejeição de data nula; carregamento de solicitações ativas; tratamento de listas vazias	9

Tabela 18 – continuação

Classe de teste	Contexto	Camada	Comportamentos verificados	Casos
CreateDonationRequestUseCaseTest	request	Aplicação	Criação de solicitação com dados válidos; validação de urgência e metas; geocodificação sob demanda; rejeição de entidades inexistentes	7
CancelDonationRequestUseCaseTest	request	Aplicação	Cancelamento pelo proprietário; rejeição de cancelamento por terceiro; rejeição de solicitação inexistente; registro obrigatório de justificativa	7
CloseDonationRequestUseCaseTest	request	Aplicação	Encerramento formal de solicitações de doação atendidas ou expiradas	3
UpdateDonationRequestUseCaseTest	request	Aplicação	Alteração de meta de bolsas e prorrogação de prazo; validações de datas futuras	7
GetDonationRequestsByPartyIdUseCaseTest	request	Aplicação	Listagem de pedidos por participante; enriquecimento com status de cumprimento em tempo real	7
FindEligibleDonorsUseCaseTest	request	Aplicação	Retorno de doadores elegíveis; exclusão de inaptos por intervalo sanitário ou peso insuficiente	5
GetRecommendedRequestsUseCaseTest	request	Aplicação	Elegibilidade do doador; descarte de solicitação com meta atingida; ordenação ponderada por distância, urgência e prazo; filtro de compatibilidade	9
NotifyPotentialDonorsUseCaseTest	request	Aplicação	Disparo de notificações assíncronas para candidatos elegíveis; idempotência e consistência	6

Tabela 18 – continuação

Classe de teste	Contexto	Camada	Comportamentos verificados	Casos
`DonationTest`	donation	Domínio	Registro de intenção e doação realizada; conclusão de doação pendente; reagendamento com novas datas; cancelamento; validação de concorrência	15
`CreateDonationUseCaseTest`	donation	Aplicação	Criação de registro de doação com dados válidos; validações de autorização e notificação	8
`CompletePendingDonationUseCaseTest`	donation	Aplicação	Conclusão de doação pendente pelo hemocentro; atualização imediata do histórico sanitário do doador	7
`ReschedulePendingDonationUseCaseTest`	donation	Aplicação	Reagendamento de doação pendente para nova data válida	6
`GetDonorDonationHistoryUseCaseTest`	donation	Aplicação	Consulta de histórico individual de doações realizadas pelo participante autenticado	2
`BloodmatchApplicationTests`	–	*Smoke*	Carregamento completo do contexto de inicialização Spring Boot sem erros de configuração	1
Total	**6 contextos**	**4 camadas**	**50 classes de teste**	**323**